DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

Master's degree course in Mathematics

Kolmogorov–Arnold Theorem and Its Applications to Neural Networks

Supervisor:
Ch.mo prof.
Vittorio Colao

Student:
Ugo G. S. Campolo
235681

ACADEMIC YEAR 2025/2026

Ai miei genitori,
che hanno sempre creduto in me,
anche quando io ho smesso di farlo.

A Elio,
il mio primo professore.

Introduction

The aim of this thesis is to study the Kolmogorov–Arnold theorem, its limitations, and its applications to neural networks. The structure of the thesis follows a path that begins with the statement of the Kolmogorov–Arnold theorem, providing a complete proof. Subsequently, it discusses the limitations of this theorem as discovered by Vitushkin, leading to modern applications in the field of neural networks and the numerical solution of partial differential equations (PDEs).

Each chapter takes a fundamental scientific article from the literature as its primary reference.

Chapter 1 is based on the original paper by Kolmogorov [1], in which the celebrated representation theorem is proven. In the original work, three fundamental lemmas are stated without proof. One of the main objectives of this thesis is to provide a complete and rigorous proof of these results. During this analysis, a specific problem emerged regarding the translation of the second lemma from the original Russian[1] to the English version[2]. Even when adopting the Russian formulation, it was not possible to fully reconstruct the proof of Kolmogorov's second lemma; therefore, an alternative version of the lemma 2 is proposed, which nevertheless allows for the attainment of the theorem's main result.

Chapter 2 is based on the work of Vitushkin [3], which analyzes the structural limits of the Kolmogorov–Arnold theorem when the base functions of the representation are required to be differentiable. In this context, Vitushkin demonstrates that, under such regularity assumptions, the Kolmogorov representation is no longer complete within the space of continuous functions. Unlike Kolmogorov's original approach, Vitushkin provides complete proofs for all stated results. His conclusions highlight a fundamental obstacle to the practical use of the theorem, showing how seemingly natural regularity constraints introduce substantial limitations in computational applications.

Chapter 3 is dedicated to the application of the Kolmogorov–Arnold theorem to modern neural networks, specifically Kolmogorov–Arnold Networks (KANs)[4]. The authors of these neural architectures justify their expressivity through the representation theorem; however, the theoretical limitations highlighted by Vitushkin play a central role in understanding the actual capabilities and constraints of these networks.

Chapter 4 presents a numerical application of KANs to the resolution of a partial differential equation, following the framework proposed in recent literature[5]. The results obtained show competitive performance compared to state-of-the-art Physics-Informed Neural Networks (PINNs).

Contents

Introduction **I**

1 Kolmogorov-Arnold Theorem **1**
1.1 Kolmogorov-Arnold Theorem 1
1.2 Construction of the Functions $\psi^{p,q}$ 2
1.3 Construction of the Functions χ^q and Proof of the Theorem 11

2 Vitushkin's Theorem **15**
2.1 Introduction 15
2.2 Notation and definitions 15
2.3 Estimating the difference of integrals of a superposition term along nearby level curves 18
2.4 Elimination of dependent terms 31
2.5 Reduction of linear superpositions to a form with independent terms 41
2.6 The set of linear superpositions in the space of continuous functions is closed 49
2.7 The set of linear superpositions in the space of continuous functions is nowhere dense 56

3 Kolmogorov-Arnold Networks **60**
3.1 A New Perspective on the Kolmogorov-Arnold Representation Theorem . . 60
3.2 The KAN Architecture 61
3.3 Approximation Capabilities and Scaling Laws of KANs 64

4 Dual-PINN with efficient-KAN for 1D Fokker–Planck **65**
4.1 Introduction 65
4.2 Python code 66
4.3 Imports, Logging and Device 66
4.3.1 Library Imports and KAN Module 66
4.3.2 Logging Configuration and Device Selection 67
4.4 Centralized Configuration and Fokker–Planck Model 67
4.4.1 Config Dataclass 67
4.4.2 Drift and Exact Stationary Solution 69
4.5 Pre-allocated Tensors and Loss Components 70
4.5.1 Grids for Normalization and Boundary Conditions 70
4.5.2 Fokker–Planck Residual 70
4.5.3 Boundary Loss via Augmented Lagrangian 71
4.5.4 Role Loss for the Dual Decomposition 72
4.5.5 Schedulers for Gamma and Boundary Weight 73
4.5.6 Normalization and Positivity Loss 73

4.6 Internal Sampling Strategies . . . 74
4.6.1 Uniform Sampling and Phase 1 . . . 74
4.6.2 Residual-based Adaptive Sampling (Phase 2) . . . 74
4.7 Training for a Single Seed . . . 75
4.7.1 Initialization and Optimizer . . . 75
4.7.2 Phase 0: Boundary Pre-training . . . 76
4.7.3 Phases 1 and 2: Full Dual-PINN Training . . . 77
4.7.4 ALM Updates and Logging . . . 78
4.7.5 Evaluation and Metrics for a Single Seed . . . 79
4.8 Multi-seed Loop and Statistics . . . 81
4.8.1 Multi-seed Training Loop . . . 81
4.8.2 Multi-seed Statistics . . . 82
4.9 Best Model, Plotting, and Saving Results . . . 82
4.9.1 Restoring and Evaluating the Best Model . . . 82
4.9.2 Plot of Exact vs Dual-PINN KAN Solution . . . 83
4.9.3 Saving JSON and TXT Reports . . . 84
4.10 Numerical Results: Stationary Fokker–Planck 1D (Double-Well) . . . 87
4.10.1 Experimental Setup . . . 87
4.10.2 Best Model and Qualitative Comparison . . . 88
4.10.3 Quantitative Metrics (Best Seed) . . . 89
4.10.4 Multi-seed Performance and Robustness . . . 89

Chapter 1

Kolmogorov-Arnold Theorem

Introduction

The first chapter states and proves the Kolmogorov–Arnold theorem. The discussion is based on Kolmogorov's original paper[1], in which three lemmas are stated without a complete proof; in this thesis, a detailed proof of these lemmas is provided instead. Following the structure of Kolmogorov's proof, the chapter is divided into two main parts: the first demonstrates the existence of the inner functions $\psi^{p,q}$, while the second establishes the existence of the outer functions $\chi^q(y)$, thereby completing the proof of the theorem's statement.

1.1 Kolmogorov-Arnold Theorem

Theorem 1 (Kolmogorov-Arnold Theorem)**.** *For every integer $n \geq 2$, there exist continuous real functions $\psi^{p,q}(x)$ on the closed unit interval*

$$E^1 = [0, 1]$$

such that every continuous real function

$$f(x_1, \ldots, x_n) \quad on \quad E^n = [0, 1]^n$$

can be represented as

$$f(x_1, \ldots, x_n) = \sum_{q=1}^{2n+1} \chi^q \left[\sum_{p=1}^{n} \psi^{p,q}(x_p) \right], \tag{1.1}$$

where $\chi^q(y)$ are continuous real functions.

The theorem asserts that every continuous multivariate function can be represented as a superposition of one-dimensional functions. It will be shown that the inner functions $\psi^{p,q}(x)$ are independent of f and rely solely on the problem's dimension, whereas the outer function $\chi^q(y)$ is dependent on f.

1.2 Construction of the Functions $\psi^{p,q}$

In this section, we prove the existence of the inner functions $\psi^{p,q}(x)$, which are independent of f. From now on, the indices p, q, k, i are integers with:

$$1 \leq p \leq n, \quad 1 \leq q \leq 2n+1, \quad k = 1, 2, \ldots, \quad 1 \leq i \leq m_k = (9n)^k + 1.$$

When summing or multiplying within these ranges, we do not explicitly indicate the limits.

For each **fixed** k and q, we define the closed intervals

$$A^q_{k,i} = \left[\frac{1}{(9n)^k}\left(i - 1 - \tfrac{q}{3n}\right), \frac{1}{(9n)^k}\left(i - \tfrac{1}{3n} - \tfrac{q}{3n}\right) \right], \quad i = 1, \ldots, m_k. \tag{1.2}$$

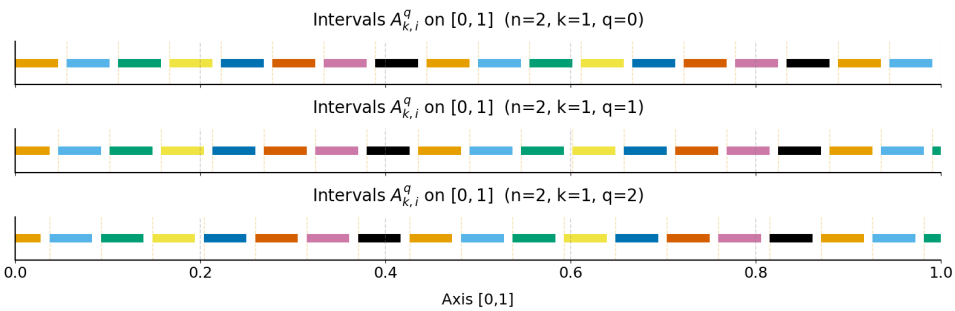
Varying i, the interval $A^q_{k,i}$ translates from left to right, leaving a gap with endpoints

$$G^q_{k,i} = \left(\frac{1}{(9n)^k}\left(i - \frac{1}{3n} - \frac{q}{3n}\right), \frac{1}{(9n)^k}\left(i - \frac{q}{3n}\right) \right) \tag{1.3}$$

Each interval $A^q_{k,i}$ has length

$$\left|A^q_{k,i}\right| = \frac{1}{(9n)^k}\left(1 - \tfrac{1}{3n}\right),$$

We observe that, as k increases, the length of the intervals $A^k_{q,i}$ decreases. When the index i varies, the intervals translate from left to right leaving a gap between consecutive elements. Finally, varying the parameter q produces a translation of the entire system of intervals.



We also define, for fixed k and q, the cubes:

$$S^q_{k,i_1\ldots i_n} = \prod_{p=1}^{n} A^q_{k,i_p} \tag{1.4}$$

To prove the main theorem and the existence of inner functions, we need three lemmas. We begin with the first one.

Lemma 1. *The system of all cubes $S^q_{k,i_1\dots i_n}$ with constant k and variable $q, i_1, \dots, i_n$ covers the unit cube E^n in such a way that every point belonging to E^n is covered at least $n+1$ times.*

Proof. The Lemma is proved in three steps

Step 1: The gaps for different q are disjoint.

For a fixed q, the intervals $A^q_{k,i}$ cover $[0,1]$ except for the gaps (open intervals)

$$G^q_{k,i}\left(\frac{1}{(9n)^k}\left(i-\frac{1}{3n}-\frac{q}{3n}\right), \frac{1}{(9n)^k}\left(i-\frac{q}{3n}\right)\right)$$

these gaps have length

$$\frac{1}{3n(9n)^k}$$

and varying i, between one gap and the next there is a distance of

$$\frac{1}{(9n)^k}\left(1-\frac{1}{3n}\right)=\frac{3n-1}{(3n)(9n)^k}$$

We want to show that for different q the gaps are disjoint. For this purpose, let q_1 and q_2 be such that $q_1 \neq q_2$, with $q_1, q_2 \in \mathbb{N}, \quad 1 \leq q_1, q_2 \leq 2n+1$. We can then note that the gaps shift by

$$\frac{1}{(9n)^k}\cdot\frac{q_2-q_1}{3n}=\frac{q_2-q_1}{3n(9n)^k}$$

and since

$$1 \leq |q_2-q_1| \leq 2n$$

there are two cases. The first is when we translate by 1 and there is no intersection between the gaps because $|q_2-q_1| \geq 1$ and each gap has width $\frac{1}{3n(9n)^k}$. The other case, where there could be a non-empty intersection between the gaps if we cumulate the translations. This is not the case either, because even if we sum all the translations, we have a gap that would translate by $\frac{2n}{3n(9n)^k}$, which is still smaller than the distance between a gap and the next one (by varying i) of $\frac{3n-1}{3n(9n)^k}$.

Step 2: Coverage of E^1

Consider the case $n=1$. The intervals $A^q_{k,i}$ for a fixed q have gaps of width $\frac{1}{3n(9n)^k}$.

When q increases by 1, the entire system of intervals translates by:

$$\frac{1}{(9n)^k}\cdot\frac{1}{3n}=\frac{1}{3n(9n)^k}$$

This translation is exactly equal to the width of the gaps. Therefore, when the system translates, the new intervals precisely fill the gaps of the previous system.

From Step 1, the gaps for different q are disjoint. With $2n+1$ translated systems, every point in $[0,1]$ that is not covered by one system will be covered by a subsequent system. Therefore, the union of all intervals $A^q_{k,i}$ for $q=1,2,\dots,2n+1$ completely covers $E^1=[0,1]$.

Step 3 each point $x_p \in [0,1]$ belongs to at least one interval $A^q_{k,i}$ for some value of q and i.

Now consider a point $x = (x_1, \ldots, x_n) \in E^n$. The point x belongs to the cube $S^q_{k,i_1 \ldots i_n}$ if and only if $x_p \in A^q_{k,i_p}$ for every $p = 1, \ldots, n$.

For each coordinate x_p, from Step 2 we know that there are values of q for which x_p is covered by some interval $A^q_{k,i}$.

Since the gaps for different values of q are disjoint (Step 1), each coordinate x_p of $x = (x_1, \ldots, x_n)$ can fall into at most one gap. Thus, in the worst case for each coordinate, we lose at most one value of q.

With n coordinates and $2n+1$ values of q, in the worst case we lose at most n values of q, leaving us with at least $(2n+1) - n = n+1$ values of q for which the point x belongs to the corresponding cube $S^q_{k,i_1 \ldots i_n}$.

Therefore, every point $x \in E^n$ is covered at least $n+1$ times.

$\square$

The second lemma concerns the existence of constants that will be used to construct $\psi^{p,q}$. This lemma differs from the original formulation. In the original paper, point 2 of Lemma 2 was, in fact, inaccurately translated from Russian [1]into English[2]. Furthermore, even when referring directly to the Russian version[1], points 2 and 3 appear difficult to achieve using the same constant ϵ_k. For this reason, we introduce a new constant, denoted by τ_k, and provide a proof for the following version of the lemma.

Lemma 2. *The constants $\lambda^{p,q}_{k,i}$, ϵ_k, τ_k can be chosen in such a way that the following conditions hold:*

1. $\lambda^{p,q}_{k,i} < \lambda^{p,q}_{k,i+1} \leq \lambda^{p,q}_{k,i} + \frac{1}{2^k}$
2. $|\lambda^{p,q}_{k+1,i'} - \lambda^{p,q}_{k,i}| \leq \tau_k - \tau_{k+1}$. *If the intervals $A^q_{k,i}$ and $A^q_{k+1,i'}$ intersect.*
3. *The intervals $\Delta^q_{k,i_1,\ldots,i_n} = \left[\sum_{p=1}^n \lambda^{p,q}_{k,i_p}, \sum_{p=1}^n \lambda^{p,q}_{k,i_p} + n\epsilon_k\right]$ are pairwise disjoint.*
4. $\epsilon_k < \tau_k \leq \frac{1}{2^k}$

Proof. We must show that such constants $\lambda^{p,q}_{k,i}$, ϵ_k and τ_k exist. **To this end, we choose $\lambda^{p,q}_{k,i}$ as any element of the interval $A^q_{k,i}$.**We prove each claim in turn.

1. We show the first point

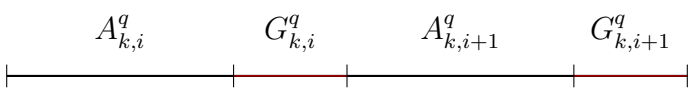


Figure 1.1: drawing not to scale

We note that with this choice the first condition is satisfied. Indeed, $\lambda^{p,q}_{k,i} \in A^q_{k,i}$ and $\lambda^{p,q}_{k,i+1} \in A^q_{k,i+1}$.

We note that with this choice we certainly have $\lambda^{p,q}_{k,i} < \lambda^{p,q}_{k,i+1}$; the inequality is strict and holds due to the presence of the gap $G^q_{k,i}$ between the intervals.

To have the other inequality, we note that:

$$|A^q_{k,i}| + |G^q_{k,i}| + |A^q_{k,i+1}| = 2\frac{1}{(9n)^k}\left(1 - \frac{1}{3n}\right) + \frac{1}{(3n)(9n)^k} = \frac{6n-1}{(3n)(9n)^k}$$

and this quantity is less than $\frac{1}{2^k}$. Thus the first point is done.

2. we show the second point

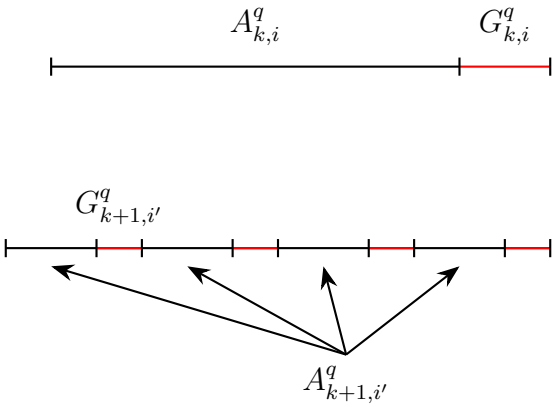


Figure 1.2: drawing not to scale

Still using this same choice for $\lambda^{p,q}_{k,i}$, **we take τ_k as twice the summation from k onwards of the length of $A^q_{k,i}$:**

$$\tau_k = 2\sum_{j=k}^{\infty} |A^q_{j,i}| = 2\sum_{j=k}^{\infty} \frac{3n-1}{3n(9n)^j} = 2 \cdot \frac{3(3n-1)}{9n-1}\frac{1}{(9n)^k}$$

With this choice we have

1) $\tau_k > 0$
2) $\tau_k > \tau_{k+1}$
3) $\tau_k - \tau_{k+1} = 2|A^q_{k,i}|$

We note that $\frac{|A^q_{k,i}|}{|A^q_{k+1,i'}|} = 9n$. That is, $A^q_{k,i}$ is $9n$ times $A^q_{k+1,i'}$. By hypothesis $A^q_{k,i}$ intersects $A^q_{k+1,i'}$, so the distance between an element of $A^q_{k,i}$ and an element of $A^q_{k+1,i'}$ does not exceed twice the length of $A^q_{k,i}$. Thus we have

$$|\lambda^{k,q}_{k+1,i'} - \lambda^{k,q}_{k,i}| \leq 2|A^q_{k,i}| = \tau_k - \tau_{k+1}$$

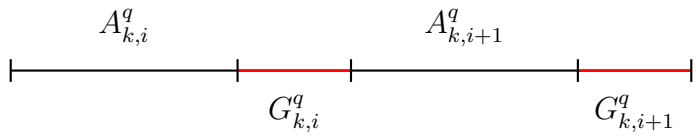
3. To show point 3, again with the choice of $\lambda^{p,q}_{k,i} \in A^q_{k,i}$, **we choose** ϵ_k

$$\epsilon_k = \frac{1}{n} \cdot \frac{1}{2}|G^q_{k,i}| = \frac{1}{n} \cdot \frac{1}{2}\left(\frac{1}{3n}\frac{1}{(9n)^k}\right)$$

It can be seen iteratively.

Case $n = 1$:

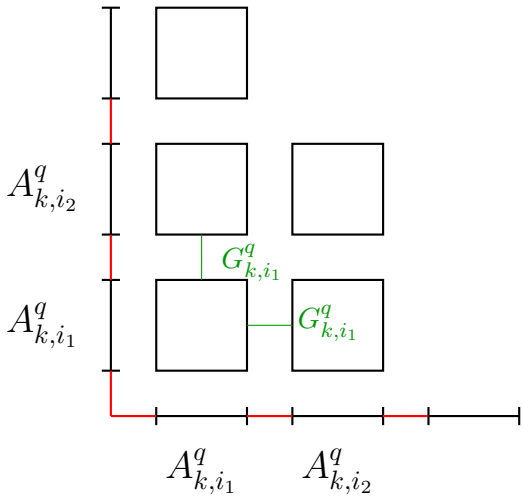
$$\Delta^q_{k,i} = [\lambda^{p,q}_{k,i}, \lambda^{p,q}_{k,i} + \epsilon_k]$$



We must show that for different i these intervals are disjoint. To be disjoint, ϵ_k must be chosen smaller than the length of the gap $G^q_{k,i}$, and indeed it is smaller.

Case $n = 2$:

$$\Delta^q_{k,i_1,i_2} = [\lambda^{p,q}_{k,i_1} + \lambda^{p,q}_{k,i_2}, \lambda^{p,q}_{k,i_1} + \lambda^{p,q}_{k,i_2} + 2\epsilon_k]$$



The sum of these constants $\lambda^{p,q}_{k,i_p}$ can be found inside the squares above. For these intervals to be disjoint, $2\epsilon_k$ must be less than the length of the gap, and indeed it is. The case $n > 2$ is analogous.

4. We have $\epsilon_k < |G^q_{k,i}|$ and $\tau_k > 2|A^q_{k,i}|$. Since $|G^q_{k,i}| < |A^q_{k,i}|$, we therefore have $\epsilon_k < \tau_k$. We also have that $\tau_k < \frac{1}{2^k}$ indeed

$$\tau_k = 2\sum_{j=k}^{\infty} \frac{3n-1}{3n(9n)^j} = 2 \cdot \frac{3(3n-1)}{9n-1}\frac{1}{(9n)^k} < \frac{1}{2^k}$$

□

Remark 1.1. We note that the constants $\lambda^{p,q}_{k,i}$ can be chosen arbitrarily, as long as they are within the interval $A^q_{k,i}$.

This lemma proves the existence of the inner functions $\psi^{p,q}(x)$, we will make use of the estimates from points $1, 2$ and 4 of Lemma 2.

Lemma 3. *For fixed p, q the conditions:*

5. $\lambda^{p,q}_{k,i} \leq \psi^{p,q}(x) \leq \lambda^{p,q}_{k,i} + \epsilon_k$

for $x \in A^q_{k,i}$ determine a unique continuous function $\psi^{p,q}$ on E^1.

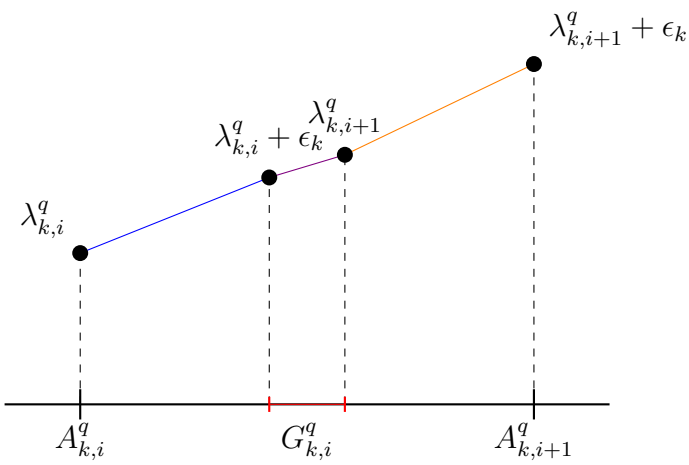


Figure 1.3: drawing not to scale of $\psi^{p,q}_k$

Proof. For convenience, we write the intervals $A^q_{k,i}$ and $G^q_{k,i}$ as:

$$A^q_{k,i} = [a^q_{k,i}, b^q_{k,i}]$$

$$G^q_{k,i} = [b^q_{k,i}, a^q_{k,i+1}]$$

We fix p, q. For each $k \geq 1$ we define $\psi^{p,q}_k : [0, 1] \to \mathbb{R}$ in this way:

On each $A^q_{k,i}$, the function is the line:

$$\psi^{p,q}_k(x) := \lambda^{p,q}_{k,i} + \epsilon_k \frac{x - a^q_{k,i}}{b^q_{k,i} - a^q_{k,i}} \quad \text{for } x \in A^q_{k,i}$$

This is the line that increases from $\lambda^{p,q}_{k,i}$ to $\lambda^{p,q}_{k,i} + \epsilon_k$.

On each $G^q_{k,i}$, the function is the line connecting the values at the endpoints:

$$\psi^{p,q}_k(x) := \frac{a^q_{k,i+1} - x}{a^q_{k,i+1} - b^q_{k,i}}(\lambda^{p,q}_{k,i} + \epsilon_k) + \frac{x - b^q_{k,i}}{a^q_{k,i+1} - b^q_{k,i}}\lambda^{p,q}_{k,i+1} \quad \text{for } x \in G^q_{k,i} = (b^q_{k,i}, a^q_{k,i+1})$$

That is, it is the line that connects $\lambda^{p,q}_{k,i} + \epsilon_k$ and $\lambda^{p,q}_{k,i+1}$.

By construction, $\psi^{p,q}_k$ is continuous in $[0, 1]$ and moreover it holds that:

$$\lambda^{p,q}_{k,i} \leq \psi^{p,q}_k(x) \leq \lambda^{p,q}_{k,i} + \epsilon_k \quad \text{on } A^q_{k,i}$$

Let's see if the sequence of functions converges uniformly. If it converges uniformly, then the limit $\psi^{p,q}$ is continuous.

For $x \in A^q_{k,i}$ we obtain that

$$\sup_{x \in A^q_{k,i}} \left|\psi^{p,q}_{k+1}(x) - \psi^{p,q}_k(x)\right| \leq \left|\lambda^{p,q}_{k+1,i} + \varepsilon_{k+1} - \lambda^{p,q}_{k,i}\right|$$

$$\leq \left|\tau_k - \tau_{k-1} + \varepsilon_{k+1}\right| \quad \text{(lemma 2 point 2)}$$

$$\leq \left|\tau_k\right| + \left|\tau_{k+1}\right| + \left|\varepsilon_{k+1}\right| \xrightarrow[k\to\infty]{} 0$$

for $\varepsilon_k < \tau_k \leq \frac{1}{2^k}$ by point 4 of Lemma 2.

If $x \in G^q_{k,i}$, then

$$\sup_{x \in G^q_{k,i}} \left|\psi^{p,q}_{k+1}(x) - \psi^{p,q}_k(x)\right| \leq \left|\lambda^{p,q}_{k+1,i+1} - \lambda^{p,q}_{k,i}\right|$$

$$= \left|\lambda^{p,q}_{k+1,i+1} + \lambda^{p,q}_{k+1,i} - \lambda^{p,q}_{k+1,i} - \lambda^{p,q}_{k,i}\right|$$

$$\leq \left|\tfrac{1}{2^k} + \tau_k - \tau_{k-1}\right| \quad \text{(lemma 2 point 1 and point 2)}$$

$$\leq \tfrac{1}{2^k} + \left|\tau_k\right| + \left|\tau_{k-1}\right| \xrightarrow[k\to\infty]{} 0.$$

Therefore $\psi^{p,q}_k$ converges uniformly to $\psi^{p,q}$.

We therefore have a Cauchy sequence; the space of continuous functions with the given norm is complete, so this limit is unique.

□

Proposition 1 (Separation property)**.** *From conditions 5 (Lemma 3) and (3) (Lemma 2) it follows that*

6.
$$\sum_{p=1}^{n} \psi^{p,q}(x_p) \;\in\; \Delta^q_{k,i_1,\ldots,i_n}, \quad \textit{for every } (x_1,\ldots,x_n) \in S^q_{k,i_1\ldots i_n}.$$

Proof. If $(x_1,\ldots,x_n) \in S^q_{k,i_1\ldots i_n}$, then $x_p \in A^q_{k,i_p}$ for $p = 1,\ldots,n$. For each p, condition 5(lemma 3) guarantees
$$\lambda^{p,q}_{k,i_p} \;\le\; \psi^{p,q}(x_p) \;\le\; \lambda^{p,q}_{k,i_p} + \epsilon_k.$$
Summing over p,
$$\sum_{p=1}^{n} \lambda^{p,q}_{k,i_p} \;\le\; \sum_{p=1}^{n} \psi_{p,q}(x_p) \;\le\; \sum_{p=1}^{n} \lambda^{p,q}_{k,i_p} + n\,\epsilon_k.$$
By definition, $\Delta^q_{k,i_1,\ldots,i_n} = \left[\sum_{p=1}^{n} \lambda^{p,q}_{k,i_p}, \sum_{p=1}^{n} \lambda^{p,q}_{k,i_p} + n\,\epsilon_k\right]$. Thus the sum $\sum_p \psi^{p,q}(x_p)$ falls exactly in $\Delta^q_{k,i_1,\ldots,i_n}$. □

Remark 1.2 (Universality of $\psi^{p,q}$). The functions $\psi^{p,q}$ depend only on the dimension n, not on the function f to be represented.

1.3 Construction of the Functions χ^q and Proof of the Theorem

On establishing the existence of the functions ψ^{pq} and the constants $\lambda^{pq}_{k,i}$ and ϵ_k possessing properties 1)-6), we proceed to the proof of the main theorem. The desired functions $\chi^q(y)$ will be constructed in the form of a limit:

$$\chi^q = \lim_{r\to\infty} \chi^q_r$$

where $\chi^q_0 \equiv 0$. For $r > 0$, the functions χ^q_r will be defined by induction on r, simultaneously with a sequence of natural numbers k_r.

We will use the following notation:

$$f_r(x_1, ..., x_n) = \sum_q \chi^q_r \left[\sum_p \psi^{pq}(x_p)\right] \tag{1.5}$$

$$M_r = \sup_{E^n} |f - f_r| \tag{1.6}$$

It is obvious that for the base case ($r = 0$),
we have $f_0 \equiv 0$ and thus $M_0 = \sup_{E^n} |f|$.

Assume that the continuous functions χ^q_{r-1} and the number k_{r-1} have already been determined. This also determines the continuous function f_{r-1} on E^n. Since the diameters of the cubes $S^q_{k,i_1...i_n}$ tend to zero as $k \to \infty$, we can choose an integer k_r so large that the oscillation of the difference $f - f_{r-1}$ does not exceed $\frac{1}{2n+2}M_{r-1}$ on any single cube $S^q_{k_r,i_1...i_n}$.

Now, we define the iterative correction.
Let $\xi^q_{k,i}$ be arbitrary points belonging to the corresponding closed intervals $A^q_{k,i}$. For any value y that falls within a closed interval $\Delta^q_{k_r,i_1...i_n}$, we define the next function in the sequence, χ^q_r, as follows:

$$\chi^q_r(y) = \chi^q_{r-1}(y) + \frac{1}{n+1}\left[f(\xi^q_{k_r,i_1}, ..., \xi^q_{k_r,i_n}) - f_{r-1}(\xi^q_{k_r,i_1}, ..., \xi^q_{k_r,i_n})\right] \tag{1.7}$$

From the definition of M_{r-1}, the magnitude of this added term is bounded. This gives the crucial inequality:

$$|\chi^q_r(y) - \chi^q_{r-1}(y)| \le \frac{1}{n+1}M_{r-1} \tag{1.8}$$

Outside the closed intervals $\Delta^q_{k_r,i_1...i_n}$, the function χ^q_r is defined arbitrarily, with the only constraint being the preservation of continuity and inequality (1.8).

We now estimate the new error, $f - f_r$, at an arbitrary point $(x_1, ..., x_n) \in E^n$. By definition:

$$f - f_r = (f - f_{r-1}) - \sum_q \left\{\chi^q_r \left[\sum_p \psi^{pq}(x_p)\right] - \chi^q_{r-1}\left[\sum_p \psi^{pq}(x_p)\right]\right\} \tag{1.9}$$

We represent the sum $\sum_q$ as two separate sums, $\Sigma' + \Sigma''$. The sum Σ' extends over a specific set of $n + 1$ values of q for which the point $(x_1, ..., x_n)$ is contained in one of the cubes $S^q_{k_r,i_1...i_n}$ (by Lemma 1, such a set of cubes exists). The sum Σ'' extends over the remaining n values of q.

By virtue of (1.7), for each term in Σ', we can rewrite the correction term. Since the oscillation of $f - f_{r-1}$ is small within the cube, we can relate the value at the sample point ξ^q to the value at our arbitrary point x:

$$\begin{aligned}\chi_r^q \left[\sum_p \psi^{pq}(x_p)\right] - \chi_{r-1}^q \left[\sum_p \psi^{pq}(x_p)\right] &= \frac{1}{n+1}\left[f(\xi_{k_r,i_1}^q, ..., \xi_{k_r,i_n}^q) - f_{r-1}(\xi_{k_r,i_1}^q, ..., \xi_{k_r,i_n}^q)\right] \\ &= \frac{1}{n+1}\left[f(x_1, ..., x_n) - f_{r-1}(x_1, ..., x_n)\right] + \frac{\omega^q}{n+1},\end{aligned} \tag{1.10}$$

where ω^q is the small residual error due to this substitution. Based on our choice of k_r, this error is bounded:

$$|\omega^q| \leq \frac{1}{2n+2} M_{r-1} \tag{1.11}$$

We now break down the derivation of the main inequality step-by-step, starting from equation (1.9).

$$f - f_r = (f - f_{r-1}) - \sum_{\Sigma'} (\chi_r^q - \chi_{r-1}^q) - \sum_{\Sigma''} (\chi_r^q - \chi_{r-1}^q)$$

Now, we substitute the expression from (1.10) for the $n+1$ terms in the Σ' sum:

$$= (f - f_{r-1}) - \sum_{\Sigma'} \left(\frac{1}{n+1}(f - f_{r-1}) + \frac{\omega^q}{n+1}\right) - \sum_{\Sigma''} (\chi_r^q - \chi_{r-1}^q)$$

The first part of the sum over Σ' consists of $n+1$ identical terms, so they sum up to cancel the initial error term:

$$= (f - f_{r-1}) - (n+1) \cdot \frac{1}{n+1}(f - f_{r-1}) - \frac{1}{n+1} \sum_{\Sigma'} \omega^q - \sum_{\Sigma''} (\chi_r^q - \chi_{r-1}^q)$$

The main error terms $(f - f_{r-1})$ cancel out perfectly. This is the key step of the proof:

$$f - f_r = -\frac{1}{n+1} \sum_{\Sigma'} \omega^q - \sum_{\Sigma''} (\chi_r^q - \chi_{r-1}^q)$$

Now, we have

$$|f - f_r| \leq \frac{1}{n+1} \sum_{\Sigma'} |\omega^q| + \sum_{\Sigma''} |\chi_r^q - \chi_{r-1}^q|$$

Finally, we use the bounds established in (1.11) for the first part and (1.8) for the n terms in the second part.

$$|f - f_r| \leq \frac{1}{2n+2} M_{r-1} + \frac{n}{n+1} M_{r-1} = \frac{2n+1}{2n+2} M_{r-1} \tag{1.12}$$

Since inequality (1.12) holds for any point $(x_1, ..., x_n) \in E^n$, we have shown that the maximum error decreases at each step:

$$M_r \leq \frac{2n+1}{2n+2} M_{r-1}$$

which, by induction, leads to:

$$M_r \leq \left(\frac{2n+1}{2n+2}\right)^r M_0 \tag{1.13}$$

From (1.8) and (1.13), it follows that the absolute values of the differences $\chi_r^q - \chi_{r-1}^q$ are bounded by the terms of an absolutely convergent geometric series. Therefore, for each q, the sequence of functions $\{\chi_r^q\}_{r=1}^{\infty}$ converges uniformly to a continuous limit function χ^q as $r \to \infty$.

As $r \to \infty$, $M_r \to 0$, which means the sequence of functions f_r converges uniformly to f. This completes the proof of Kolmogorov's superposition theorem.

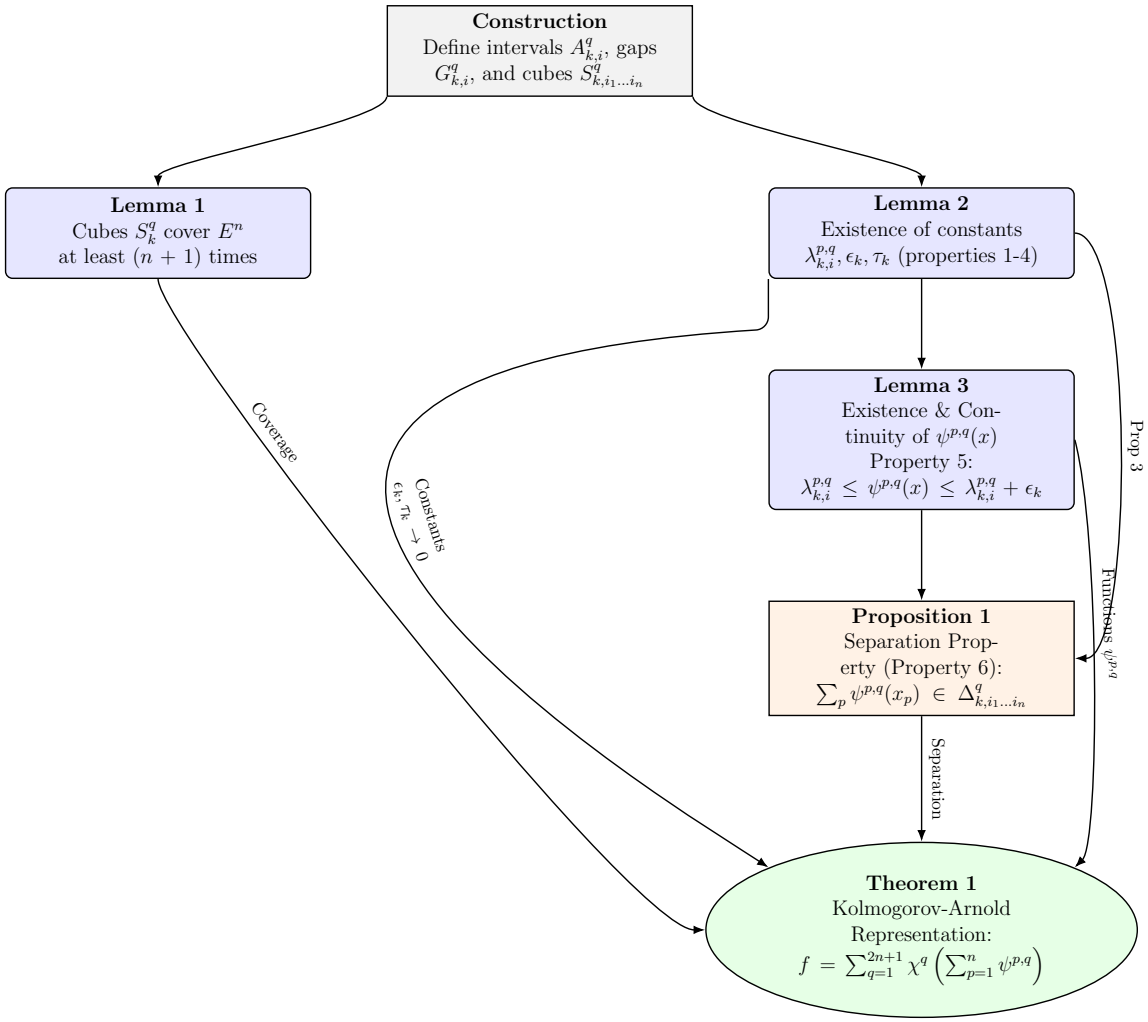


Figure 1.4: Diagram of the logical dependencies in the proof of the Kolmogorov-Arnold Theorem.

Chapter 2

Vitushkin's Theorem

2.1 Introduction

Vitushkin's theorem shows that the universality provided by the Kolmogorov–Arnold representation theorem is highly sensitive to regularity assumptions. If the inner functions $\psi^{p,q}$ are required to be differentiable (for instance of class C^1), the class of superpositions they generate becomes too restricted and no longer coincides with the whole space of continuous functions.

The aim of this chapter is to present a proof of this negative result. More precisely, we show that imposing differentiability on the inner functions leads to a family of superpositions that is *not dense* in the space of continuous functions $C(E^n)$. Consequently, there exist continuous functions that cannot be represented by superpositions constructed with differentiable inner functions.

In the previous chapter we established the Kolmogorov–Arnold theorem, which asserts that every continuous multivariate function can be represented as a superposition of functions of a single variable. Such representations involve two types of functions: the outer functions χ^q, which depend on the specific function f, and the inner functions $\psi^{p,q}$, which depend only on the dimension n. This naturally leads to the following question: can the universal inner functions $\psi^{p,q}$ be chosen with additional regularity properties, such as differentiability? Vitushkin showed that this is not possible [3], demonstrating that regularity constraints, although natural from an analytical viewpoint, drastically reduce the expressive power of Kolmogorov superpositions.

2.2 Notation and definitions

Throughout this work all functions are assumed to be defined on their domains and continuous unless otherwise specified. A function is said to be *continuously differentiable* if all first partial derivatives exist and are continuous.

Notation 2.1. We denote by

$$z = (x, y) \in \mathbb{R}^2$$

a point of the Euclidean plane.

This notation allows functions of two variables to be treated as functions of a single vector variable.

Definition 2.2 (Euclidean distance)**.** For $z_1, z_2 \in \mathbb{R}^2$, the Euclidean distance is defined by

$$\rho(z_1, z_2) = \|z_1 - z_2\|.$$

The distance induces the standard topology and metric structure of the plane.

Definition 2.3 (Gradient)**.** Let $q : \mathbb{R}^2 \to \mathbb{R}$ be continuously differentiable. The gradient of q at z is

$$\nabla q(z) = \left(\frac{\partial q}{\partial x}(z), \frac{\partial q}{\partial y}(z) \right).$$

The gradient gives the direction of maximal increase of the function and is orthogonal to its level sets.

Definition 2.4 (Jacobian determinant)**.** Let $F = (q_1, q_2) : \mathbb{R}^2 \to \mathbb{R}^2$ be continuously differentiable. The Jacobian determinant of F at (x, y) is

$$J_F(x, y) = \det DF(x, y) = \begin{vmatrix} \dfrac{\partial q_1}{\partial x} & \dfrac{\partial q_1}{\partial y} \\ \dfrac{\partial q_2}{\partial x} & \dfrac{\partial q_2}{\partial y} \end{vmatrix}.$$

The Jacobian measures the local variation of area induced by the mapping.

Definition 2.5 (Region)**.** A *region* is an open subset of $\mathbb{R}^2$(not assuming connectedness).

Definition 2.6 (Closed region)**.** A *closed region* is the closure of a region.

Regions represent domains where analytical and geometric arguments are performed.

Definition 2.7 (Image of a set)**.** Let $q : \mathbb{R}^2 \to \mathbb{R}$ and $D \subset \mathbb{R}^2$. The image of D under q is

$$q(D) = \{q(z) : z \in D\}.$$

Definition 2.8 (Inverse image)**.** For $\delta \subset \mathbb{R}$,

$$q^{-1}(\delta) = \{z \in \mathbb{R}^2 : \ q(z) \in \delta\}.$$

Definition 2.9 (Modulus of continuity)**.** Let $G \subset \mathbb{R}^2$. A function $p : G \to \mathbb{R}$ admits a modulus of continuity $\omega(\delta)$ if for every $\delta > 0$

$$\rho(z_1, z_2) \leq \delta \quad \Rightarrow \quad |p(z_1) - p(z_2)| \leq \omega(\delta)$$

for all $z_1, z_2 \in G$.

The modulus of continuity provides quantitative control on oscillations of the function.

Definition 2.10 (Level set)**.** Let $q : \mathbb{R}^2 \to \mathbb{R}$ and $t \in \mathbb{R}$. The level set of level t is

$$e(q, t) = \{z \in \mathbb{R}^2 : \ q(z) = t\}.$$

Level sets describe the geometric structure of a function through curves along which the function remains constant.

Definition 2.11 (Curve)**.** Let $I \subset \mathbb{R}$ be an interval. A *(parametrized) curve* in $\mathbb{R}^2$ is a continuous map

$$\mathbf{r} : I \to \mathbb{R}^2.$$

Its *trace* (or image) is the set

$$e = \mathbf{r}(I) \subset \mathbb{R}^2.$$

In the sequel, when convenient, we identify a curve with its trace e.

Definition 2.12 (Regular curve)**.** A curve $\mathbf{r} : I \to \mathbb{R}^2$ is called *regular* if $\mathbf{r}$ is continuously differentiable and

$$\mathbf{r}'(t) \neq 0 \qquad \text{for all } t \in I.$$

Regularity excludes cusps and stationary points.

Definition 2.13 (Tangent vector)**.** Let $\mathbf{r} : I \to \mathbb{R}^2$ be a regular curve. For $t \in I$, the tangent vector at the point $z = \mathbf{r}(t)$ is

$$\mathbf{r}'(t).$$

Definition 2.14 (Unit tangent vector)**.** Under the assumptions above, the unit tangent vector at $z = \mathbf{r}(t)$ is

$$\tau(e, z) = \frac{\mathbf{r}'(t)}{\|\mathbf{r}'(t)\|}.$$

The unit tangent vector determines the local direction of the curve.

Definition 2.15 (Angle between vectors)**.** Let τ_1, τ_2 be unit vectors. The angle between them is

$$\gamma(\tau_1, \tau_2) = \arccos\big(|\langle \tau_1, \tau_2 \rangle|\big).$$

Definition 2.16 (Length of a curve)**.** If $\mathbf{r} : [a, b] \to \mathbb{R}^2$ is a continuously differentiable curve, its length is

$$h_1(e) = \int_a^b \|\mathbf{r}'(t)\| \, dt = \int_a^b \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2} \, dt = \int_e ds.$$

Definition 2.17 (One–dimensional diameter)**.** For a set $e \subset \mathbb{R}^2$,

$$d_1(e) = \sup_{z_1, z_2 \in e} \rho(z_1, z_2).$$

The diameter measures the maximal spatial extension of the set.

Definition 2.18 (Big–O estimate)**.** Let f and g be real-valued functions defined in a neighborhood of a point $a \in \mathbb{R}$. We write

$$f(x) = O(g(x)) \quad \text{as } x \to a$$

if there exist constants $C > 0$ and $\delta > 0$ such that

$$|f(x)| \leq C \, |g(x)|$$

for all x satisfying $0 < |x - a| < \delta$.

Definition 2.19 (Hausdorff distance)**.** For sets $A_1, A_2 \subset \mathbb{R}^2$,

$$\rho(A_1, A_2) = \max\left\{ \sup_{z_1 \in A_1} \inf_{z_2 \in A_2} \rho(z_1, z_2), \ \sup_{z_2 \in A_2} \inf_{z_1 \in A_1} \rho(z_1, z_2) \right\}.$$

The Hausdorff distance measures how far two sets are from coinciding as geometric objects.

2.3 Estimating the difference of integrals of a superposition term along nearby level curves

Let G be a region in the plane of variables x and y, and let $q_1(x, y)$ and $q_2(x, y)$ be continuously differentiable functions that satisfy the following conditions in this region:

a) the partial derivatives with respect to x and y have a modulus of continuity $\omega(\delta)$;

   This means that if two points are within a distance δ of each other, their derivatives also differ by at most $\omega(\delta)$.

b) the inequalities

$$0 < \gamma \leq |\operatorname{grad}[q_i(x, y)]| \leq \frac{1}{\gamma} < \infty \quad (i = 1, 2),$$

   are satisfied everywhere in G, where γ is a constant;

   Geometrically, this prevents the level curves from "flattening out" or "becoming vertical."

c) for every point $(x, y) \in G$, the absolute value of the acute angle formed by the level curves of the functions $q_1(x, y)$ and $q_2(x, y)$ passing through this point is greater than a positive constant γ.

   The level curves of q_1 and q_2 are never almost parallel: the angle between their tangent vectors is always greater than γ.

Lemma 1. *Let e'_{q_2} and e''_{q_2} be two level curves of the function q_2, and e'_{q_1} and e''_{q_1} be level curves of the function q_1; let $[a', a''] \subset G$ be the segment of the curve e'_{q_1} with endpoints $a' \in e'_{q_2}$ and $a'' \in e''_{q_2}$, and $[b', b'']$ be the segment of the curve e''_{q_1} with endpoints $b' \in e'_{q_2}$ and $b'' \in e''_{q_2}$. Then*

$$h_1([b', b'']) \leq h_1([a', a'']) \times (1 + c_1(\gamma)\omega(\delta)),$$

where $\delta = d_1([a', a''] \cup [b', b''])$, and $c_1(\gamma)$ depends only on γ.

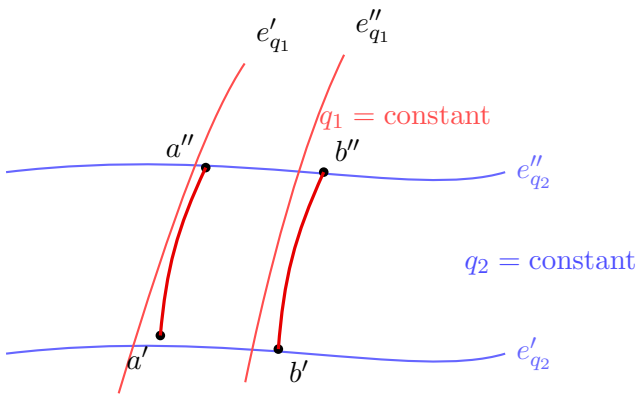


Figure 2.1: Relationship between the level curves of q_1 and q_2 and the segments $[a', a'']$, $[b', b'']$.

The length of the second segment does not differ too much from that of the first. The difference is controlled by $\omega(\delta)$, i.e., by the smoothness of the derivatives.

In other words:

$\rightarrow$ "If the derivatives change slowly, then the geometry of the level curves also changes slowly."

Proof. Since $q_2(a'') - q_2(a') = q_2(b'') - q_2(b')$, we have

$$\int_{s \in [a', a'']} \frac{\partial q_2}{\partial s} ds = \int_{s \in [b', b'']} \frac{\partial q_2}{\partial s} ds.$$

Consequently (Mean Value Theorem),

$$\frac{\partial q_2(a^*)}{\partial s} h_1([a', a'']) = \frac{\partial q_2(b^*)}{\partial s} h_1([b', b'']),$$

where $\frac{\partial q_2(a^*)}{\partial s}$ and $\frac{\partial q_2(b^*)}{\partial s}$ are the derivatives at the points $a^* \in [a', a'']$ and $b^* \in [b', b'']$ along the curves $[a', a'']$ and $[b', b'']$, respectively. We show that

$$\frac{\partial q_2(a^*)}{\partial s} = \frac{\partial q_2(b^*)}{\partial s} + O(\gamma)\omega(\delta). \tag{2.1}$$

Denote by q_2^* the derivative of q_2 at the point b^* in the direction $\tau(e'_{q_1}, a^*)$ and set $\alpha = \gamma\{\tau[e''_{q_1}, b^*], \tau[e'_{q_1}, a^*]\}$. From conditions a) and b) it follows that

$$\frac{\partial q_2(a^*)}{\partial s} = q_2^* + O(1)\omega(\delta) \quad \text{and} \tag{2.2}$$

$$\alpha = O(\gamma)\omega(\delta) \tag{2.3}$$

Denote by β_1 and β_2 the angles formed by the vectors $\tau[e''_{q_1}, b^*]$ and $\tau[e'_{q_1}, a^*]$ with the vector $\operatorname{grad}[q_2(b^*)]$. We have

$$|q_2^* - \frac{\partial q_2(b^*)}{\partial s}| = |\operatorname{grad}[q_2(b^*)]||\cos\beta_2 - \cos\beta_1| = O(\gamma)\alpha = O(\gamma)\omega(\delta). \tag{2.4}$$

Hence,

$$\begin{aligned}
\frac{\partial q_2(a^*)}{\partial s} &= q_2^* + O(1)\omega(\delta) \\
&= \frac{\partial q_2(b^*)}{\partial s} + O(1)\left\{|q_2^* - \frac{\partial q_2(b^*)}{\partial s}| + \omega(\delta)\right\} \\
&= \frac{\partial q_2(b^*)}{\partial s} + O(\gamma)\omega(\delta).
\end{aligned}$$

Consequently,

$$\begin{aligned}
h_1([b', b'']) &= h_1([a', a''])\frac{\partial q_2(a^*)}{\partial s}\left(\frac{\partial q_2(b^*)}{\partial s}\right)^{-1} \\
&= h_1([a', a''])\left(1 + O(\gamma)\omega(\delta)\left(\frac{\partial q_2(b^*)}{\partial s}\right)^{-1}\right) \\
&= h_1([a', a''])(1 + O(\gamma)\omega(\gamma)),
\end{aligned}$$

since by virtue of b) we have

$$\left|\frac{\partial q_2(b^*)}{\partial s}\right| \geq |\operatorname{grad}[q_2(b^*)]|\sin\gamma \tag{2.5}$$

(the denominator is nonzero). This proves the lemma.

□

Detailed proof of equation (2.2) ,(2.3),(2.4) e (2.5)

After presenting Lemma 1 proof, we provide here a detailed justification of equations (2.2), (2.3) ,(2.4) e (2.5) that are used in the argument. In the original proof, these formulas are stated as consequences of conditions (a) and (b) without explicit derivation. The aim of this section is therefore to verify these steps one by one.

Explanation of Relation 2.2

The purpose of this relation is to show that the directional derivative of q_2 at the point a^* is close to the directional derivative evaluated at the nearby point b^* but taken along the same direction.

By definition,

$$\frac{\partial q_2(a^*)}{\partial s} = \operatorname{grad}[q_2(a^*)] \cdot \tau(e'_{q_1}, a^*),$$

while

$$q_2^* = \operatorname{grad}[q_2(b^*)] \cdot \tau(e'_{q_1}, a^*).$$

Hence,

$$\left|\frac{\partial q_2(a^*)}{\partial s} - q_2^*\right| = \left|(\mathrm{grad}[q_2(a^*)] - \mathrm{grad}[q_2(b^*)]) \cdot \tau(e'_{q_1}, a^*)\right|.$$

Applying the Cauchy–Schwarz inequality and using that $\tau(e'_{q_1}, a^*)$ is a unit vector,

$$\leq \| \mathrm{grad}[q_2(a^*)] - \mathrm{grad}[q_2(b^*)]\|.$$

Condition (a) states that the partial derivatives possess a common modulus of continuity $\omega(\delta)$. Since $\rho(a^*, b^*) \leq \delta$,

$$\left|\frac{\partial q_2}{\partial x}(a^*) - \frac{\partial q_2}{\partial x}(b^*)\right| \leq \omega(\delta),$$
$$\left|\frac{\partial q_2}{\partial y}(a^*) - \frac{\partial q_2}{\partial y}(b^*)\right| \leq \omega(\delta).$$

Therefore,

$$\| \mathrm{grad}[q_2(a^*)] - \mathrm{grad}[q_2(b^*)]\| \leq \sqrt{2}\,\omega(\delta),$$

which yields

$$\frac{\partial q_2(a^*)}{\partial s} = q_2^* + O(1)\omega(\delta).$$

Explanation of Relation 2.3

The quantity α denotes the angle between the tangent vectors of the level curves e'_{q_1} and e''_{q_1} at nearby points.

Since gradients are orthogonal to level curves,

$$\tau(e'_{q_1}, a^*) \perp \mathrm{grad}[q_1(a^*)], \qquad \tau(e''_{q_1}, b^*) \perp \mathrm{grad}[q_1(b^*)].$$

Hence the angle α between the tangents equals the angle between the corresponding gradient vectors.

Let

$$\mathbf{u}_{a^*} = \frac{\mathrm{grad}[q_1(a^*)]}{\| \mathrm{grad}[q_1(a^*)]\|}, \qquad \mathbf{u}_{b^*} = \frac{\mathrm{grad}[q_1(b^*)]}{\| \mathrm{grad}[q_1(b^*)]\|}.$$

For small variations,

$$\alpha \approx \|\mathbf{u}_{a^*} - \mathbf{u}_{b^*}\|.$$

Condition (a) guarantees that the gradient varies continuously with modulus $\omega(\delta)$, while condition (b) ensures that $\| \mathrm{grad}\, q_1\| \geq \gamma > 0$. Together these imply

$$\alpha = O(\gamma)\omega(\delta).$$

Explanation of Relation 2.4

Both q_2^* and $\frac{\partial q_2(b^*)}{\partial s}$ are directional derivatives evaluated at the same point b^*, but taken along two different directions.

Using the formula for directional derivatives,

$$\frac{\partial q_2(b^*)}{\partial s} = \mathrm{grad}[q_2(b^*)] \cdot \tau[e''_{q_1}, b^*] = \| \mathrm{grad}[q_2(b^*)]\| \cos\beta_1,$$
$$q_2^* = \mathrm{grad}[q_2(b^*)] \cdot \tau[e'_{q_1}, a^*] = \| \mathrm{grad}[q_2(b^*)]\| \cos\beta_2.$$

Subtracting the two expressions gives

$$|q_2^* - \tfrac{\partial q_2(b^*)}{\partial s}| = \|\operatorname{grad}[q_2(b^*)]\| |\cos\beta_2 - \cos\beta_1|.$$

The difference $|\cos\beta_2 - \cos\beta_1|$ is controlled by the angle between the two directions, which equals α. Since

$$\alpha = O(\gamma)\omega(\delta)$$

and condition (b) provides the bound

$$\|\operatorname{grad}[q_2(b^*)]\| \leq \frac{1}{\gamma},$$

the whole expression is of order

$$O(\gamma)\omega(\delta).$$

Explanation of Relation 2.5

To justify equation (2.5), observe that

$$\left|\frac{\partial q_2(b^*)}{\partial s}\right| = \|\operatorname{grad}[q_2(b^*)]\| \, |\cos\theta|,$$

where θ denotes the angle between $\operatorname{grad}[q_2(b^*)]$ and the tangent vector $\tau[e''_{q_1}, b^*]$.

By condition (c), the angle between the tangent vectors $\tau[e''_{q_1}]$ and $\tau[e''_{q_2}]$ is bounded below by γ. Since the gradient $\operatorname{grad}[q_2]$ is orthogonal to the tangent $\tau[e''_{q_2}]$, it follows that $\theta = \frac{\pi}{2} \pm \phi$, where $\phi \geq \gamma$ is the angle between the two tangents. Hence

$$|\cos\theta| = |\cos(\tfrac{\pi}{2} \pm \phi)| = \sin\phi \geq \sin\gamma,$$

which yields (2.5).

Lemma 2. *Let $q_m(x,y)$ $(m = 1,2,\ldots,N)$ be continuously differentiable functions. In every region D, we can find a subregion $G \subset D$, determine a constant $\gamma > 0$, and reindex the functions $\{q_m(x,y)\}$ with two indices so that the functions*

$$q_i^k(x,y) = q_m(x,y) \quad (i = 0,1,2,\ldots,n; \quad k = 1,2,\ldots,m_i; \quad \sum_{i=0}^{n} m_i = N)$$

obtained after reindexing satisfy the following conditions:

1. *when $i = 0$, $q_i^k = const$ in G; and when $i > 0$,*

$$\gamma \leq |\operatorname{grad}[q_i^k(x,y)]| \leq \frac{1}{\gamma}$$

   for every point $(x,y) \in G$;

2. *the functions $q_i^k(x,y)$ $(i > 0$ fixed, $k = 1,2,\ldots,m_i)$ have identical sets of level curves in the region G; more precisely, within G,*

$$q_i^k(x,y) \equiv \phi_i^{kl}(q_i^l(x,y)),$$

   where $\phi_i^{kl}(t)$ is a strictly monotonic and continuously differentiable function of t;

3. *when $i \neq j$ $(i,j \neq 0)$, then for every k and l, the absolute value of the acute angle formed by the level curves of the functions $q_i^k(x,y)$ and $q_j^l(x,y)$ passing through any point $(x,y) \in G$ is greater than γ.*

Explanation of the Lemma

The family $i = 0$ consists of all functions that are constant inside the region G. For these functions the gradient vanishes identically,

$$\operatorname{grad}[q_0^k(x,y)] \equiv 0,$$

so they play no role in the local geometry: they do not determine directions of variation and therefore do not generate meaningful level curves.

All remaining families, namely those with $i > 0$, contain the non-trivial functions whose gradients are uniformly bounded away from both zero and infinity. More precisely,

$$\gamma \leq |\operatorname{grad}[q_i^k(x,y)]| \leq \frac{1}{\gamma}.$$

This assumption guarantees that the corresponding level curves are well defined and non-degenerate.

This subdivision is useful for the study of superpositions. The terms belonging to the family $i = 0$ are straightforward to treat because the functions q_0^k are constant. The sum can then be analyzed family by family, since within each fixed family i all functions q_i^k possess the same geometry of level curves, which simplifies the computations. Moreover, the interaction between different families $(i \neq j)$ remains controlled: their gradients determine distinct directions of variation, and the corresponding level curves do not run parallel but intersect at a definite angle.

Proof. We divide the proof into four steps.

Step 1: From D to G^* — Separating Constant and Non-constant Functions

The proof begins by invoking the continuity of the partial derivatives. Since the partial derivatives are continuous, the magnitude of the gradient $|\operatorname{grad} q_m|$ is also a continuous function.

Restricting the analysis to a sufficiently small subregion $G^* \subset D$, it follows that for every function q_m exactly one of the following alternatives holds throughout G^*: either $\operatorname{grad} q_m(x, y) \equiv 0$, meaning that the function is constant, or $|\operatorname{grad} q_m(x, y)| > c > 0$, meaning that the function never becomes flat. This separation distinguishes the trivial constant functions from the non-constant ones and establishes Condition 1 of the lemma.

Step 2: From G^* to G^{**} — Fixing Geometric Relationships

The argument then considers pairs of functions through the Jacobian determinant

$$D\left(\frac{q_r, q_s}{x, y}\right) = \det(\nabla q_r, \nabla q_s) = |\nabla q_r||\nabla q_s| \sin\theta,$$

where θ denotes the angle between the gradient vectors. The Jacobian measures the geometric relation between the corresponding level curves.

If the Jacobian is non-zero at a point, the gradient vectors are not parallel; since level curves are orthogonal to gradients, the level curves themselves are not parallel and intersect at a non-zero angle. Conversely, if the Jacobian vanishes identically in a region, the gradient vectors remain parallel everywhere, implying that the level curves are mutually tangent and therefore belong to the same family of level curves.

Because the Jacobian is itself continuous, being composed of continuous partial derivatives, one may further restrict the domain to a smaller region $G^{**} \subset G^*$ in which, for every pair (q_r, q_s), one of these two geometric situations holds uniformly throughout the region. This prepares the verification of Conditions 2 and 3.

Step 3: From G^{**} to G — Establishing Functional Dependence

The final refinement uses a consequence of the theory underlying the Implicit Function Theorem. Whenever the Jacobian of two functions q_r and q_s vanishes identically in G^{**}, the functions are functionally dependent; that is, one may be expressed as a transformation of the other.

After possibly shrinking the domain once more to a region $G \subset G^{**}$, it follows that for every such pair there exists a strictly monotone and continuously differentiable function ϕ satisfying

$$q_s(x, y) = \phi(q_r(x, y)).$$

This establishes Condition 2 for all functions belonging to the same family.

Step 4: The Final Re-indexing

Having obtained a region G where all geometric relations remain stable, the last step is organizational. The original collection of functions $\{q_m\}$ is re-indexed using two indices (i, k) according to their behavior inside G.

All functions that are constant in G receive the family index $i = 0$. A non-constant function is then selected and assigned to a new family, for instance $i = 1$, and every remaining unclassified function sharing the same level curves (equivalently, having identically vanishing Jacobian with it) is placed in the same family, distinguished by the second index k. Repeating this procedure produces the families $i = 2, 3, \ldots, n$ until all functions are classified.

By construction, the re-indexed functions q_i^k satisfy all three conditions of the lemma inside the region G. $\square$

Analysis of the Superposition

Consider, in the region G, a superposition of the form

$$\sum_{i=0}^{n}\sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y)),$$

where:

1. $\{f_i^k(t)\}$ are continuous functions of one variable,
2. $\{p_i^k(x,y)\}$ are continuous functions satisfying $|p_i^k(x,y)| \leq \frac{1}{\gamma}$ in G,
3. $\{q_i^k(x,y)\}$ are continuously differentiable functions satisfying in G the conditions (1), (2), and (3) of Lemma 2.

Let $\omega(\delta)$ denote the common modulus of continuity in G of the functions

$$\{p_i^k(x,y);\ \frac{\partial q_i^k(x,y)}{\partial x};\ \frac{\partial q_i^k(x,y)}{\partial y}\}.$$

Let $[a', a'']$ and $[b', b'']$ be segments of level curves of the functions $\{q_i^k(x,y)\}$ (for fixed $i > 0$) lying in G. Define

$$\alpha = h_1([a', a'']); \qquad \delta = \rho([a', a''], [b', b'']);$$

$$\epsilon = \sup\left|\sum_{i=0}^{n}\sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y))\right|; \qquad m = \max_{i,k}\sup|f_i^k(q_i^k(x,y))|,$$

where the supremum is taken over all points $(x,y) \in [a', a''] \cup [b', b'']$.

Lemma 3. *If δ is sufficiently small ($\omega(\delta) \leq c_2(\gamma)$), then for every $i > 0$,*

$$\left| \int_{s\in[a',a'']} \sum_{k=1}^{m_i} p_i^k(s) f_i^k(q_i^k(s)) ds - \int_{s\in[b',b'']} \sum_{k=1}^{m_i} p_i^k(s) f_i^k(q_i^k(s)) ds \right| \leq c_3(\gamma)(\alpha\epsilon + ma\omega(\delta) + m\delta),$$

where the constants $c_2(\gamma)$ and $c_3(\gamma)$ depend only on γ.

Proof. The goal is to compare the integral of a sum of family functions i along a level curve $s \in [a', a'']$ with a similar integral along a nearby level curve $s \in [b', b'']$. We will show that their difference is small, controlled by the distance δ between the curves, the smoothness of the functions (via $\omega(\delta)$), and the maximum amplitude ϵ of the total superposition.

Step 1: Decomposing the Problem

The strategy is to use an "add and subtract" technique. Instead of directly estimating the difference for the family with index i, we bound it with two more manageable terms:

1. The difference between the integrals of the **complete superposition** (sum over all indices $j = 0, \ldots, n$).
2. The difference between the integrals of the terms with an index **different from** i (sum over $j \neq i$).

Applying the triangle inequality $|A| = |(A + B) - B| \leq |A + B| + |B|$, we get:

$$\begin{aligned}
&\left| \int_{[a',a'']} \sum_{k=1}^{m_i} p_i^k f_i^k ds - \int_{[b',b'']} \sum_{k=1}^{m_i} p_i^k f_i^k ds \right| \\
&\leq \underbrace{\left| \int_{[a',a'']} \sum_{j=0}^{n} \sum_{k=1}^{m_j} p_j^k f_j^k ds - \int_{[b',b'']} \sum_{j=0}^{n} \sum_{k=1}^{m_j} p_j^k f_j^k ds \right|}_{\text{Term 1: Complete Superposition}} \\
&\quad + \underbrace{\sum_{j\neq i} \left| \int_{[a',a'']} \sum_{k=1}^{m_j} p_j^k f_j^k ds - \int_{[b',b'']} \sum_{k=1}^{m_j} p_j^k f_j^k ds \right|}_{\text{Term 2: "Cross" Terms } (j \neq i)}
\end{aligned}$$

We will now estimate these two terms separately.

Step 2: Bounding Term 1 (Complete Superposition)

This term is the easiest to bound. By definition, ϵ is the supremum of the absolute value of the complete superposition on both curves.

$$\left| \sum_{j=0}^{n} \sum_{k=1}^{m_j} p_j^k(s) f_j^k(q_j^k(s)) \right| \leq \epsilon \quad \forall s \in [a', a''] \cup [b', b'']$$

Using the standard inequality for integrals ($|\int g ds| \leq \int |g| ds \leq \text{length}(curve) \cdot \sup |g|$), we obtain:

$$\begin{aligned}
&\left| \int_{[a',a'']} \left(\sum \dots \right) ds - \int_{[b',b'']} \left(\sum \dots \right) ds \right| \\
&\leq \left| \int_{[a',a'']} \left(\sum \dots \right) ds \right| + \left| \int_{[b',b'']} \left(\sum \dots \right) ds \right| \\
&\leq h_1([a',a'']) \cdot \epsilon + h_1([b',b'']) \cdot \epsilon \\
&= (\alpha + h_1([b',b''])) \epsilon
\end{aligned}$$

From **Lemma 1**, we know that the lengths of nearby curves are almost equal: $h_1([b',b'']) \approx h_1([a',a'']) = \alpha$. More precisely, $h_1([b',b'']) \leq \alpha(1 + O(\gamma)\omega(\delta))$. For small δ, this length is close to α. Thus, the total contribution is of the order $O(\gamma)\alpha\epsilon$.

Step 3: Bounding Term 2 ("Cross" Terms $j \neq i$)

This is the most technical step. It relies on a key geometric guarantee derived from the conditions of the previous lemma (Lemma 2).

From (1), (2), and (3), there exist a sufficiently small constant $c_2(\gamma)$ and a sufficiently large constant $c_3(\gamma)$ such that if $\omega(\delta) \leq c_2(\gamma)$ and, for a point $a \in [a',a'']$, the inequalities $h_1([a',a]) \geq c_3(\gamma)\delta$ and $h_1([a,a'']) \geq c_3(\gamma)\delta$ hold, then for every $j \neq i$ ($j > 0$), the level curve of q_j^k passing through a intersects $[b',b'']$, the level curve of q_i^k.

This guarantee allows us to proceed. We fix an index $j \neq i$ and analyze the difference:

$$\Delta_j = \left| \int_{[a',a'']} \sum_{k=1}^{m_j} p_j^k f_j^k ds - \int_{[b',b'']} \sum_{k=1}^{m_j} p_j^k f_j^k ds \right|$$

The idea is to discretize the integrals and compare the corresponding terms of the Riemann sums.

1. **Geometric Construction:** Based on the guarantee above, we can create a correspondence. We divide the arc $[a',a'']$ into many small segments $[a_r, a_{r+1}]$. For each point a_r on the "central" part of the arc (i.e., not within $c_3(\gamma)\delta$ of the endpoints), we are **guaranteed** that the level curve of q_j^k passing through it will intersect the arc $[b',b'']$ at a unique point b_r. This construction creates the correspondence between the points $\{a_r\}$ and $\{b_r\}$.

2. **Approximation with Riemann Sums:**

$$\begin{aligned}
\int_{[a',a'']} p_j^k f_j^k ds &\approx \sum_r p_j^k(a_r) f_j^k(q_j^k(a_r)) h_1([a_r, a_{r+1}]) \\
\int_{[b',b'']} p_j^k f_j^k ds &\approx \sum_r p_j^k(b_r) f_j^k(q_j^k(b_r)) h_1([b_r, b_{r+1}])
\end{aligned}$$

3. **Term-by-Term Comparison:** We analyze the difference between the terms of the two sums.

- **The term** $f_j^k(q_j^k)$: By construction, a_r and b_r lie on the **same** level curve of the function q_j^k. Therefore, $q_j^k(a_r) = q_j^k(b_r)$, and consequently $f_j^k(q_j^k(a_r)) = f_j^k(q_j^k(b_r))$. **This term is identical!**
- **The term** p_j^k: The functions p_j^k have a modulus of continuity $\omega(\delta)$. Since the distance $\rho(a_r, b_r) \leq \delta$, we have $|p_j^k(a_r) - p_j^k(b_r)| \leq \omega(\delta)$. Thus $p_j^k(b_r) = p_j^k(a_r) + O(\omega(\delta))$.
- **The length term** h_1: The points $(a_r, a_{r+1}, b_{r+1}, b_r)$ form a "quadrilateral" bounded by level curves of families i and j. By **Lemma 1**, the lengths of the opposite sides are almost equal: $h_1([b_r, b_{r+1}]) = h_1([a_r, a_{r+1}])(1 + O(\gamma)\omega(\delta))$.

4. **Calculating the Difference:** Putting the pieces together, the general term of the difference of the Riemann sums is:

$$\begin{aligned}
&p_j^k(a_r) f_j^k(q_j^k(a_r)) h_1([a_r, a_{r+1}]) - p_j^k(b_r) f_j^k(q_j^k(b_r)) h_1([b_r, b_{r+1}]) \\
&= p_j^k(a_r) f_j^k(q_j^k(a_r)) h_1([a_r, a_{r+1}]) \\
&\quad - \underbrace{(p_j^k(a_r) + O(\omega(\delta)))}_{\text{from continuity of } p_j^k} \underbrace{f_j^k(q_j^k(a_r))}_{\text{since } q_j^k(a_r)=q_j^k(b_r)} \underbrace{h_1([a_r, a_{r+1}])(1 + O(\gamma)\omega(\delta))}_{\text{from Lemma 1}}
\end{aligned}$$

Expanding the product, we notice that the main terms cancel out. The remaining terms are small, of the order $\omega(\delta)$.

$$\approx -f_j^k(q_j^k(a_r)) h_1([a_r, a_{r+1}]) \cdot [O(\omega(\delta)) + p_j^k(a_r) O(\gamma)\omega(\delta)]$$

The absolute value of this is bounded by $m \cdot h_1([a_r, a_{r+1}]) \cdot O(\gamma)\omega(\delta)$.

Summing over all segments, the total length is $\sum h_1([a_r, a_{r+1}]) \approx \alpha$. Therefore, the total difference of the integrals is of the order $O(\gamma) m \alpha \omega(\delta)$.

To this, we must add an error for the "edges" of the arcs that might not have a correspondence, of a length proportional to δ. This introduces a term $O(\gamma) m \delta$.

The final estimate for a single $j \neq i$ is:

$$\Delta_j \leq O(\gamma) m (\delta + \alpha\omega(\delta))$$

Step 4: Conclusion

We now combine the estimates from Steps 2 and 3.

$$\begin{aligned}
&\left| \int_{[a', a'']} \sum_{k=1}^{m_i} \dots ds - \int_{[b', b'']} \sum_{k=1}^{m_i} \dots ds \right| \\
&\leq \underbrace{O(\gamma)\alpha\epsilon}_{\text{From Step 2}} + \underbrace{\sum_{j \neq i} O(\gamma) m (\delta + \alpha\omega(\delta))}_{\text{From Step 3}} \\
&\leq c_4(\gamma)\alpha\epsilon + N \cdot c_5(\gamma) m (\delta + \alpha\omega(\delta))
\end{aligned}$$

where N is the total number of functions. Since N and γ are fixed constants, we can group all constants into a single term $c_3(\gamma)$, thus obtaining the thesis:

$$\leq c_3(\gamma)(\alpha\epsilon + m\delta + m\alpha\omega(\delta))$$

This completes the proof. $\square$

2.4 Elimination of dependent terms

On a closed and bounded set D, we consider the space of linear superpositions of the form $\sum_{k=1}^{m} p_k(x,y) f_k(q(x,y))$ $((x,y) \in D)$. Here the functions $\{p_k(x,y)\}$ and $q(x,y)$ are continuous and fixed, and $\{f_k(t)\}$ are arbitrary continuous functions of one variable. We assume that the function $q(x,y)$ is such that for any sequence $t_n \in q(D) \to t \in q(D)$, we have $\rho[e(q,t_n) \cap D, e(q,t) \cap D] \to 0$. Let us set

$$\lambda(t, D, q, p_1, ..., p_m) = \inf_{\{c_k\}} \sup_{(x,y) \in e(q,t) \cap D} \left| \sum_{k=1}^{m} c_k p_k(x,y) \right|,$$

where the infimum is taken over all sets of numbers $\{c_k\}$ for which $\max_k |c_k| = 1$. The function $\lambda(t, D, q, \{p_k\})$, as a function of t, is defined only on the set $q(D)$.

Lemma 4. *The function $\lambda(t, D, q, \{p_k\})$ depends continuously on t.*

Proof. The goal is to prove that the function $\lambda(t)$, which measures the degree of linear independence of the functions $\{p_k\}$ along a specific level curve $e(q,t)$, is continuous. The intuition is that if the level curve $e(q,t)$ moves continuously as t changes (a key assumption of the problem), then this measure of independence should also change continuously, without any abrupt jumps.

We will prove this by first establishing a uniform continuity property for the components that define $\lambda(t)$, and then showing how this property implies the continuity of $\lambda(t)$ itself.

Step 1: Defining the Core Components

Let's first define the function whose supremum is being taken. For any vector of coefficients $c = \{c_k\}$ such that $\max_k |c_k| = 1$, let's define the function:

$$S(c,t) = \sup_{(x,y) \in e(q,t) \cap D} \left| \sum_{k=1}^{m} c_k p_k(x,y) \right|$$

With this notation, the function $\lambda(t)$ is simply the infimum of $S(c,t)$ over all valid coefficient vectors c:

$$\lambda(t) = \inf_{\{c_k\}: \max_k |c_k| = 1} S(c,t)$$

Our proof will hinge on showing that $S(c,t)$ is continuous in t in a way that is **uniform** across all choices of c.

Step 2: Establishing Uniform Continuity of $S(c,t)$

The core of the argument lies in two key properties: the equicontinuity of the linear combinations and the continuous dependence of the level sets on t.

1. **Equicontinuity:** Consider the family of functions $\Phi_c(x,y) = \sum_{k=1}^{m} c_k p_k(x,y)$, where the coefficients are bounded, e.g., $\max_k |c_k| \leq 1$. Since each $p_k(x,y)$ is continuous on the compact set D, it is uniformly continuous. A finite linear combination

of uniformly continuous functions is also uniformly continuous. The family $\{\Phi_c\}$ is **equicontinuous** on D. This means that for any $\epsilon > 0$, there exists a single $\delta_{\text{equi}} > 0$ such that for **any** function Φ_c in the family, if $\rho(z_1, z_2) < \delta_{\text{equi}}$, then $|\Phi_c(z_1) - \Phi_c(z_2)| < \epsilon/2$. The same δ_{equi} works for all c.

2. **Continuous Dependence of Level Sets:** The problem assumes that if $t_n \to t$, then $\rho[e(q, t_n) \cap D, e(q, t) \cap D] \to 0$. This is continuity in the Hausdorff distance. It implies that for any $\delta_{\text{equi}} > 0$, we can find a $\delta_t > 0$ such that if $|t_1 - t_2| < \delta_t$, the Hausdorff distance between the level sets $E_1 = e(q, t_1) \cap D$ and $E_2 = e(q, t_2) \cap D$ is less than δ_{equi}.

Now, let's combine these two facts. Let $\epsilon > 0$ be given. From equicontinuity, we find the corresponding δ_{equi}. From the level set continuity, we find the corresponding δ_t. Let $|t_1 - t_2| < \delta_t$. We want to show that $|S(c, t_1) - S(c, t_2)| < \epsilon$ for any c.

Let z_1 be any point in E_1. Since the Hausdorff distance $\rho(E_1, E_2) < \delta_{\text{equi}}$, there must exist a point $z_2 \in E_2$ such that $\rho(z_1, z_2) < \delta_{\text{equi}}$. By equicontinuity, this means $|\Phi_c(z_1) - \Phi_c(z_2)| < \epsilon/2$. Using the triangle inequality, $|\Phi_c(z_1)| \leq |\Phi_c(z_2)| + \epsilon/2$. Since $z_2 \in E_2$, we know $|\Phi_c(z_2)| \leq \sup_{z \in E_2} |\Phi_c(z)| = S(c, t_2)$. So, $|\Phi_c(z_1)| \leq S(c, t_2) + \epsilon/2$. This holds for any $z_1 \in E_1$, so we can take the supremum over z_1:

$$S(c, t_1) = \sup_{z_1 \in E_1} |\Phi_c(z_1)| \leq S(c, t_2) + \epsilon/2$$

By a symmetric argument (starting with a point in E_2), we get:

$$S(c, t_2) \leq S(c, t_1) + \epsilon/2$$

Combining these two inequalities gives $|S(c, t_1) - S(c, t_2)| \leq \epsilon/2 < \epsilon$. Crucially, this holds for **any** choice of c (with $\max_k |c_k| \leq 1$), because we used the single δ_{equi} from equicontinuity.

Step 3: Proving the Continuity of $\lambda(t)$

Now that we have established that $S(c, t)$ is continuous in t uniformly for all c, proving the continuity of $\lambda(t) = \inf_c S(c, t)$ is straightforward.

Let $\epsilon > 0$. From Step 2, there exists a $\delta > 0$ such that if $|t_1 - t_2| < \delta$, then $|S(c, t_1) - S(c, t_2)| < \epsilon$ for all c. This is equivalent to the two inequalities:

$$\begin{aligned} S(c, t_2) &< S(c, t_1) + \epsilon \quad \text{for all } c \\ S(c, t_1) &< S(c, t_2) + \epsilon \quad \text{for all } c \end{aligned}$$

Let's take the infimum over all c in the first inequality:

$$\inf_c S(c, t_2) \leq \inf_c (S(c, t_1) + \epsilon)$$

Since ϵ is a constant with respect to c, we have $\inf_c (S(c, t_1) + \epsilon) = (\inf_c S(c, t_1)) + \epsilon$. This gives:

$$\lambda(t_2) \leq \lambda(t_1) + \epsilon$$

Similarly, taking the infimum of the second inequality gives:

$$\lambda(t_1) \leq \lambda(t_2) + \epsilon$$

These two results combine to show that $|\lambda(t_1) - \lambda(t_2)| \leq \epsilon$ whenever $|t_1 - t_2| < \delta$. This is the definition of continuity for $\lambda(t)$.

This completes the proof. $\square$

Definition 2.20 (ϵ-net in a metric space)**.** Let (X, d) be a metric space. A subset $A \subset X$ is called an *ε-net* in X if the following condition holds:

$$\forall x \in X, \quad \exists a \in A \text{ such that } d(x, a) < \varepsilon.$$

In other words, every point in X is at a distance less than ε from at least one point in A.

Lemma 5. *The function $\lambda(t, D, q, \{p_k\})$ depends continuously on D in the sense that there exists a function $\mu(\epsilon) \to 0$ as $\epsilon \to 0$ with the property: if the set $D_\epsilon \subset D$ is such that, for every t, $D_\epsilon \cap e(q, t)$ forms an ϵ-net in the set $e(q, t) \cap D$, then*

$$\max_{t \in q(D)} |\lambda(t, D, q, \{p_k\}) - \lambda(t, D_\epsilon, q, \{p_k\})| \leq \mu(\epsilon).$$

Proof. The lemma's goal is to show that the function $\lambda(t)$ is robust to small changes in its domain of evaluation. If we replace the full set D with a dense subset D_ϵ (specifically, an ϵ-net), the value of $\lambda(t)$ should not change significantly. The proof relies on the uniform continuity (equicontinuity) of the functions being analyzed.

Step 1: Comparing Supremums on D and D_ϵ

Let's first analyze how the supremum of a single linear combination is affected by restricting the domain from D to D_ϵ. Let $\Phi_c(x, y) = \sum_{k=1}^{m} c_k p_k(x, y)$ be one such combination. We will compare:

$$S(c, D, t) = \sup_{(x,y) \in e(q,t) \cap D} |\Phi_c(x, y)| \quad \text{and} \quad S(c, D_\epsilon, t) = \sup_{(x,y) \in e(q,t) \cap D_\epsilon} |\Phi_c(x, y)|$$

1. **Equicontinuity:** As established previously, the family of all functions $\{\Phi_c\}$ (where $\max_k |c_k| \leq 1$) is equicontinuous on the compact set D. This is the crucial property. It means that there is a single function $\mu(\epsilon)$, which tends to 0 as $\epsilon \to 0$, such that for **any** choice of c, if $\rho(z_1, z_2) < \epsilon$, then $|\Phi_c(z_1) - \Phi_c(z_2)| < \mu(\epsilon)$.

2. **ϵ-net Property:** We are given that for any t, the set $E_\epsilon = e(q, t) \cap D_\epsilon$ is an ϵ-net for the set $E = e(q, t) \cap D$. By definition, this means that for any point $z \in E$, there exists a point $z_\epsilon \in E_\epsilon$ such that the distance $\rho(z, z_\epsilon) < \epsilon$.

3. **Combining the Properties:** Let's show that $S(c, D, t)$ is close to $S(c, D_\epsilon, t)$. Since $D_\epsilon \subset D$, it is clear that the supremum over the smaller set cannot exceed the supremum over the larger set:

   $$S(c, D_\epsilon, t) \leq S(c, D, t) \implies S(c, D, t) - S(c, D_\epsilon, t) \geq 0$$

   For the other direction, let $z_{\max} \in E$ be a point where $|\Phi_c(x, y)|$ is maximized (or comes arbitrarily close to its supremum). Since E_ϵ is an ϵ-net, there is a point $z_\epsilon \in E_\epsilon$ with $\rho(z_{\max}, z_\epsilon) < \epsilon$. By equicontinuity, we have $|\Phi_c(z_{\max}) - \Phi_c(z_\epsilon)| < \mu(\epsilon)$. Using the reverse triangle inequality, $|\Phi_c(z_{\max})| < |\Phi_c(z_\epsilon)| + \mu(\epsilon)$. But $|\Phi_c(z_\epsilon)|$ is at most the supremum over all points in E_ϵ. So,

   $$S(c, D, t) = |\Phi_c(z_{\max})| < \sup_{z' \in E_\epsilon} |\Phi_c(z')| + \mu(\epsilon) = S(c, D_\epsilon, t) + \mu(\epsilon)$$

Combining these two inequalities, we have shown that for any c and any t:

$$0 \leq S(c, D, t) - S(c, D_\epsilon, t) < \mu(\epsilon)$$

The function $\mu(\epsilon)$ is independent of c and t due to the uniform nature of equicontinuity.

Step 2: Transition to the Infimum (λ)

Now we use the uniform bound from Step 1 to prove the result for $\lambda(t)$, which is the infimum over all c.

Let $\lambda(t, D)$ and $\lambda(t, D_\epsilon)$ denote the values of the function for the respective domains. From the fact that $S(c, D_\epsilon, t) \leq S(c, D, t)$ for all c, it follows directly by taking the infimum that:

$$\lambda(t, D_\epsilon) \leq \lambda(t, D) \quad \Longrightarrow \quad \lambda(t, D) - \lambda(t, D_\epsilon) \geq 0$$

This provides the lower bound. For the upper bound, we choose a specific coefficient vector. Since the set of coefficients is compact and the supremum function is continuous with respect to them, there exists a coefficient vector c^ϵ that realizes the infimum for the domain D_ϵ:

$$\lambda(t, D_\epsilon) = S(c^\epsilon, D_\epsilon, t) = \sup_{(x,y) \in e(q,t) \cap D_\epsilon} \left| \sum_{k=1}^{m} c_k^\epsilon p_k(x, y) \right|$$

Now, we use this specific c^ϵ to bound $\lambda(t, D)$. By definition, $\lambda(t, D)$ is the infimum over *all* possible coefficient vectors, so it must be less than or equal to the value for this particular vector c^ϵ:

$$\lambda(t, D) \leq S(c^\epsilon, D, t)$$

Now we can form the difference we want to bound:

$$\lambda(t, D) - \lambda(t, D_\epsilon) \leq S(c^\epsilon, D, t) - S(c^\epsilon, D_\epsilon, t)$$

But from Step 1, we know that this difference is less than $\mu(\epsilon)$, regardless of the choice of coefficients. In particular, it holds for our chosen c^ϵ.

$$S(c^\epsilon, D, t) - S(c^\epsilon, D_\epsilon, t) < \mu(\epsilon)$$

Combining everything, we have shown that for any given t:

$$0 \leq \lambda(t, D) - \lambda(t, D_\epsilon) < \mu(\epsilon)$$

Since this inequality holds for all $t \in q(D)$ and the bound $\mu(\epsilon)$ does not depend on t, we can take the maximum over all t to obtain the final result.

This completes the proof. $\square$

Lemma 6. *Let F be a closed set on the t-axis; $F \subset q(D)$. For each $t \in F$ let us suppose that there exists one and only one system of numbers $\{c_k\}$ ($\max_k |c_k| = 1$) such that $\sum_{k=1}^m c_k p_k(x, y) \equiv 0$ on the set $e(q, t) \cap D$. Then each of the functions $\{c_k(t)\}$ depends continuously on t on the set F.*

Proof. The goal is to show that the functions $c_k(t)$, which give the coefficients of the unique null linear combination for a given level curve t, are continuous. The proof relies on the sequential definition of continuity: we will show that for any sequence $t_n \to t$, the corresponding sequence of coefficients $c_k(t_n)$ must converge to $c_k(t)$. The **uniqueness** of the coefficients for any given t is the central hypothesis that makes the proof work.

Step 1: Setting up the Proof using Sequential Continuity

To prove that a function is continuous on a set F, it is sufficient to show that for any sequence $\{t_n\}$ in F that converges to a point $t \in F$, the sequence of function values $\{c_k(t_n)\}$ converges to $c_k(t)$.

Let's take an arbitrary sequence $t_n \in F$ such that $t_n \to t$, where $t \in F$. We consider the corresponding sequence of coefficient vectors, $\{c(t_n)\}$. Each vector is normalized such that $\max_k |c_k(t_n)| = 1$. This means the sequence of vectors is bounded and lies within the unit cube in $\mathbb{R}^m$. By the Bolzano-Weierstrass theorem, every bounded sequence in $\mathbb{R}^m$ has at least one convergent subsequence. Our goal is to show that all convergent subsequences converge to the same point, which must be $c(t)$.

Step 2: Analyzing Limit Points with limsup and liminf

Let's analyze the limit points of the sequence of coefficients $\{c_k(t_n)\}$ for a fixed k. Since the sequence is bounded (by 1), its limit superior and limit inferior must exist and be finite. Let:

$$\tilde{c}_k = \limsup_{n\to\infty} c_k(t_n) \quad \text{and} \quad \tilde{\tilde{c}}_k = \liminf_{n\to\infty} c_k(t_n)$$

By definition of limsup, there exists a subsequence $\{t_{n_j}\}$ of $\{t_n\}$ such that $\lim_{j\to\infty} c_k(t_{n_j}) = \tilde{c}_k$ for all k. Similarly, there is another subsequence (which we won't need to name explicitly) that converges to the liminf. Our strategy is to show that both the vector of limsups, $\tilde{c} = \{\tilde{c}_k\}$, and the vector of liminfs, $\tilde{\tilde{c}} = \{\tilde{\tilde{c}}_k\}$, satisfy the null-combination property at the limit point t.

Step 3: Taking the Limit of the Null-Combination Identity

For each term in our subsequence $\{t_{n_j}\}$, the defining property holds:

$$\sum_{k=1}^m c_k(t_{n_j}) p_k(x, y) = 0, \quad \text{for all } (x, y) \in e(q, t_{n_j}) \cap D$$

Now we take the limit as $j \to \infty$. We analyze what happens to each part of this equation:

1. The coefficients converge: $\lim_{j\to\infty} c_k(t_{n_j}) = \tilde{c}_k$.
2. The domain of evaluation converges: The problem statement assumes that the level sets depend continuously on t in the Hausdorff distance, so $\rho[e(q, t_{n_j}) \cap D, e(q, t) \cap D] \to 0$.
3. The functions $p_k(x, y)$ are continuous.

Let z be any point on the limit level curve, $z \in e(q, t) \cap D$. Due to the convergence of the level sets, there exists a sequence of points $z_j \in e(q, t_{n_j}) \cap D$ such that $z_j \to z$. For each j, the identity holds at z_j: $\sum_{k=1}^{m} c_k(t_{n_j})p_k(z_j) = 0$. Taking the limit as $j \to \infty$ and using the continuity of the functions p_k, we get:

$$\sum_{k=1}^{m} \left(\lim_{j\to\infty} c_k(t_{n_j})\right) \left(\lim_{j\to\infty} p_k(z_j)\right) = 0 \implies \sum_{k=1}^{m} \tilde{c}_k p_k(z) = 0$$

Since this holds for any point z on the limit level curve, we have established that the vector of limsups $\tilde{c}$ satisfies the null-combination property:

$$\sum_{k=1}^{m} \tilde{c}_k p_k(x, y) \equiv 0 \quad \text{on the set } e(q, t) \cap D$$

An identical argument holds for the vector of liminfs $\tilde{\tilde{c}}$.

Step 4: Invoking the Uniqueness Hypothesis

We have now found two systems of coefficients, $\tilde{c} = \{\tilde{c}_k\}$ and $\tilde{\tilde{c}} = \{\tilde{\tilde{c}}_k\}$, that make the linear combination identically zero on the set $e(q, t) \cap D$. At this point, we invoke the central hypothesis of the lemma: for any given $t \in F$, there is **one and only one** such system of coefficients.

Therefore, both of our derived systems must be identical to the unique system associated with t, which is $\{c_k(t)\}$.

$$\tilde{c}_k = c_k(t) \quad \text{and} \quad \tilde{\tilde{c}}_k = c_k(t) \quad \text{for all } k = 1, \ldots, m.$$

We have shown that for any k,

$$\limsup_{n\to\infty} c_k(t_n) = \liminf_{n\to\infty} c_k(t_n) = c_k(t).$$

A well-known result from analysis states that if the limit superior and limit inferior of a bounded sequence are equal, then the sequence converges to that common value. Therefore, $\lim_{n\to\infty} c_k(t_n) = c_k(t)$.

Since our initial choice of a convergent sequence $t_n \to t$ was arbitrary, we have proven that the functions $c_k(t)$ are continuous on the set F.

This completes the proof. □

Lemma 7. *Suppose that $\lambda(t, D, q, \{p_k\}) \equiv 0$ on a non-empty portion δ of the set $q(D)$. Then there exists a non-empty portion $\delta^* \subset \delta$ and an index l such that for any continuous functions $\{f_k(t)\}$, there exist continuous functions $\{f_k^*(t)\}$ such that*

$$\sum_{k \neq l} f_k^*(q(x,y))p_k(x,y) = \sum_{k=1}^{m} f_k(q(x,y))p_k(x,y)$$

on the set $q^{-1}(\delta^*) \cap D$.

Proof. The lemma states that if the functions $\{p_k\}$ are linearly dependent on a "thick" set of level curves (i.e., for t in an interval δ), then at least one of these functions is redundant. We can eliminate it from the superposition by appropriately modifying the other coefficient functions $\{f_k\}$. We will prove this by induction on m, the number of functions in the set $\{p_k\}$.

Base Case (m=1): The statement is trivial. If $\lambda(t, D, q, \{p_1\}) = 0$, this means $\sup |p_1(x,y)| = 0$ on the level curve $e(q,t)$. So, $p_1(x,y) \equiv 0$ on $q^{-1}(\delta) \cap D$. The sum is $f_1(q)p_1(x,y)$, which is already zero. The sum with the $l = 1$ term eliminated is an empty sum, which is also zero. So the equality holds.

Inductive Hypothesis: Assume the lemma is true for any collection of $m-1$ functions.

Inductive Step: Let's define δ_k as the set of all $t \in \delta$ where the smaller collection of functions $\{p_1, \ldots, p_{k-1}, p_{k+1}, \ldots, p_m\}$ is linearly dependent. That is:

$$\delta_k = \{t \in \delta \mid \lambda(t, D, q, \{p_i\}_{i \neq k}) = 0\}$$

By Lemma 4, each set δ_k is a closed set. We now consider two exhaustive cases based on the nature of these sets.

Case 1: One of the sets δ_k contains an interval

Suppose that for some index k (say, $k = 1$), the set δ_1 contains a non-empty interval, which we'll call $\delta_1' \subset \delta_1$.

Interpretation: On this entire interval of level curves corresponding to δ_1', the subset of functions $\{p_2, \ldots, p_m\}$ is already linearly dependent. This is a problem involving only $m-1$ functions.

Applying the Inductive Hypothesis: Since the lemma holds for $m-1$ functions, we can apply it to the set $\{p_2, \ldots, p_m\}$ on the interval δ_1'. This means there exists a sub-interval $\delta^* \subset \delta_1'$ and an index $l \in \{2, \ldots, m\}$ such that we can eliminate p_l from any superposition involving only $\{p_2, \ldots, p_m\}$. Specifically, for the sum $\sum_{i=2}^{m} f_i(q)p_i(x,y)$, there exist new functions $\{f_i^*\}_{i \neq 1,l}$ such that:

$$\sum_{i=2}^{m} f_i(q(x,y))p_i(x,y) = \sum_{i \neq 1,l} f_i^*(q(x,y))p_i(x,y) \quad \text{on } q^{-1}(\delta^*) \cap D$$

Now, let's consider our original sum with all m terms:

$$\sum_{i=1}^{m} f_i(q)p_i = f_1(q)p_1 + \sum_{i=2}^{m} f_i(q)p_i$$

Substitute the result from the inductive hypothesis:

$$= f_1(q)p_1 + \sum_{i \neq 1,l} f_i^*(q)p_i$$

To match the form required by the lemma, we can simply define a new function $f_1^*(t) = f_1(t)$. The expression then becomes:

$$= f_1^*(q)p_1 + \sum_{i \neq 1,l} f_i^*(q)p_i = \sum_{i \neq l} f_i^*(q)p_i$$

We have successfully eliminated the function p_l from the original sum. Thus, the lemma holds in this case.

Case 2: No set δ_k contains an interval

Suppose that for every $k \in \{1, \ldots, m\}$, the set δ_k is nowhere dense (it's a closed set with no interior). The union of a finite number of nowhere dense sets, $\bigcup_{k=1}^m \delta_k$, is also nowhere dense.

Interpretation: We can find a "clean" sub-interval $\delta^* \subset \delta$ that is completely disjoint from all the "problematic" sets δ_k.

$$\delta^* \subset \delta \setminus \bigcup_{k=1}^m \delta_k$$

For any $t \in \delta^*$, we have two conditions:

1. $\lambda(t, D, q, \{p_k\}_{k=1}^m) = 0$ (since $\delta^* \subset \delta$). The full set is linearly dependent.
2. $\lambda(t, D, q, \{p_i\}_{i \neq k}) \neq 0$ for any k (since $t \notin \delta_k$). No proper subset of $m-1$ functions is linearly dependent.

The crucial consequence is that for any $t \in \delta^*$, the null linear combination $\sum_{k=1}^m c_k(t)p_k(x,y) = 0$ must have all coefficients $c_k(t)$ be non-zero. If any $c_k(t)$ were zero, it would imply a linear dependence among the other $m-1$ functions, which would mean $t \in \delta_k$, a contradiction.

Uniqueness and Continuity of Coefficients: We now show that for each $t \in \delta^*$, the set of coefficients $\{c_k(t)\}$ is unique (up to a global scaling factor, fixed by the normalization $\max_k |c_k| = 1$). Assume, for contradiction, that two distinct sets of coefficients, $\{c_k\}$ and $\{c'_k\}$, exist for the same t.

$$\sum c_k p_k = 0 \quad \text{and} \quad \sum c'_k p_k = 0$$

Since all coefficients are non-zero, we can eliminate p_1:

$$c'_1 \sum c_k p_k - c_1 \sum c'_k p_k = 0 \implies \sum_{k=2}^m (c'_1 c_k - c_1 c'_k) p_k = 0$$

Since $\{c_k\}$ and $\{c'_k\}$ are distinct, at least one of the new coefficients $(c'_1 c_k - c_1 c'_k)$ must be non-zero. This equation shows a linear dependence among $\{p_2, \ldots, p_m\}$, which implies

$t \in \delta_1$. This contradicts our choice of δ^*. Therefore, the coefficients $\{c_k(t)\}$ are uniquely determined for each $t \in \delta^*$.

By Lemma 6, these unique functions $c_k(t)$ must be continuous on δ^*.

Eliminating a Function: Since each $c_k(t)$ is continuous and non-zero on δ^*, we can express one function, say p_1, in terms of the others:

$$p_1(x,y) = \sum_{k=2}^{m} \left(-\frac{c_k(q(x,y))}{c_1(q(x,y))} \right) p_k(x,y) \quad \text{for } (x,y) \in q^{-1}(\delta^*) \cap D$$

Now we substitute this into the original superposition:

$$\begin{aligned}
\sum_{k=1}^{m} f_k(q) p_k(x,y) &= f_1(q) p_1(x,y) + \sum_{k=2}^{m} f_k(q) p_k(x,y) \\
&= f_1(q) \left[\sum_{k=2}^{m} -\frac{c_k(q)}{c_1(q)} p_k(x,y) \right] + \sum_{k=2}^{m} f_k(q) p_k(x,y) \\
&= \sum_{k=2}^{m} \left[f_k(q) - f_1(q) \frac{c_k(q)}{c_1(q)} \right] p_k(x,y)
\end{aligned}$$

We define the new continuous functions $f_k^*(t)$ for $k \in \{2, \ldots, m\}$ as:

$$f_k^*(t) = f_k(t) - f_1(t) \frac{c_k(t)}{c_1(t)}$$

The expression becomes $\sum_{k=2}^{m} f_k^*(q) p_k(x,y)$, which has successfully eliminated p_1 (our chosen index $l = 1$).

Both cases lead to the desired conclusion, completing the inductive step. This proves the lemma. □

2.5 Reduction of linear superpositions to a form with independent terms

Let us fix the continuous functions $p_i^k(x,y)$ and the continuously differentiable functions $q_i(x,y)$ $(i = 0,1,2,...,n;\ k = 1,2,...,m_i)$, $n \geq 2$, where $\{q_i(x,y)\}$ satisfy in D conditions (1) and (3) of Lemma 2, and consider in D superpositions of the form

$$\sum_{i=0}^{n}\sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i(x,y)),$$

where $\{f_i^k(t)\}$ are arbitrary continuous functions of one variable. We call a closed and bounded region $G \subset D$ polyhedral if the boundary of G consists of a finite number of mutually non-intersecting simple closed contours which are unions of a finite number of segments of level curves of the functions $q_i(x,y)$ $(i = 1,2,...,n)$. Let $G \subset D$ be a polyhedral region. We denote by Γ_i the set of those $t \in q_i(G)$ for which the set $e(q_i,t) \cap G$ contains a segment of a level curve belonging to the boundary of G. For each i the set Γ_i consists of a finite number of points. By property (1) of the functions $\{q_i(x,y)\}$, for each i and for all points $t_0 \in q_i(G) \setminus \Gamma_i$ there exists $\lim_{t\to t_0} e(q_i,t) = e(q_i,t_0)$. If $t_0 \in \Gamma_i$, then the last assertion is not necessarily true, but in any case there exists $\lim_{t\to +t_0} e(q_i,t) \subset e(q_i,t_0)$ and $\lim_{t\to -t_0} e(q_i,t) \subset e(q_i,t_0)$, where the limit is taken over the points $t \in q_i(G)$. Here the limit is understood in the sense of the distance $\rho(e(q_i,t), e(q_i,t_0))$.

Lemma 8. *There exists a region $G \subset D$ and a system of numbers $\tau_i^k = 0$ or 1 $(i = 0,1,2,...,n;\ k = 1,2,...,m_i)$ such that*

(4) for each i and for any continuous functions $\{\phi_i^k(t)\}$, there exist continuous functions $\{f_i^k(t)\}$ such that in G

$$\sum_{k=1}^{m_i} p_i^k(x,y)\phi_i^k(q_i(x,y)) \equiv \sum_{k=1}^{m_i} \tau_i^k p_i^k(x,y) f_i^k(q_i(x,y));$$

(5) for any polyhedral region $G^* \subset G$ and any i, the set $\{t : \lambda(t, G^*, q_i, p_i^{k_1}, ..., p_i^{k_s}) = 0\}$ is nowhere dense in $q_i(G^*)$, where $k_1, ..., k_s$ is the set of all values of k for which $\tau_i^k = 1$.*

Proof. The purpose of this lemma is to simplify the superposition by identifying and removing redundant terms within each family i. The result is a "basis" set of functions (marked with $\tau_i^k = 1$) that are as linearly independent as possible across the level curves. The proof proceeds by constructing this basis for each family i.

Part 1: The Trivial Case ($i = 0$)

The family $i = 0$ is special because the function $q_0(x,y)$ is constant. This means that $q_0(D)$ is just a single point, so the functions $f_0^k(q_0(x,y))$ are just constants. The superposition term for $i = 0$ is simply a linear combination of the fixed functions $\{p_0^k(x,y)\}$.

1. **Finding a Basis:** This is a standard problem in linear algebra. We consider the vector space spanned by the functions $\{p_0^k(x,y)\}_{k=1}^{m_0}$ over a region $G_0 \subset D$. We can find a basis for this space, say $\{p_0^{k_1}, p_0^{k_2}, \ldots, p_0^{k_s}\}$.

2. **Setting the Flags τ_0^k:** We set $\tau_0^k = 1$ if p_0^k is in our chosen basis, and $\tau_0^k = 0$ otherwise.

3. **Verifying Condition (4):** By the definition of a basis, any function in the original set $\{p_0^k\}$ can be written as a linear combination of the basis functions. Therefore, any superposition $\sum_{k=1}^{m_0} p_0^k \phi_0^k$ can be rewritten as a sum involving only the basis functions, satisfying condition (4).

4. **Verifying Condition (5*):** By definition, basis functions are linearly independent. This means that for the set $\{p_0^{k_1}, \ldots, p_0^{k_s}\}$, the only way to make the combination $\sum_{j=1}^{s} c_j p_0^{k_j}$ identically zero is if all coefficients c_j are zero. Consequently, $\lambda(t, G^*, q_0, \{p_0^{k_j}\})$ will be strictly greater than zero. The set of points where it is zero is empty, which is a nowhere dense set.

This handles the $i = 0$ case and gives us an initial region G_0.

Part 2: The Iterative Reduction for Families $i > 0$

For each family $i \in \{1, \ldots, n\}$, we perform an iterative process to eliminate dependent terms until the remaining set satisfies condition (5*).

1. **The Goal:** Start with all $\tau_i^k = 1$. Our goal is to switch some of these to 0 until the remaining set of functions $\{p_i^k \mid \tau_i^k = 1\}$ satisfies condition (5*).

2. **The Test:** At each step, we check if the current set of "active" functions $\{p_i^k\}$ satisfies (5*). That is, we check if the set of t-values where they become linearly dependent, $Z = \{t : \lambda(t) = 0\}$, is nowhere dense.

3. **What if the Test Fails?:** If the test fails, the set Z is **not** nowhere dense. The function $\lambda(t)$ is continuous except for a finite number of jump discontinuities (on any polyhedral region). A closed set that is not nowhere dense must contain an interval. Therefore, if the test fails, there must be a non-empty interval δ where $\lambda(t) \equiv 0$.

4. **Applying Lemma 7:** We are now in the exact situation described by the previous lemma (Lemma 7). This lemma is the engine of our reduction. It tells us that if $\lambda(t) \equiv 0$ on an interval δ, then there exists a sub-interval δ^* and an index l such that the function p_i^l is redundant. It can be expressed in terms of the other functions in the set, and we can eliminate it from the superposition within the corresponding spatial region $q_i^{-1}(\delta^*) \cap D$.

5. **The Iteration:** We perform this elimination by setting the corresponding flag τ_i^l to 0. We have now reduced the number of active functions by one. We then repeat the test (Step 2) with this smaller set.

Part 3: Termination and the Final Region G

This iterative process must eventually stop for each family i. We start with a finite number (m_i) of functions, and in each step where the test fails, we eliminate one. The process for a given family i terminates when the remaining set of active functions finally passes the test, meaning condition (5*) is satisfied.

By performing this reduction for all families $i = 1, \ldots, n$, we may need to shrink our working domain. The eliminations are guaranteed to work on specific regions, and we find a common subregion $G \subset G_0 \subset D$ where all the final conditions hold simultaneously.

The final set of flags $\{\tau_i^k\}$ and the region G satisfy the lemma:

- **Condition (4) is satisfied** because we constructed the flags by eliminating terms that were expressible through the others. The active terms ($\tau_i^k = 1$) form a basis for the full set.
- **Condition (5*) is satisfied** because the iterative process for each family was designed to terminate precisely when this condition was met.

This completes the proof. □

A region $G \subset D$ is called regular if, first, it is polyhedral and, second, there exists a number $\gamma_G > 0$ such that for each $i > 0$ and each $t \in q_i(G)$ the set $e(q_i, t) \cap G$ is the union of a finite number of simple arcs, each of which has length not less than γ_G. A point A on the boundary of the polyhedral region G is called a vertex if it belongs simultaneously to two segments of the level curves of $q_i(x, y)$ and $q_j(x, y)$ $(i \neq j)$ on the boundary of G. Every polyhedral region has a finite number of vertices.

Lemma 9. *For any polyhedral region G and any neighborhood U of the vertices of this region, we can construct a regular region $G^* \subset G$ such that $G \setminus U \subset G^*$.*

Proof. The goal of this lemma is to show that any "polyhedral" region can be slightly modified to become "regular". A regular region is well-behaved in the sense that any level curve slicing through it does so along an arc of substantial length, avoiding infinitesimally small intersections. The problem areas are the "vertices" (corners) of the region. The proof provides a constructive algorithm to "shave off" these corners in a precise way to fix the problem.

The proof works by defining a measure of "non-regularity" and showing how to iteratively reduce this measure to zero.

Step 1: Defining the Problem and a Metric for "Non-Regularity"

A region G might fail to be regular if, for some function q_i, a level curve $e(q_i, t)$ intersects G in an arc of very short (or zero) length. This typically happens at a vertex A_m, where the level curve just "grazes" the region at that corner point without properly entering it.

To quantify this, we define a "grazing count" for each vertex A_m:

- Let $k_m(A_m)$ be the number of functions q_i whose level curve through A_m is problematic, meaning it touches G only at A_m within a small neighborhood U_m of the vertex.

We then define an overall measure of non-regularity for the entire region G:

$$k(G) = \max_m k_m(A_m)$$

This value represents the maximum number of grazing level curves at the "worst" vertex of the region. The region G is regular if and only if $k(G) = 0$. If $k(G) > 0$, our task is to reduce this number.

Step 2: The Surgical "Cut" to Fix a Problematic Vertex

Suppose $k(G) > 0$. This means there is at least one vertex, let's call it A_m, where $k_m(A_m) > 0$. Let q_{im} be one of the functions whose level curve is grazing A_m.

The core of the proof is a surgical modification of the region G near this vertex. We "cut off" a small slice of G near A_m. The slice we remove is the set of points (x, y) within the neighborhood U_m that are very close to the grazing level curve, defined by:

$$G^*_{im} = G \setminus \{(x, y) \in U_m : |q_{im}(x, y) - q_{im}(A_m)| < \epsilon\}$$

For a sufficiently small $\epsilon > 0$, this operation has a predictable geometric effect:

- The original vertex A_m is removed from the boundary of the region.
- Two new vertices, A'_m and A''_m, are created where the boundaries of the cut intersect the old boundary of G.
- The crucial outcome: The function q_{im} is no longer a "grazing" function at the new vertices. The number of grazing functions at A'_m and A''_m is now one less than it was at A_m. That is, $k(A'_m) = k(A_m) - 1$ and $k(A''_m) = k(A_m) - 1$.

We have successfully reduced the "grazing count" at the location of the old vertex.

Step 3: The Iterative Reduction of $k(G)$

We can apply the surgical cut from Step 2 to all vertices that exhibit the maximum grazing count, $k(G)$. We define a new region G_1^* by taking the intersection of all such cuts:

$$G_1^* = \bigcap_{m:k_m>0} G_{im}^*$$

The new region G_1^* is still polyhedral, but its "worst" vertices are now less problematic. The maximum grazing count for this new region has been reduced by one:

$$k(G_1^*) = k(G) - 1$$

We can repeat this entire procedure. At each step, we identify the worst vertices of the current region and apply the cutting procedure, which reduces the maximum grazing count by one. Since we start with a finite integer $k(G)$, after exactly $k(G)$ repetitions of this process, we will arrive at a final polyhedral region, which we call G^*, satisfying:

$$k(G^*) = 0$$

Step 4: Conclusion

A region G^* with $k(G^*) = 0$ has no grazing level curves at any of its vertices. This implies that any level curve intersecting G^* must do so along arcs of non-zero length. Since the region is compact, there must be a minimum length for these arcs, so G^* is a regular region.

By construction, all the cuts were made within the specified neighborhood U of the original vertices. This guarantees that the part of the original region that was already far from the vertices remains untouched. Therefore, $G \setminus U \subset G^*$.

We have successfully constructed a regular region G^* that meets all the conditions of the lemma. This completes the proof. $\square$

Lemma 10. *There exists a set $G \subset D$, a number $\lambda > 0$, and a set of numbers $\tau_i^k = 0$ or 1 ($i = 0, 1, ..., n$; $k = 1, 2, ..., m_i$) such that condition (4) of Lemma 8 is satisfied, and also the conditions*

(5) for any i and $t \in q_i(G)$ and for any functions $\{f_i^k(t)\}$

$$\max_{(x,y)\in e(q_i,t)\cap G} \left| \sum_{k=1}^{m_i} \tau_i^k p_i^k(x,y) f_i^k(q_i(x,y)) \right| \geq \lambda \max_k |\tau_i^k f_i^k(t)|;$$

(6) G is a regular region.

Proof. The goal of this lemma is to find a single "gold-standard" region G where our simplified superposition is both geometrically well-behaved (**regular**) and robustly linearly independent. Condition (5) is a strong form of linear independence: it states that the functions $\{p_i^k\}$ are not just independent, but "uniformly" independent, with the measure of their independence (λ) bounded away from zero across the entire region.

The proof is a sophisticated **inductive construction**. It builds a sequence of increasingly refined regions, $G_0 \supset G_1 \supset \cdots \supset G_n = G$, where each region G_i is made "good" for the family of functions i without spoiling the "goodness" already achieved for families $j < i$.

Step 1: The Starting Point (Using Lemma 8)

We begin by applying the previous lemma (8). It guarantees the existence of a region $G^* \subset D$ and a set of flags τ_i^k that already satisfy two important properties:

1. Condition (4) holds: The superposition can be written using a minimal "basis" set of functions $\{p_i^k\}$ (those with $\tau_i^k = 1$).

2. A weak independence condition holds: For this basis set, the set of t-values where the functions become linearly dependent ($\lambda(t) = 0$) is **nowhere dense**.

For simplicity in the following steps, we will assume that the flags have already been applied and we are only working with the basis functions (i.e., we assume all $\tau_i^k = 1$ for the set of functions under consideration). Our task is to strengthen the "nowhere dense" condition to a "uniformly positive" condition.

Step 2: The Inductive Construction Strategy

We will construct a sequence of regular regions $G_0, G_1, \ldots, G_n$. The process is iterative:

1. **Base Case:** We start with an arbitrary regular region $G_0 \subset G^*$. We can find one using Lemma 9.

2. **Inductive Hypothesis:** Assume we have constructed a regular region G_{i-1} such that for all families $j < i$, the strong independence condition (5) holds. That is, there's a $\lambda_{i-1} > 0$ such that $\lambda(t, G_{i-1}, q_j, \ldots) > \lambda_{i-1}$ for all $j < i$.

3. **Inductive Step:** We will construct a new regular region $G_i \subset G_{i-1}$ such that the strong independence condition now holds for all families $j \leq i$.

The core of the proof lies in the careful construction of G_i from G_{i-1}.

Step 3: The "Prune and Regularize" Algorithm (Constructing G_i from G_{i-1})

This is a two-stage process for refining the region for family i.

Stage A: Pruning the Region Our current region G_{i-1} is problematic for family i because $\lambda(t, G_{i-1}, q_i, \dots)$ can get arbitrarily close to zero. We fix this by "pruning" away the parts of the region corresponding to these near-dependencies.

1. For a small $\delta > 0$, we define the "safe set" of t-values for family i: $\alpha_\delta = \{t \in q_i(G_{i-1}) \mid \lambda(t) > \delta\}$.
2. Since the "bad" set (where $\lambda \approx 0$) is nowhere dense, the "safe set" α_δ will make up the bulk of the interval $q_i(G_{i-1})$. We select the N longest components of this safe set, call this α_δ^N.
3. We create a new, pruned region G_{i-1}^* by keeping only the parts of G_{i-1} that correspond to these safe t-values: $G_{i-1}^* = G_{i-1} \cap q_i^{-1}(\overline{\alpha}_\delta^N)$.

By choosing δ small enough and N large enough, this new region G_{i-1}^* is now an excellent $\epsilon/2$**-net** for the original region G_{i-1}.

Stage B: Regularizing the Pruned Region The pruning process in Stage A creates a new polyhedral region G_{i-1}^*, but its new boundaries may have introduced non-regular vertices.

1. We identify the new vertices of G_{i-1}^* and a small neighborhood $U(\epsilon)$ around them.
2. We apply **Lemma 9** (the previous lemma). This allows us to find a **regular** region $G_i \subset G_{i-1}^*$ by "shaving off" the problematic corners inside $U(\epsilon)$.

The final region G_i for this step is now both regular and consists only of "safe" level curves for family i.

Step 4: Verifying the Properties of the New Region G_i

We must check that our new region G_i still satisfies the conditions for the previous families $j < i$. This is where the ϵ-net argument is crucial.

- Because G_i is an ϵ-net of G_{i-1}, **Lemma 5** tells us that the value of $\lambda(t, G_i, q_j, \dots)$ for $j < i$ is very close to the value of $\lambda(t, G_{i-1}, q_j, \dots)$. Since the value for G_{i-1} was safely above λ_{i-1}, for a small enough ϵ, the value for G_i will also be safely above a new, slightly smaller constant (e.g., $\lambda_{i-1}/2$).
- For the new family i, we constructed G_i from points where $\lambda(t) > \delta$. The regularization step might slightly decrease this, but it will still be bounded above a positive constant (e.g., $\delta/2$).

We have successfully constructed a new regular region G_i where the strong independence condition holds for all $j \leq i$.

We must verify that the newly constructed region G_i still satisfies the required conditions for all previously considered families $j < i$. At this stage the ϵ–net argument becomes essential.

Because G_i is an ϵ–net of G_{i-1}, Lemma 5 ensures that the quantity $\lambda(t, G_i, q_j, \dots)$, for every $j < i$, differs only slightly from the corresponding value $\lambda(t, G_{i-1}, q_j, \dots)$. Since the latter was bounded below by λ_{i-1}, choosing ϵ sufficiently small guarantees that the value associated with G_i remains bounded away from zero, for instance by a constant of order $\lambda_{i-1}/2$.

For the newly introduced family i, the region G_i was constructed from points satisfying $\lambda(t) > \delta$. The subsequent regularization may slightly decrease this bound, but the quantity remains strictly positive, for example larger than $\delta/2$.

Consequently, we obtain a new regular region G_i in which the strong independence condition is satisfied simultaneously for all indices $j \leq i$.

By repeating this inductive step for all $i = 1, \dots, n$, we finally arrive at a region $G = G_n$ that is regular and for which there exists a single $\lambda = \lambda_n > 0$ satisfying Condition (5) for all families. This completes the proof. $\square$

2.6 The set of linear superpositions in the space of continuous functions is closed

Theorem 2. *Suppose that continuous functions $p_m(x, y)$ and continuously differentiable functions $q_m(x, y)$ $(m = 1, 2, \ldots, N)$ are fixed. Then in any region D of the plane of the variables x, y, there exists a closed subregion $G \subset D$ such that the set of superpositions of the form*

$$\sum_{m=1}^{N} p_m(x, y) f_m(q_m(x, y)), \text{ where } \{f_m(t)\} \text{ are arbitrary continuous functions,}$$

is closed (in the uniform metric) in the set of all functions continuous on the set G.

Proof. **Step 1: Reduction and Setup of the Problem**

Before beginning the main proof, we apply the series of lemmas from the previous sections (specifically 2 and 10). Their combined results allow us to significantly simplify the problem. We can find a subregion $G \subset D$, establish positive constants $\gamma > 0$ and $\lambda > 0$, and re-index the original N functions.

This re-indexing sorts the functions $\{p_m, q_m\}$ into families, denoted by two indices (i, k), resulting in new sets $\{p_i^k(x, y)\}$ and $\{q_i^k(x, y)\}$. Note that $\sum m_i \leq N$ implies that some of the original functions may be discarded if they are found to be redundant.

Thanks to the lemmas, we can assert that our problem is reduced to analyzing a superposition on a region G that satisfies the following six powerful conditions:

Conditions (1), (2), (3) (from Lemma 2): These ensure the geometric regularity of the families of level curves.

- **(1) Non-degenerate gradients:** For $i > 0$, $0 < \gamma \leq |\operatorname{grad}[q_i^k]| \leq 1/\gamma$.
- **(2) Shared Geometry:** Functions within the same family i all share the same set of level curves.
- **(3) Transversality:** Level curves from *different* families $(i \neq j)$ always intersect at an angle $> \gamma$.

Condition (4') (from Lemma 8): This guarantees that our new, simplified superposition is just as expressive as the original one. *For any continuous functions $\{f_m(t)\}$, there exist continuous functions $\{f_i^k(t)\}$ such that on G:*

$$\sum_{m=1}^{N} p_m(x, y) f_m(q_m(x, y)) = \sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x, y) f_i^k(q_i^k(x, y));$$

Condition (5') (from Lemma 10): This is the crucial analytic property of **strong, uniform linear independence**. For each family i, the "basis" functions $\{p_i^k\}$ are robustly linearly independent on *every* level curve t. *For every i and $t \in q_i^k(G)$ and for any functions $\{f_i^k(t)\}$:*

$$\max_{(x,y) \in e(q_i^k, t) \cap G} \left| \sum_{k=1}^{m_i} p_i^k(x, y) f_i^k(q_i^k(x, y)) \right| \geq \lambda \max_k |f_i^k(t)|;$$

Condition (6') (from Lemma 10): This ensures the **geometric regularity of the domain**. *G is a regular region with respect to the functions $\{q_i^k(x,y)\}$.* This means that level curves do not have grazing, zero-length intersections.

The proof now proceeds by showing that any superposition satisfying these conditions forms a closed set. This is done via three key lemmas.

□

Lemma 11. *In the sets $\{q_i^k(G)\}$ we can select subsets consisting of a finite number of points $t_{i,j} \in q_i^k(G)$ $(i = 0, 1, 2, \ldots, n;\ j = 1, 2, \ldots, s_i)$ such that for any continuous functions $\{f_i^k(t)\}$ the following **stability inequality** holds:*

$$\max_{i,k} \max_{t \in q_i^k(G)} |f_i^k(t)| \quad \leq \quad c\left(\max_{(x,y)\in G} \left| \sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y)) \right| + \max_{h,i,j} |f_i^h(t_{i,j})| \right),$$

where c is a constant not depending on the functions $\{f_i^k(t)\}$.

Proof. The goal is to prove this stability estimate. It shows that the "size" of the input functions $\{f_i^k\}$ (their maximum value) is controlled by the "size" of the resulting superposition.

Step 1: Setup and Definitions

Let's define our key quantities to simplify notation:

- $\mu = \max_{i,k,t} |f_i^k(t)|$: The maximum value of any input function, which we want to bound.
- $\varepsilon_1 = \max_{(x,y)\in G} \left|\sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y))\right|$: The maximum value of the **full superposition** on the region G.
- $\varepsilon_2 = \max_{h,i,j} |f_i^h(t_{i,j})|$: The maximum value of the functions when evaluated only on the **finite, discrete set of reference points** $t_{i,j}$.

Our goal is to prove that $\mu \leq c(\varepsilon_1 + \varepsilon_2)$.

Since G is polyhedral (part of being regular), for each family i, the set of its level curves $e(q_i^k, t) \cap G$ forms a compact set. We can choose the finite set of points $\{t_{i,j}\}$ to be dense enough so that the corresponding level curves form a δ-net for all level curves in G. We will choose δ later.

Step 2: Locate the Maximum and Apply Strong Independence (5')

Let's assume the maximum μ is achieved by a specific function, say f_1^1, at a specific point $t = \alpha$.

$$|f_1^1(\alpha)| = \mu$$

(We use $f_1^1(q_1^1(a)) = \mu$, where $q_1^1(a) = \alpha$).

Now, we use **Condition (5')**, the strong linear independence, on the level curve $e(q_1^k, \alpha)$. This condition guarantees that the superposition for this family (let's call it φ_1) must also be large at some point a' on that level curve:

$$\varphi_1(x,y) = \sum_{k=1}^{m_1} p_1^k(x,y) f_1^k(q_1^k(x,y))$$

By (5'), there exists $a' \in e(q_1^k, \alpha) \cap G$ such that:

$$|\varphi_1(a')| \geq \lambda \max_k |f_1^k(\alpha)| \geq \lambda\mu$$

Step 3: Use Smoothness to Find a "Large Arc"

We know φ_1 is large at point a', but we need it to be large over a segment. By **Condition (6')**, G is regular, so the level curve at a' is a proper arc with length $\geq \gamma_G$. We can find a segment $[a', a'']$ on this arc, with length $\alpha = h_1([a', a''])$.

How much can $\varphi_1(s)$ change for a point $s \in [a', a'']$?

$$\begin{aligned}
|\varphi_1(s) - \varphi_1(a')| &= \left| \sum_{k=1}^{m_1} (p_1^k(s) - p_1^k(a')) f_1^k(\alpha) \right| \\
&\leq \sum_{k=1}^{m_1} |p_1^k(s) - p_1^k(a')| \cdot |f_1^k(\alpha)| \\
&\leq \sum_{k=1}^{m_1} \omega(\alpha) \cdot \mu = m_1 \mu \omega(\alpha)
\end{aligned}$$

We can *choose* our arc length α to be sufficiently small so that $m_1\omega(\alpha) \leq \frac{\lambda}{2}$. This makes the change small:

$$|\varphi_1(s) - \varphi_1(a')| \leq \frac{\lambda\mu}{2}$$

By the reverse triangle inequality, $\varphi_1(s)$ remains large and keeps a constant sign on the entire arc:

$$|\varphi_1(s)| \geq |\varphi_1(a')| - |\varphi_1(s) - \varphi_1(a')| \geq \lambda\mu - \frac{\lambda\mu}{2} = \frac{\lambda\mu}{2}$$

Step 4: Lower Bound the Integral on the Arc

Since $|\varphi_1(s)|$ is large on the entire arc $[a', a'']$, its integral must also be large:

$$\left| \int_{s \in [a', a'']} \varphi_1(s) ds \right| \geq (\text{length of arc}) \cdot (\text{minimum value}) = \alpha \cdot \frac{\lambda\mu}{2}$$

This gives us a **lower bound** for the integral I_a on this "bad" curve where f has its maximum.

Step 5: Compare with a "Reference" Curve (using Lemma 3)

By our δ-net construction (Step 1), our "bad" arc $[a', a'']$ (at $t = \alpha$) is close to one of the "reference" arcs. There exists an index j and a segment $[b', b''] \subset e(q_1^k, t_{1,j}) \cap G$ such that $\rho([a', a''], [b', b'']) < \delta$. Let $I_b = \int_{[b', b'']} \varphi_1(s) ds$.

Upper Bound I_b: On the reference curve $[b', b'']$, $q_1^k(s)$ is constant and equal to $t_{1,j}$. By definition of ε_2, we have $|f_1^k(t_{1,j})| \leq \varepsilon_2$.

$$|I_b| = \left| \int_{[b', b'']} \sum_{k=1}^{m_1} p_1^k(s) f_1^k(t_{1,j}) ds \right| \leq \int \sum |p_1^k(s)| |f_1^k(t_{1,j})| ds$$

Let $\beta = h_1([b', b''])$ and $c_1 = m_1 \max |p_1^k|$.

$$|I_b| \leq \int m_1 c_1 \varepsilon_2 ds = c_1 \varepsilon_2 \beta$$

Bound the Difference (Lemma 3): This lemma (which we previously called Lemma 3) bounds the difference between integrals on two nearby curves. It uses the overall superposition size ε_1:

$$|I_a - I_b| = \left| \int_{[a',a'']} \varphi_1 ds - \int_{[b',b'']} \varphi_1 ds \right| \leq c_3(\alpha\varepsilon_1 + \mu\alpha\omega(\delta) + \mu\delta)$$

Step 6: Combine the Inequalities and Solve for μ

We have a chain of inequalities. We know I_a is large:

$$\frac{\lambda\mu\alpha}{2} \leq |I_a| \leq |I_a - I_b| + |I_b|$$

Applying the bounds from Step 5:

$$\frac{\lambda\mu\alpha}{2} \leq c_3(\alpha\varepsilon_1 + \mu\alpha\omega(\delta) + \mu\delta) + c_1\varepsilon_2\beta$$

By Lemma 1, since δ is small, the lengths α and β are "commensurable" (i.e., $\beta \leq \alpha(1 + \text{small term})$). We can replace β with α and absorb the difference into a new constant c_1'.

$$\frac{\lambda\mu\alpha}{2} \leq c_3\alpha\varepsilon_1 + c_3\mu\alpha\omega(\delta) + c_3\mu\delta + c_1'\varepsilon_2\alpha$$

Now, we rearrange to isolate μ:

$$\mu\left(\frac{\lambda\alpha}{2} - c_3\alpha\omega(\delta) - c_3\delta\right) \leq c_3\alpha\varepsilon_1 + c_1'\varepsilon_2\alpha$$

We can now *choose* δ to be sufficiently small (relative to α) so that the error terms for μ are small, for example, such that:

$$c_3\alpha\omega(\delta) + c_3\delta < \frac{\lambda\alpha}{4}$$

The inequality becomes:

$$\mu\left(\frac{\lambda\alpha}{4}\right) \leq c_3\alpha\varepsilon_1 + c_1'\varepsilon_2\alpha$$

Since α (the arc length) is positive, we can divide both sides by α:

$$\mu\left(\frac{\lambda}{4}\right) \leq c_3\varepsilon_1 + c_1'\varepsilon_2$$

Finally, setting $c = \max(4c_3/\lambda, 4c_1'/\lambda)$, we have shown:

$$\mu \leq c(\varepsilon_1 + \varepsilon_2)$$

This completes the proof of the lemma. □

Setup for the Operator

Let B be the Banach space consisting of all systems of functions $\{f_i^k(t)\}$, defined and continuous on the sets $\{q_i^k(G)\}$, with the norm

$$\|f_i^k(t)\|_B = \max_{i,k} \max_{t \in q_i^k(G)} |f_i^k(t)|.$$

We denote by $C(G)$ the space of all functions $f(x, y)$ continuous on G with the uniform metric:

$$\|f(x, y)\|_{C(G)} = \max_{(x,y)\in G} |f(x, y)|.$$

Lemma 12. *The linear operator $T : B \to C(G)$ acting by the formula*

$$T(\{f_i^k(t)\}) = f(x, y) = \sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x, y) f_i^k(q_i^k(x, y)),$$

maps bounded closed sets of B onto closed sets of $C(G)$.

Proof. This proof uses a standard argument from functional analysis. We will show that if a sequence of functions $f_n(x, y)$ in the image of T converges to a function $f(x, y)$, then $f(x, y)$ must also be in the image of T.

Step 1: Setup the Convergence Argument

Let $F \subset B$ be a **closed and bounded** set in the "input" space B. Let $T(F)$ be its image in the "output" space $C(G)$. We want to prove that $T(F)$ is a closed set.

To do this, we take an arbitrary convergent sequence $f_n(x, y)$ from the set $T(F)$. Let's say $f_n \to f$ in $C(G)$. This means:

$$\|f_n - f\|_{C(G)} \to 0 \quad \text{as } n \to \infty$$

Our goal is to prove that the limit function $f(x, y)$ also belongs to $T(F)$.

Step 2: Relate Convergent Output to Input Functions

Since each f_n is in $T(F)$, there exists a corresponding set of input functions $\{f_{i,n}^k(t)\} \in F$ such that $T(\{f_{i,n}^k\}) = f_n$.

We have a sequence of superpositions $\{f_n\}$ converging to f. We have a sequence of input functions $\{\{f_{i,n}^k\}\}$ from the set F. We need to show that this sequence of input functions also converges to some limit $\{f_i^k\} \in F$, and that $T(\{f_i^k\}) = f$.

Step 3: Using the Stability Estimate (Lemma 11)

The previous lemma provides the key tool. Let's apply the stability inequality from Lemma 2.5.1 to the *difference* of two sets of input functions, $\{f_{i,n}^k\}$ and $\{f_{i,m}^k\}$. Let $\Delta f = f_n - f_m$, and $\Delta\{f\} = \{f_{i,n}^k\} - \{f_{i,m}^k\}$. Then $T(\Delta\{f\}) = \Delta f$. Applying the inequality gives:

$$\|\{f_{i,n}^k\} - \{f_{i,m}^k\}\|_B \le c \left(\|f_n - f_m\|_{C(G)} + \max_{h,j,i} |f_{i,n}^h(t_{i,j}) - f_{i,m}^h(t_{i,j})| \right)$$

We know that $\{f_n\}$ is a convergent sequence in $C(G)$, so it is a Cauchy sequence. This means the first term $\|f_n - f_m\|_{C(G)} \to 0$ as $n, m \to \infty$.

What about the second term? The sequence of input functions $\{f_{i,n}^k\}$ is in F, which is **bounded**. This means the sequence of their values at the fixed points $t_{i,j}$ is a bounded sequence of numbers. By Bolzano-Weierstrass, we can extract a subsequence (let's re-label it n_ν) such that it converges at all the finite points $t_{i,j}$. This implies that for this subsequence, the sequence of values $\{f_{i,n_\nu}^k(t_{i,j})\}$ is a Cauchy sequence.

Let's look at our inequality again for this subsequence n_ν and n_μ:

$$\|\{f_{i,n_\nu}^k\} - \{f_{i,n_\mu}^k\}\|_B \leq c \left(\underbrace{\|f_{n_\nu} - f_{n_\mu}\|_{C(G)}}_{\to 0} + \underbrace{\max |f_{i,n_\nu}^h(t_{i,j}) - f_{i,n_\mu}^h(t_{i,j})|}_{\to 0} \right)$$

As $\nu, \mu \to \infty$, the right-hand side goes to zero. This implies that the sequence of functions $\{f_{i,n_\nu}^k\}$ is a **Cauchy sequence** in the space B.

Step 4: Convergence and Conclusion

Since B is a Banach space (a complete normed space), every Cauchy sequence converges to a limit. Therefore, there exists a limit system of functions $\{f_i^k\} \in B$ such that:

$$\|\{f_i^k\} - \{f_{i,n_\nu}^k\}\|_B \to 0$$

We also know that F is a **closed set**. Since $\{f_{i,n_\nu}^k\}$ is a sequence of functions in F that converges, its limit $\{f_i^k\}$ must also be in F.

Now we just need to check that $T(\{f_i^k\}) = f$. The operator T is a sum of multiplications by continuous functions, which makes it a bounded (and therefore continuous) linear operator. For a continuous operator, we can swap the operator and the limit:

$$f = \lim_{\nu \to \infty} f_{n_\nu} = \lim_{\nu \to \infty} T(\{f_{i,n_\nu}^k\}) = T\left(\lim_{\nu \to \infty} \{f_{i,n_\nu}^k\}\right) = T(\{f_i^k\})$$

We have shown that $f = T(\{f_i^k\})$ where $\{f_i^k\} \in F$. By definition, this means $f \in T(F)$. Since our choice of a convergent sequence $f_n \to f$ was arbitrary, we have proven that $T(F)$ contains all its limit points, and is therefore a **closed set**. □

Lemma 13. *Let B_1 and B_2 be Banach spaces. If a linear operator $T : B_1 \to B_2$ maps bounded closed sets of B_1 onto closed sets of B_2, then its domain of values is closed.*

Proof. (This is a known theorem from the theory of linear operators [9] and is stated without proof.) □

Proof of Theorem 2. We can now assemble the pieces to prove the main theorem.

1. By **Condition (4')**, the set of superpositions of the original form $\sum p_m f_m(q_m)$ is identical on G to the set of superpositions of the re-indexed form $\sum p_i^k f_i^k(q_i^k)$. Therefore, proving that one set is closed proves that the other is also closed.

2. We defined the linear operator T which maps the space of input functions B (a Banach space) to the space of output superpositions $C(G)$ (also a Banach space). The **domain of values** (i.e., the range) of this operator, $T(B)$, is precisely the set of superpositions we are studying.

3. **Lemma 12** proved that our specific operator T maps bounded closed sets of B onto closed sets of $C(G)$.

4. **Lemma 13** (a general theorem from analysis) states that any operator with this exact property must have a closed domain of values (a closed range).

By combining these lemmas, we conclude that the range of T, which is the set of all superpositions of the given form, is a closed subspace of $C(G)$.

This proves the theorem. $\square$

2.7 The set of linear superpositions in the space of continuous functions is nowhere dense

Theorem 3. *For any continuous functions $p_m(x, y)$ and continuously differentiable functions $q_m(x, y)$ $(m = 1, 2, \ldots, N)$ and any region D of the plane of the variables x, y, the set of superpositions of the form*

$$\sum_{m=1}^{N} p_m(x, y) f_m(q_m(x, y)), \text{ where } \{f_m(t)\} \text{ are arbitrary continuous functions,}$$

is nowhere dense in the space of all functions continuous in D with uniform convergence.

Proof. To prove a set is **nowhere dense**, we must show two things:

1. The set is **closed**. (This was proven by Theorem 2 in the previous section).
2. The set is a **proper subset** of the space (i.e., it does not exhaust all continuous functions).

The entire proof is dedicated to proving the second point: we will construct a simple continuous function that *cannot* be represented by the given superposition.

Step 1: Setup and Re-indexing (Applying previous lemmas)

The proof begins with a clever setup. The strategy is to add a new, simple function to the list and then apply our "reduction" lemmas (2 and 10) to this expanded set.

- **Initial Cleanup (Lemma 2):** First, we find a subregion $G^* \subset D$ and re-index the original N functions $\{q_m\}$ into families $\{\bar{q}_i^k\}$ that satisfy the geometric conditions (1, 2, 3) (non-degenerate gradients, shared geometry, transversality).
- **Adding a New Function:** We introduce an $(N + 1)$-th function. We pick a point $(x_0, y_0) \in G^*$ and a slope ν such that the line $y - y_0 = -\nu(x - x_0)$ is **not tangent** to any of the level curves at that point. This guarantees our new function is geometrically "transverse" to the others. We define:

$$q_{N+1}(x, y) = y + \nu x$$
$$p_{N+1}(x, y) = 1$$

- **Final Reduction (Lemma 10):** We now consider the full set of $N + 1$ functions. We apply Lemma 10 to this new set. This allows us to find a final region $G \subset G^{**} \subset G^*$ and a new, final re-indexing $\{p_i^k, q_i^k\}$ for $i = 0, \ldots, n+1$. This new system satisfies all the "good" conditions from the previous section:
  - (1)-(3): Geometric regularity.
  - (4'): Expressive completeness (we haven't lost anything by re-indexing).
  - (5'): Strong linear independence (with constant $\lambda > 0$).
  - (6'): A regular region G.

- **The Key Result:** The reduction process (Lemma 10) isolates our newly added function into its own family. The re-indexing is guaranteed to produce a family, which we label $i = n + 1$, such that:

$$m_{n+1} = 1, \quad p^1_{n+1}(x, y) = 1, \quad q^1_{n+1}(x, y) = y + \nu x. \tag{2.6}$$

We will now work with this final, well-behaved system on the region G.

Step 2: Define the Space of "Null Superpositions" (L)

Let L be the linear space consisting of all systems of "input" functions $\{f^k_i(t)\}$ that make the entire superposition identically zero on G:

$$L = \left\{ \{f^k_i(t)\} \in B \;\middle|\; \sum_{i=0}^{n+1} \sum_{k=1}^{m_i} p^k_i(x, y) f^k_i(q^k_i(x, y)) \equiv 0 \text{ in } G \right\}$$

Intuition: L is the **kernel** of the linear operator T we defined in Lemma 12. It represents all the "redundancies" in the system.

Lemma 14. *L is a finite-dimensional linear space.*

Proof of Lemma 14. This proof is a direct and clever application of the stability inequality from Lemma 11.

1. **Recall Lemma 11:** This lemma gives the inequality:

$$\max |f^k_i(t)| \leq c \left(\max_G \left| \sum \dots \right| + \max |f^k_i(t_{i,j})| \right)$$

(Using $\mu \leq c(\varepsilon_1 + \varepsilon_2)$ from our previous notation).

2. **Apply to the space L:** Let $\{f^k_i(t)\}$ be an element of L. By definition of L, the superposition is identically zero. This means the first term $\varepsilon_1 = 0$.

$$\max |f^k_i(t)| \leq c \left(0 + \max |f^k_i(t_{i,j})| \right)$$

3. **Interpret the Result:** This inequality shows that the entire set of functions $\{f^k_i(t)\}$ (represented by their maximum value) is completely controlled by their values on the **finite set of points** $\{t_{i,j}\}$.

4. **The "Zero Test":** The proof text states: "if $\{f^k_i(t)\} \in L$ and $f^k_i(t_{i,j}) = 0$ for all k, i, j, then...". Let's test this. If the value at every reference point is zero, then $\max |f^k_i(t_{i,j})| = 0$. Plugging this into our inequality gives: $\max |f^k_i(t)| \leq c(0) = 0$. This implies $\max |f^k_i(t)| = 0$, which means $f^k_i(t) \equiv 0$ for all i, k.

5. **Conclusion on Dimension:** We have shown that any function $\{f^k_i\} \in L$ is uniquely and completely determined by its values on the finite set of parameters $\{f^k_i(t_{i,j})\}$. The linear map from L to $\mathbb{R}^S$ (where S is the total number of points $t_{i,j}$) that maps a function to its values at these points is injective. This proves that L is isomorphic to a subspace of $\mathbb{R}^S$ and must therefore be **finite-dimensional**.

□

Lemma 15. *There exists a natural number μ such that in D the polynomial $(y+\nu x)^\mu = Q(x,y)$ is not equal to any superposition of the form $\sum_{m=1}^{N} p_m(x,y) f_m(q_m(x,y))$.*

Proof of Lemma 15. This is the core of the argument, showing that the set of superpositions is a *proper* subset.

1. **Define the space Φ:** Let Φ be the space of functions from our "special" family ($i = n+1$) that can be represented by a combination of all the *other* families ($i = 0, \ldots, n$). A function $f(y+\nu x)$ is in Φ if there exist functions $\{f_i^k\}_{i=0}^n$ such that:
$$f(y+\nu x) = -\sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y))$$

2. **Relate Φ to L:** We can rewrite this definition. Using our setup where $p_{n+1}^1 = 1$ and $q_{n+1}^1 = y+\nu x$, we set $f_{n+1}^1(t) = f(t)$. The equation becomes:
$$\sum_{i=0}^{n} \sum_{k=1}^{m_i} p_i^k(x,y) f_i^k(q_i^k(x,y)) + p_{n+1}^1 f_{n+1}^1(q_{n+1}^1) = 0$$
This is precisely the definition of an element in the space L. Therefore, Φ is the set of all f_{n+1}^1 components that can be part of a null-superposition in L.

3. **Φ is Finite-Dimensional:** The space Φ is isomorphic to a subspace of L. Since L is finite-dimensional (by Lemma 14), Φ must also be **finite-dimensional**. Let $\dim(\Phi) = l$.

4. **The Polynomial Argument:** Consider the set of $l+1$ simple polynomials:
$$P = \{(y+\nu x), (y+\nu x)^2, \ldots, (y+\nu x)^{l+1}\}$$
All these functions are of the form $f(y+\nu x)$. They are also well-known to be **linearly independent**.

5. **Conclusion:** We have a set P containing $l+1$ linearly independent functions, but the space Φ (which contains functions of this form) only has dimension l. It is impossible for all $l+1$ functions from P to be contained in an l-dimensional space.

6. Therefore, at least one of these polynomials, $Q(x,y) = (y+\nu x)^\mu$ (for some $1 \le \mu \le l+1$), is **not** in Φ.

7. "Not in Φ" means $Q(x,y)$ *cannot* be represented as a superposition of the other n families. By Condition (4'), this means $Q(x,y)$ cannot be represented by the original superposition $\sum_{m=1}^{N} p_m f_m(q_m)$.

This proves the lemma: we have found a continuous function $Q(x,y)$ that is not in the set. □

Final Proof of Theorem 2

We now have all the necessary components.

1. By **Lemma 15**, we have proven that the set of superpositions (let's call it S) is a **proper subset** of $C(G)$, since we found a function $Q(x, y)$ that is in $C(G)$ but not in S.

2. By **Theorem 2** (from the previous section), we know that the set S is a **closed linear subspace** of $C(G)$.

A proper, closed linear subspace of a Banach space (like $C(G)$) is, by definition, **nowhere dense**. Since the set S is nowhere dense in $C(G)$, it is also nowhere dense in $C(D)$. This completes the proof of the theorem.

□

Corollary 1. *For any continuous functions $p_m(x_1, \ldots, x_n)$ and continuously differentiable functions $q_m(x_1, \ldots, x_n)$ $(m = 1, \ldots, N)$ and any region D, the set of superpositions of the form*

$$\sum_{m=1}^{N} p_m(x_1, \ldots, x_n) f_m(q_m(x_1, \ldots, x_n), x_2, \ldots, x_{n-1}),$$

where $\{f_m(t, x_2, \ldots, x_{n-1})\}$ are arbitrary continuous functions of $(n-1)$ variables, is nowhere dense in the space of all functions continuous in D with uniform convergence.

Proof Idea. This corollary extends the 2-variable theorem to n variables. The proof works by treating the variables $x_2, \ldots, x_{n-1}$ as fixed "parameters". For any fixed set of these parameters, the superposition becomes a function of x_1 and x_n. We can then apply Theorem 3 to this "slice" of the function space to show that we cannot represent all continuous functions on that slice. Since the set of superpositions fails to be complete on a slice, it cannot be complete in the full n-dimensional space. □

Chapter 3

Kolmogorov-Arnold Networks

3.1 A New Perspective on the Kolmogorov-Arnold Representation Theorem

The foundational theorem by Kolmogorov and Arnold (Chapter 1) asserts that any continuous multivariate function can be decomposed into a finite superposition of continuous univariate functions and additions. Formally, for an arbitrary continuous mapping $f : [0,1]^n \to \mathbb{R}$, there exist continuous functions $\phi_{q,p} : [0,1] \to \mathbb{R}$ and $\Phi_q : \mathbb{R} \to \mathbb{R}$ such that:

$$f(\mathbf{x}) = f(x_1, ..., x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right). \tag{3.1}$$

This representation unveils a remarkably elegant principle: high-dimensional functions can be reconstructed solely through the composition of 1D transformations and summation operations. Ideally, if the internal functions $\phi_{q,p}$ and Φ_q were smooth or differentiable, this theorem would offer a constructive methodology for approximating arbitrary smooth functions using elementary components. Such a characteristic would bestow the theorem with significant practical utility in designing neural architectures based on differentiable compositions, beyond its theoretical significance.

However, subsequent analysis by Vitushkin demonstrated that (Chapter 2), in the general case, the internal functions in Kolmogorov's construction are non-differentiable.

Despite this limitation, the core insight of the theorem remains powerful: **any continuous function can, in principle, be represented through a finite composition of univariate nonlinear transformations and addition.** This concept is the driving force behind modern innovations like *Kolmogorov–Arnold Networks (KANs)*[4], which reinterpret the theorem within a deep learning framework by utilizing learnable, differentiable univariate functions.

3.2 The KAN Architecture

Consider a supervised learning task with input–output pairs $\{(\mathbf{x}_i, y_i)\}$, where the objective is to approximate a function f such that $y_i \approx f(\mathbf{x}_i)$. Following Eq. (3.1), this goal is attainable if we can determine appropriate univariate functions $\phi_{q,p}$ and Φ_q that satisfy the Kolmogorov–Arnold representation. This insight motivates the construction of a neural network that explicitly parameterizes Eq. (3.1).

Given that the functions to be learned are univariate, each component can be modeled as a B-spline curve with learnable coefficients linked to local B-spline basis functions (refer to Figure 3.2, right). This formulation establishes the prototype of a *Kolmogorov–Arnold Network* (KAN). Its computational graph is rigorously defined by Eq. (3.1) and is depicted in Figure 3.1(b) for an input dimension of $n = 2$.

A distinguishing feature of this architecture is that nonlinear activation functions are located on the *edges* of the graph, whereas the *nodes* strictly perform summation. Consequently, the model resembles a two-layer neural network with a hidden layer width of $2n+1$, accurately mirroring the structure dictated by the Kolmogorov–Arnold theorem.

It is important to emphasize that Kolmogorov–Arnold Networks (KAN) differ fundamentally from standard Multi-Layer Perceptrons (MLPs) at a structural level. In classical MLP architectures (Figure 3.1 a,c), nonlinear activation functions are applied at the *nodes*, while the *edges* of the network carry only linear weights. As a consequence, each neuron computes a fixed nonlinear transformation of a weighted linear combination of its inputs.

To further clarify this distinction we repeat that the *nodes* perform only linear summations with fixed coefficients, whereas the *edges* are endowed with learnable *nonlinear* functions. These edge-wise nonlinearities explicitly parameterize univariate functions, in direct correspondence with the Kolmogorov–Arnold representation theorem. This structural distinction leads to a network in which expressivity is concentrated on the connections rather than on the neurons themselves.

As a result, while MLPs rely on fixed activation functions and adaptive linear weights, KANs employ adaptive nonlinear transformations along the edges and simple summation operators at the nodes, yielding a computational graph that more faithfully mirrors the theoretical decomposition prescribed by the Kolmogorov–Arnold theorem.

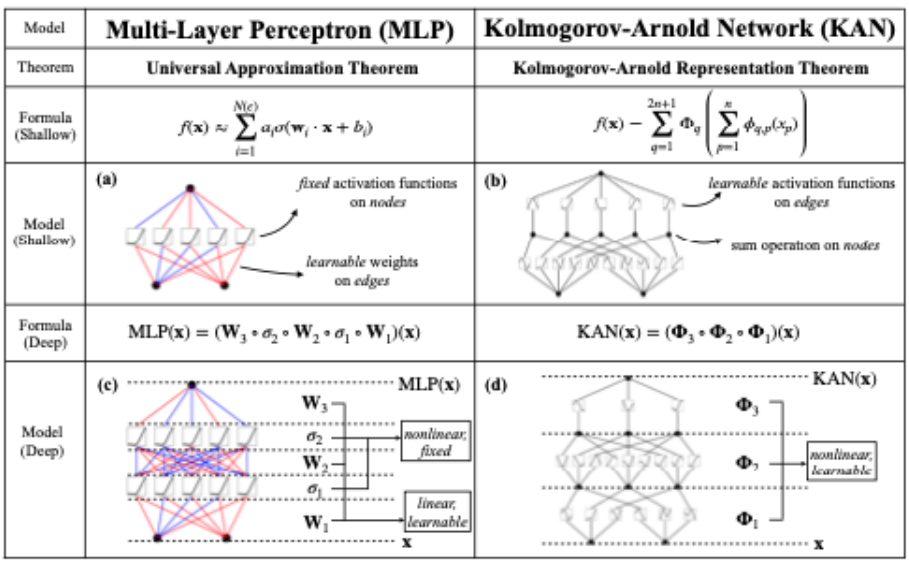


Figure 3.1: MLP AND KAN STRUCTURE

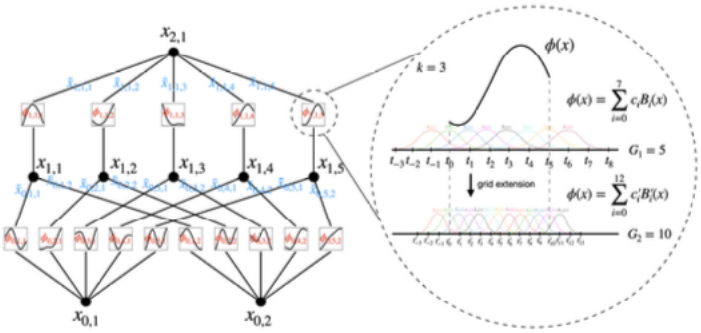


Figure 3.2: Left: Notations of activations that flow through the network. Right: an activation function is parameterized as a B-spline, which allows switching between coarse-grained and fine-grained grids.

Extending the depth of KANs is non-trivial, as the original theorem describes a inherently shallow, two-layer structure. Currently, no rigorous "generalized" version of the theorem exists to theoretically underpin deeper architectures, largely due to Vitushkin's constraints.

A breakthrough is achieved by drawing an analogy between Multi-Layer Perceptrons (MLPs) and KANs. In MLPs, defining a single layer (comprising linear transformations and nonlinearities) allows for stacking to increase depth. To construct deep KANs, one must essentially define a "KAN layer." A KAN layer mapping n_{in}-dimensional inputs to n_{out}-dimensional outputs can be formalized as a matrix of 1D functions:

$$\mathbf{\Phi} = \{\phi_{q,p}\}, \quad p = 1, 2, \cdots, n_{\text{in}}, \quad q = 1, 2 \cdots, n_{\text{out}} \tag{3.2}$$

where $\phi_{q,p}$ denote functions with trainable parameters. In the context of the Kolmogorov-Arnold theorem, the inner functions constitute a KAN layer with $n_{\text{in}} = n$ and $n_{\text{out}} = 2n + 1$, while the outer functions form a layer with $n_{\text{in}} = 2n + 1$ and $n_{\text{out}} = 1$. Thus, the representation in Eq. (3.1) is effectively a composition of two KAN layers. This clarifies the definition of deep Kolmogorov-Arnold representations: they are formed by stacking multiple KAN layers.

We introduce the following notation. The architecture's shape is denoted by an integer array:

$$[n_0, n_1, \cdots, n_L]. \tag{3.3}$$

Here, n_i represents the node count in the ith layer. The i-th neuron in layer l is denoted by (l, i), with activation value $x_{l,i}$. Between layers l and $l+1$, there exist $n_l n_{l+1}$ activation functions. The function connecting (l, i) to $(l + 1, j)$ is:

$$\phi_{l,j,i}, \quad l = 0, \cdots, L - 1, \quad i = 1, \cdots, n_l, \quad j = 1, \cdots, n_{l+1}. \tag{3.4}$$

The pre-activation for $\phi_{l,j,i}$ is $x_{l,i}$, and the post-activation is $\tilde{x}_{l,j,i} \equiv \phi_{l,j,i}(x_{l,i})$. The activation of neuron $(l + 1, j)$ is the sum of all incoming post-activations:

$$x_{l+1,j} = \sum_{i=1}^{n_l} \tilde{x}_{l,j,i} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i}), \quad j = 1, \cdots, n_{l+1}. \tag{3.5}$$

Expressing this in matrix notation:

$$\mathbf{x}_{l+1} = \underbrace{\begin{pmatrix} \phi_{l,1,1}(\cdot) & \phi_{l,1,2}(\cdot) & \cdots & \phi_{l,1,n_l}(\cdot) \\ \phi_{l,2,1}(\cdot) & \phi_{l,2,2}(\cdot) & \cdots & \phi_{l,2,n_l}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1}(\cdot) & \phi_{l,n_{l+1},2}(\cdot) & \cdots & \phi_{l,n_{l+1},n_l}(\cdot) \end{pmatrix}}_{\mathbf{\Phi}_l} \mathbf{x}_l. \tag{3.6}$$

where $\mathbf{\Phi}_l$ is the function matrix for the lth layer. A general KAN is the composition of L layers. For an input $\mathbf{x}_0 \in \mathbb{R}^{n_0}$, the output is:

$$\mathrm{KAN}(\mathbf{x}) = (\mathbf{\Phi}_{L-1} \circ \mathbf{\Phi}_{L-2} \circ \cdots \circ \mathbf{\Phi}_1 \circ \mathbf{\Phi}_0)\mathbf{x}. \tag{3.7}$$

To align this with Eq. (3.1), assuming $n_L = 1$ and defining $f(\mathbf{x}) \equiv \mathrm{KAN}(\mathbf{x})$, we can write:

$$f(\mathbf{x}) = \sum_{i_{L-1}=1}^{n_{L-1}} \phi_{L-1,1,i_{L-1}} \left(\sum_{i_{L-2}=1}^{n_{L-2}} \cdots \left(\sum_{i_1=1}^{n_1} \phi_{1,i_2,i_1} \left(\sum_{i_0=1}^{n_0} \phi_{0,i_1,i_0}(x_{i_0}) \right) \right) \cdots \right) \tag{3.8}$$

While this expression is cumbersome, our layer abstraction offers a cleaner intuition. The original representation in Eq. (3.1) is equivalent to a 2-Layer KAN of shape $[n, 2n+1, 1]$. Crucially, since all operations are differentiable, KANs are trainable via backpropagation. In contrast, an MLP interleaves affine transformations $\mathbf{W}$ and non-linearities σ:

$$\mathrm{MLP}(\mathbf{x}) = (\mathbf{W}_{L-1} \circ \sigma \circ \mathbf{W}_{L-2} \circ \sigma \circ \cdots \circ \mathbf{W}_1 \circ \sigma \circ \mathbf{W}_0)\mathbf{x}. \tag{3.9}$$

MLPs treat linear transforms and nonlinearities as separate entities ($\mathbf{W}$ and σ), whereas KANs integrate them within $\mathbf{\Phi}$. Figures 3.1 (c) and (d) illustrate a three-layer MLP versus a three-layer KAN to highlight these structural differences.

Implementation Details. While the layer definition in Eq. (3.5) is simple, optimizing it effectively is challenging. Key techniques include:

(1) **Residual Activation Functions.** A basis function $b(x)$ is included (analogous to residual connections), making the activation $\phi(x)$ the sum of $b(x)$ and the spline:

$$\phi(x) = w_b b(x) + w_s \mathrm{spline}(x). \tag{3.10}$$

Typically, $b(x) = \mathrm{silu}(x) = x/(1+e^{-x})$. The spline component is a linear combination of B-splines:

$$\mathrm{spline}(x) = \sum_i c_i B_i(x) \tag{3.11}$$

with trainable coefficients c_i (see Figure 2.2). Although w_b and w_s are theoretically redundant, retaining them aids in controlling the activation magnitude.

(2) **Initialization Scales.** Activations are initialized with $w_s \approx 0$ and $\mathrm{spline}(x) \approx 0$. w_b uses Xavier initialization, standard for MLP linear layers.

(3) **Spline Grid Updates.** Grids are updated dynamically based on input activations, addressing the issue where values may drift outside the bounded regions of fixed splines during training.

3.3 Approximation Capabilities and Scaling Laws of KANs

To underscore the dependence on the grid size, we denote the functions as $\mathbf{\Phi}_l^G$ and $\phi_{l,i,j}^G$ in place of the notation in Eq. (3.5) and Eq. (3.6).

Theorem 4 (Approximation theory, KAT)**.** *Let* $\mathbf{x} = (x_1, x_2, \cdots, x_n)$*. Suppose that a function* $f(\mathbf{x})$ *admits a representation*

$$f = (\mathbf{\Phi}_{L-1} \circ \mathbf{\Phi}_{L-2} \circ \cdots \circ \mathbf{\Phi}_1 \circ \mathbf{\Phi}_0)\mathbf{x}, \tag{3.12}$$

as in Eq. (3.7), where each one of the $\phi_{l,i,j}$ *are* $(k+1)$*-times continuously differentiable. Then there exists a constant* C *depending on* f *and its representation, such that we have the following approximation bound in terms of the grid size* G*: there exist* k*-th order B-spline functions* $\phi_{l,i,j}^G$ *such that for any* $0 \leq m \leq k$ *we have the bound*

$$||f - (\mathbf{\Phi}_{L-1}^G \circ \mathbf{\Phi}_{L-2}^G \circ \cdots \circ \mathbf{\Phi}_1^G \circ \mathbf{\Phi}_0^G)\mathbf{x}||_{C^m} \leq CG^{-k-1+m} \tag{3.13}$$

Here we adopt the notation of C^m-norm measuring the magnitude of derivatives up to order m: $||g||_{C^m} = \max_{|\beta| \leq m} \sup_{\mathbf{x} \in [0,1]^n} |D^\beta g(\mathbf{x})|$.

Proof. By the classical 1D B-spline theory [7] and the fact that $\phi_{l,i,j}$ as continuous functions can be uniformly bounded on a bounded domain, we know that there exist finite-grid B-spline functions $\phi_{l,i,j}^G$ such that for any $0 \leq m \leq k$,

$$||(\phi_{l,i,j} \circ \mathbf{\Phi}_{l-1} \circ \cdots \circ \mathbf{\Phi}_0)\mathbf{x} - (\phi_{l,i,j}^G \circ \mathbf{\Phi}_{l-1} \circ \cdots \circ \mathbf{\Phi}_0)\mathbf{x}||_{C^m} \leq CG^{-k-1+m}$$

with a constant C independent of G. We fix those B-spline approximations. Therefore we have that the residue R_l defined via

$$R_l := (\mathbf{\Phi}_{L-1}^G \circ \cdots \circ \mathbf{\Phi}_{l+1}^G \circ \mathbf{\Phi}_l \circ \cdots \circ \mathbf{\Phi}_0)\mathbf{x} - (\mathbf{\Phi}_{L-1}^G \circ \cdots \circ \mathbf{\Phi}_{l+1}^G \circ \mathbf{\Phi}_l^G \circ \cdots \circ \mathbf{\Phi}_0)\mathbf{x}$$

satisfies

$$||R_l||_{C^m} \leq CG^{-k-1+m},$$

with a constant independent of G. Finally notice that

$$f - (\mathbf{\Phi}_{L-1}^G \circ \mathbf{\Phi}_{L-2}^G \circ \cdots \circ \mathbf{\Phi}_1^G \circ \mathbf{\Phi}_0^G)\mathbf{x} = R_{L-1} + R_{L-2} + \cdots + R_1 + R_0$$

we know that (3.13) holds. □

Chapter 4

Dual-PINN with efficient-KAN for 1D Fokker–Planck

4.1 Introduction

In this section, we use the framework presented in the reference article [5], which introduces a *Dual-PINN* structure for solving stationary Fokker–Planck equations. The original methodology couples two neural sub-networks — one dedicated to capturing the interior behavior of the stationary probability density and another to accurately enforce the boundary conditions — within an Augmented Lagrangian formulation. This dual-network approach effectively separates the interior and boundary contributions, improving both numerical stability and convergence in the presence of stiff potentials.

In our implementation, we have reproduced the same overall training framework and optimization scheme as proposed in the paper, but we replace the traditional multilayer perceptrons (MLPs) with the recently developed *Kolmogorov–Arnold Network* (KAN) architecture. KANs are inspired by the Kolmogorov–Arnold superposition theorem and employ adaptive nonlinear basis functions, which allow for improved expressivity and interpretability compared to conventional MLPs.

Specifically, we apply this Dual-PINN + KAN architecture to the one-dimensional stationary Fokker–Planck equation associated with a double-well potential, given by

$$0 = -\frac{d}{dx}\big(\mu(x)\,\rho(x)\big) + D\,\frac{d^2\rho(x)}{dx^2},$$

where $\mu(x)$ is the drift term and D denotes the diffusion coefficient.

In this benchmark, we consider the *steady-state* (time-independent) Fokker–Planck equation on a bounded interval $[x_{\min}, x_{\max}]$ with Dirichlet boundary conditions

$$\rho(x_{\min}) = 0, \qquad \rho(x_{\max}) = 0,$$

and the probability-mass constraint

$$\int_{x_{\min}}^{x_{\max}} \rho(x)\,dx = 1,$$

so that ρ represents a normalized stationary probability density on the computational domain.In practice, this constraint is enforced numerically on a fixed grid via a trapezoidal rule.

The goal is to approximate the stationary density $\rho(x)$, which exhibits two distinct peaks corresponding to the wells of the potential.

4.2 Python code

In this section, we present the Python code for solving the stationary Fokker–Planck equation in 1D using a *Dual-PINN* approach based on `efficient-kan`[6]. The solution $\rho(x)$ is represented as the sum of two KAN networks,

$$\rho(x) \approx \rho_D(x) + \rho_B(x),$$

and the training is organized into multiple phases with adaptive internal sampling and an Augmented Lagrangian strategy for boundary conditions.

We report the full code[8], divided into logical blocks, and explain each part.

4.3 Imports, Logging and Device

4.3.1 Library Imports and KAN Module

```python
import math
import copy
import logging
import numpy as np
import time
import json
from dataclasses import dataclass, asdict
from typing import Tuple
from pathlib import Path
from datetime import datetime

import torch
import torch.nn as nn
import torch.autograd as autograd
import matplotlib.pyplot as plt

# =============== efficient-KAN ============================
# Make sure you have installed:
# pip install "git+https://github.com/Blealtan/efficient-kan.git"
from efficient_kan import KAN
```

This block:

- imports standard Python modules (`math`, `copy`, `time`, `json`, `pathlib`, `datetime`);
- imports `dataclass` and `Tuple` to build a centralized configuration class;
- imports `numpy` for basic numerical operations and statistics over multiple seeds;
- imports `torch`, `torch.nn` and `torch.autograd` for neural networks and automatic differentiation;
- imports `matplotlib.pyplot` to generate the final plot;
- imports the `KAN` network from the `efficient-kan` library [6], which implements Kolmogorov–Arnold Networks.

4.3.2 Logging Configuration and Device Selection

```python
# =============== LOGGING ============================
logging.basicConfig(
    level=logging.INFO,
    format='[%(asctime)s] %(levelname)s: %(message)s',
    datefmt='%H:%M:%S'
)
logger = logging.getLogger(__name__)

# =============== DEVICE ============================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
logger.info(f"Using device: {device}")

if device.type == 'cuda':
    logger.info(f"GPU: {torch.cuda.get_device_name(0)}")
    logger.info(f"Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
```

In this block:

- a global `logger` is configured to print timestamped messages with level and text;
- the code automatically selects `"cuda"` if a GPU is available, otherwise falls back to `"cpu"`;
- in case of GPU, the script logs the device name and the total available memory.

4.4 Centralized Configuration and Fokker–Planck Model

4.4.1 Config Dataclass

```python
# =============== CENTRALIZED CONFIGURATION ==================
@dataclass
class Config:
    """Centralized configuration for all model parameters."""
    # Fokker-Planck physical parameters
    a: float = 0.3
    b: float = 0.5
    sigma: float = 0.5
    x_min: float = -2.2
    x_max: float = 2.2

    # Network architecture
    layers_D: Tuple[int, ...] = (1, 2, 2, 1)
    layers_B: Tuple[int, ...] = (1, 2, 2, 1)

    # Training
    N_int: int = 128
    max_epochs: int = 1200
```

```python
    phase0_epochs: int = 100  # BC pre-training
    phase1_epochs: int = 150  # uniform curriculum
    lr: float = 1e-3
    grad_clip_max_norm: float = 1.0

    # Normalization and sampling
    N_norm: int = 400
    phase2_sampling_min_factor: int = 4
    phase2_sampling_max_factor: int = 16

    # ALM (Augmented Lagrangian Method)
    rho_alm_init: float = 10.0
    rho_max: float = 865.0
    eta_rho: float = 1.5
    Lambda_clip: float = 10.0
    k_update_lambda: int = 10
    h_update_rho: int = 80

    # Loss weights
    w_pde: float = 1.0
    w_norm: float = 0.5
    w_pos: float = 0.1

    # Schedulers
    gamma_min: float = 0.0
    gamma_max: float = 0.1
    w_bc_min: float = 0.2
    w_bc_max: float = 1.0

    # Role loss
    alpha_int: float = 1e-3
    alpha_bd: float = 1e-3
    tau: float = 0.6

    # Multi-seed
    seeds: Tuple[int, ...] = (42, 44, 46, 48, 50)

    @property
    def D(self) -> float:
        """Diffusion coefficient."""
        return self.sigma**2 / 2.0

    @property
    def phase2_epochs(self) -> int:
        """Remaining epochs for Phase 2."""
        return self.max_epochs - self.phase0_epochs - self.
            phase1_epochs


config = Config()
```

```
logger.info(f"Config: max_epochs={config.max_epochs}, N_int={
    config.N_int}, lr={config.lr}")
```

This dataclass collects all parameters used in the experiment:

- physical parameters of the Fokker–Planck equation (`a`, `b`, `sigma`, domain endpoints);
- the architecture of the Dual-KAN networks, given as layer widths `(1, 2, 2, 1)`;
- training hyperparameters: number of internal collocation points (`N_int`), total epochs, subdivided into Phase 0, Phase 1, and Phase 2;
- number of points for normalization, factors controlling the adaptive sampling in Phase 2;
- ALM parameters for boundary constraints: initial penalty, maximum penalty, growth rate, clipping for Lagrange multipliers and update frequencies;
- weights of the different loss terms: PDE residual, normalization, positivity;
- scheduler parameters controlling the evolution of the boundary weight and the role-loss weight;
- parameters for the role loss (`alpha_int`, `alpha_bd`, `tau`);
- the list of random seeds used for the multi-seed experiment.

The properties `D` and `phase2_epochs` give, respectively, the diffusion coefficient $D = \sigma^2/2$ and the number of epochs allocated to Phase 2.

4.4.2 Drift and Exact Stationary Solution

```
def mu(x: torch.Tensor) -> torch.Tensor:
    """Drift: a x - b x^3."""
    return config.a * x - config.b * x**3


# exact unnormalized solution
def rho_unnorm(x: torch.Tensor) -> torch.Tensor:
    z = (1.0 / (2.0 * config.sigma**2)) * (2.0 * config.a * x**2
        - config.b * x**4)
    return torch.exp(z)


def rho_exact(x: torch.Tensor) -> torch.Tensor:
    """Exact density normalized on [x_min, x_max]."""
    with torch.no_grad():
        rho_u = rho_unnorm(x)
        x1d = x.view(-1)
        r1d = rho_u.view(-1)
        Z = torch.trapz(r1d, x1d)
        return rho_u / (Z + 1e-8)
```

The function `rho_exact` provides the steady-state reference density on $[x_{\min}, x_{\max}]$ by first evaluating an unnormalized closed-form profile and then computing the normalization constant numerically (trapezoidal rule) so that $\int_{x_{\min}}^{x_{\max}} \rho(x)\, dx = 1$.

In this block:

- the drift is defined as $\mu(x) = ax - bx^3$, corresponding to a double-well potential;
- `rho_unnorm` implements the unnormalized stationary density

$$\rho_{\text{unnorm}}(x) = \exp\left(\frac{1}{2\sigma^2}\left(2ax^2 - bx^4\right)\right);$$

- `rho_exact` normalizes this density over the interval $[x_{\min}, x_{\max}]$ using a trapezoidal integration, returning the exact double-peak stationary solution of the Fokker–Planck equation.

4.5 Pre-allocated Tensors and Loss Components

4.5.1 Grids for Normalization and Boundary Conditions

```python
# =============== PRE-ALLOCATED TENSORS ==================
x_norm = torch.linspace(config.x_min, config.x_max, config.N_norm).view(-1, 1).to(device)
x_bc = torch.tensor([[config.x_min], [config.x_max]], dtype=torch.float32).to(device)
g_bc = torch.zeros_like(x_bc)  # rho = 0 at the endpoints
```

Here:

- `x_norm` is a fixed grid of `N_norm` points used to enforce normalization and positivity of the density;
- `x_bc` contains the boundary points $(x_{\min}, x_{\max})$;
- `g_bc` encodes the boundary conditions $\rho(x_{\min}) = \rho(x_{\max}) = 0$.

4.5.2 Fokker–Planck Residual

```python
# =============== LOSS COMPONENTS ============================
def fp_residual(
    netD, netB, x: torch.Tensor
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]:
    """
    Stationary FP residual:
    0 = -((mu(x) * rho(x))_x) + D * rho_xx
    """
    x = x.clone().detach().to(device)
    x.requires_grad_(True)

    rhoD = netD(x)
```

```
    rhoB = netB(x)
    rho = rhoD + rhoB

    ones = torch.ones_like(rho)

    # (mu * rho)_x
    mu_rho = mu(x) * rho
    d_mu_rho_dx = autograd.grad(
        mu_rho,
        x,
        grad_outputs=torch.ones_like(mu_rho),
        create_graph=True,
    )[0]

    # rho_x, rho_xx
    rho_x = autograd.grad(
        rho,
        x,
        grad_outputs=ones,
        create_graph=True,
    )[0]
    rho_xx = autograd.grad(
        rho_x,
        x,
        grad_outputs=torch.ones_like(rho_x),
        create_graph=True,
    )[0]

    res = -d_mu_rho_dx + config.D * rho_xx
    return res, rhoD, rhoB, rho
```

This function:

- evaluates the two networks `netD` and `netB` on the input points `x` and forms $\rho = \rho_D + \rho_B$;
- uses automatic differentiation to compute $(\mu\rho)_x$, ρ_x, and ρ_{xx};
- returns the stationary Fokker–Planck residual

$$\mathrm{res}(x) = -\partial_x\big(\mu(x)\rho(x)\big) + D\rho''(x),$$

  along with ρ_D, ρ_B and ρ.

4.5.3 Boundary Loss via Augmented Lagrangian

```
def boundary_alm_loss(
    netD,
    netB,
    lambda_bc: torch.Tensor,
    rho_alm: float,
    w_bc: float,
```

```
) -> tuple[torch.Tensor, torch.Tensor]:
    """ALM loss for boundary constraints."""
    rhoD_bc = netD(x_bc)
    rhoB_bc = netB(x_bc)
    rho_bc = rhoD_bc + rhoB_bc
    c = rho_bc - g_bc  # BC violation

    # w_bc(t) modulates the weight of boundary constraints
    L_alm = w_bc * torch.mean(lambda_bc * c + 0.5 * rho_alm * c
        **2)
    return L_alm, c.detach()
```

In this block:

- the boundary values of ρ_D and ρ_B are computed and summed to form ρ_{bc};
- the constraint violation is $c = \rho_{\text{bc}} - g_{\text{bc}}$;
- the ALM loss
$$L_{\text{ALM}} = w_{\text{bc}}\, \mathbb{E}\big[\lambda_{\text{bc}}\, c + \tfrac{1}{2}\rho_{\text{alm}}\, c^2\big]$$
is used to enforce the boundary conditions with a combination of Lagrange multipliers and quadratic penalty;
- the violation `c` is returned detached from the computational graph, so that it can be used to update the multipliers outside of the gradient flow.

4.5.4 Role Loss for the Dual Decomposition

```
def role_loss(
    x: torch.Tensor,
    rhoD: torch.Tensor,
    rhoB: torch.Tensor,
) -> torch.Tensor:
    """
    L_role = alpha_int * E[w_bd * |rhoD|^2] + alpha_bd * E[w_in *
        |rhoB|^2]
    with w_bd = exp(-d/tau), where d is the minimum distance from
        the boundary.
    """
    d_left = (x - config.x_min).abs()
    d_right = (config.x_max - x).abs()
    d = torch.minimum(d_left, d_right)

    w_bd = torch.exp(-d / config.tau)
    w_in = 1.0 - w_bd

    L = config.alpha_int * torch.mean(w_bd * (rhoD**2)) \
        + config.alpha_bd * torch.mean(w_in * (rhoB**2))
    return L
```

The role loss is designed to specialize the two networks:

- the distance $d(x)$ from the nearest boundary point is computed;
- a boundary weight $w_{\text{bd}}(x) = \exp(-d(x)/\tau)$ and its complement $w_{\text{in}}(x) = 1 - w_{\text{bd}}(x)$ are defined;
- the loss
$$L_{\text{role}} = \alpha_{\text{int}}\, \mathbb{E}[w_{\text{bd}}\rho_D^2] + \alpha_{\text{bd}}\, \mathbb{E}[w_{\text{in}}\rho_B^2]$$
discourages ρ_D from dominating near the boundary and penalizes ρ_B in the interior, encouraging a dual decomposition of the density.

4.5.5 Schedulers for Gamma and Boundary Weight

```python
def gamma_schedule(
    epoch_idx: int,
    T: int,
    gamma_min: float = 0.0,
    gamma_max: float = 0.1,
) -> float:
    """Cosine scheduler for gamma in [gamma_min, gamma_max]."""
    t = float(epoch_idx)
    return gamma_min + 0.5 * (gamma_max - gamma_min) * (
        1.0 + math.cos(math.pi * t / T)
    )


def w_bc_schedule(
    epoch_idx: int,
    T: int,
    w_min: float = 0.2,
    w_max: float = 1.0,
) -> float:
    """
    Boundary-constraint weight decreasing linearly from w_max to w_min
    over T epochs.
    """
    t = float(epoch_idx)
    frac = min(max(t / T, 0.0), 1.0)
    return w_max - (w_max - w_min) * frac
```

These two utility functions:

- define a cosine schedule for `gamma`, the weight of the role loss, varying between `gamma_min` and `gamma_max` over T epochs;
- define a linear schedule for `w_bc`, the weight of the boundary constraints, decreasing from `w_max` to `w_min` as training progresses.

4.5.6 Normalization and Positivity Loss

```python
def norm_and_pos_loss(netD, netB) -> tuple[torch.Tensor, torch.Tensor]:
    """Penalizes negativity and non-unit normalization."""
    rho_all = netD(x_norm) + netB(x_norm)

    # negativity penalty
    neg = torch.relu(-rho_all)
    L_pos = torch.mean(neg**2)

    rho_flat = rho_all.view(-1)
    x_flat = x_norm.view(-1)
    integral = torch.trapz(rho_flat, x_flat)
    L_norm = (integral - 1.0) ** 2
    return L_norm, L_pos
```

On the fixed grid `x_norm`, this function:

- penalizes negative values of the density by applying a squared penalty on $\max\{-\rho, 0\}$;
- approximates the integral $\int \rho$ by a trapezoidal rule and penalizes deviations from unity via $(\text{integral} - 1)^2$.

4.6 Internal Sampling Strategies

4.6.1 Uniform Sampling and Phase 1

```python
# =============== INTERNAL SAMPLING ==========================
def sample_uniform(N: int) -> torch.Tensor:
    """Uniform sampling on [x_min, x_max]."""
    x = config.x_min + (config.x_max - config.x_min) * torch.rand(N, 1)
    return x.to(device)


def sample_interior_phase1(N: int) -> torch.Tensor:
    """Phase 1: uniform collocation."""
    return sample_uniform(N)
```

In Phase 1, the internal collocation points are sampled uniformly in the spatial domain, as in standard PINN approaches.

4.6.2 Residual-based Adaptive Sampling (Phase 2)

```python
def sample_interior_phase2(N: int, netD, netB, epoch_ratio: float) -> torch.Tensor:
    """
    Residual-based sampling (Phase 2) with adaptive sampling:
    - generate N_cand >> N with a uniform distribution
    - evaluate the FP residual
    - select the N points with the largest residual
```

```
    - N_cand increases progressively during Phase 2

    Args:
        N: number of points to sample
        netD, netB: neural networks
        epoch_ratio: progress in Phase 2, from 0.0 to 1.0
    """
    # Progressively increase the sampling factor
    N_factor = int(
        config.phase2_sampling_min_factor +
        (config.phase2_sampling_max_factor - config.
           phase2_sampling_min_factor) * epoch_ratio
    )
    N_cand = N_factor * N

    x_cand = sample_uniform(N_cand)
    # sample N_cand points uniformly in the domain
    res_cand, _, _, _ = fp_residual(netD, netB, x_cand)
    r_val = (res_cand**2).detach().view(-1)

    # top-N points by residual
    _, idx = torch.topk(r_val, k=N, largest=True, sorted=False)
    x_sel = x_cand[idx]
    return x_sel
```

In Phase 2:

- an oversampled candidate set of size $N_{\text{cand}} = \texttt{N_factor} \cdot N$ is generated uniformly;
- the FP residual is evaluated on these candidates;
- the N points with the largest squared residual are selected as collocation points;
- the oversampling factor `N_factor` increases from `phase2_sampling_min_factor` to `phase2_sampling_max_factor` as `epoch_ratio` goes from 0 to 1, making the residual-based sampling more aggressive later in Phase 2.

4.7 Training for a Single Seed

4.7.1 Initialization and Optimizer

```
# =============== TRAINING FOR A SINGLE SEED =============
def train_for_seed(seed: int):
    start_time = time.time()

    torch.manual_seed(seed)
    if device.type == 'cuda':
        torch.cuda.manual_seed(seed)

    # new models for this seed
    netD = KAN(list(config.layers_D)).to(device)
```

```python
    netB = KAN(list(config.layers_B)).to(device)

    # Count parameters
    params_D = sum(p.numel() for p in netD.parameters())
    params_B = sum(p.numel() for p in netB.parameters())
    total_params = params_D + params_B

    # ALM variables
    lambda_bc = torch.zeros_like(g_bc, device=device)
    rho_alm = config.rho_alm_init

    optimizer = torch.optim.Adam(
        list(netD.parameters()) + list(netB.parameters()),
        lr=config.lr,
    )

    logger.info(f"\n[Seed {seed}] Starting Dual-PINN efficient-KAN training (epochs = {config.max_epochs})")
    logger.info(f"[Seed {seed}] Parameters: netD={params_D}, netB={params_B}, total={total_params}")
```

At the beginning of `train_for_seed`:

- the random seed is fixed for reproducibility (both on CPU and GPU if available);
- two KAN networks `netD` and `netB` are created using the architecture specified in `config`;
- the total number of parameters is computed and logged;
- the ALM multipliers `lambda_bc` and the penalty parameter `rho_alm` are initialized;
- the Adam optimizer is defined on the combined parameters of the two networks.

4.7.2 Phase 0: Boundary Pre-training

```python
    # ---- PHASE 0: BC pre-training ----
    for epoch in range(1, config.phase0_epochs + 1):
        optimizer.zero_grad()
        rhoD_bc = netD(x_bc)
        rhoB_bc = netB(x_bc)
        rho_bc = rhoD_bc + rhoB_bc
        L_bc0 = torch.mean((rho_bc - g_bc) ** 2)
        L_bc0.backward()

        # Gradient clipping
        torch.nn.utils.clip_grad_norm_(
            list(netD.parameters()) + list(netB.parameters()),
            max_norm=config.grad_clip_max_norm
        )

        optimizer.step()
```

```python

        if epoch == 1 or epoch % 20 == 0 or epoch == config.phase0_epochs:
            logger.info(f"[Seed {seed} | Epoch {epoch:4d} | Phase 0] L_bc0={L_bc0.item():.3e}")
```

Phase 0 performs:

- pre-training of the two networks solely on the boundary conditions $\rho(x_{\min}) = \rho(x_{\max}) = 0$ using a mean squared error;
- gradient clipping for stability, limiting the norm of the gradients;
- periodic logging of the boundary loss `L_bc0`.

4.7.3 Phases 1 and 2: Full Dual-PINN Training

```python
    # ---- PHASE 1 + PHASE 2 ----
    for epoch in range(config.phase0_epochs + 1, config.max_epochs + 1):
        # determine phase
        if epoch <= config.phase0_epochs + config.phase1_epochs:
            phase = 1
        else:
            phase = 2

        # interior sampling
        if phase == 1:
            x_int = sample_interior_phase1(config.N_int)
        else:
            # Compute progress in Phase 2 (0.0 at the beginning, 1.0 at the end)
            epoch_in_phase2 = epoch - (config.phase0_epochs + config.phase1_epochs)
            epoch_ratio = min(epoch_in_phase2 / config.phase2_epochs, 1.0)
            x_int = sample_interior_phase2(config.N_int, netD, netB, epoch_ratio)

        optimizer.zero_grad()

        # PDE residual
        res, rhoD, rhoB, _ = fp_residual(netD, netB, x_int)
        L_pde = torch.mean(res**2)

        # ALM BC
        epoch_rel = epoch - (config.phase0_epochs + 1)
        w_bc = w_bc_schedule(epoch_rel, config.max_epochs - config.phase0_epochs)
        L_alm, c_bc = boundary_alm_loss(netD, netB, lambda_bc, rho_alm, w_bc)

```

```
        # normalization + positivity
        L_norm, L_pos = norm_and_pos_loss(netD, netB)

        # dual role
        gamma = gamma_schedule(epoch - 1, config.max_epochs)
        L_role = role_loss(x_int, rhoD, rhoB)

        # Total loss with weights from config
        L_total = (
            config.w_pde * L_pde
            + L_alm
            + gamma * L_role
            + config.w_norm * L_norm
            + config.w_pos * L_pos
        )

        L_total.backward()

        # Gradient clipping
        torch.nn.utils.clip_grad_norm_(
            list(netD.parameters()) + list(netB.parameters()),
            max_norm=config.grad_clip_max_norm
        )

        optimizer.step()
```

In this main training loop:

- the current epoch determines whether we are in Phase 1 (uniform sampling) or Phase 2 (residual-based adaptive sampling);
- the internal collocation points `x_int` are drawn accordingly;
- the following loss components are computed:
  - `L_pde`: mean squared Fokker–Planck residual;
  - `L_alm`: ALM boundary loss with time-dependent weight `w_bc`;
  - `L_norm`: normalization loss enforcing $\int \rho = 1$;
  - `L_pos`: positivity loss penalizing negative values of ρ;
  - `L_role`: role loss weighted by `gamma`.
- the total loss `L_total` is a weighted combination of all these terms, with weights given by the configuration;
- gradients are backpropagated, clipping is applied, and the optimizer performs an update step.

4.7.4 ALM Updates and Logging

```python
        # update lambda (ALM)
        if epoch % config.k_update_lambda == 0:
            with torch.no_grad():
                lambda_bc[:] = torch.clamp(
                    lambda_bc + rho_alm * c_bc,
                    -config.Lambda_clip,
                    config.Lambda_clip,
                )

        # progressively increase rho_alm
        if epoch % config.h_update_rho == 0:
            rho_alm = min(rho_alm * config.eta_rho, config.rho_max)

        # logging
        if epoch == config.phase0_epochs + 1 or epoch % 50 == 0 or epoch == config.max_epochs:
            logger.info(
                f"[Seed {seed} | Epoch {epoch:4d} | Phase {phase}] "
                f"L_tot={L_total.item():.3e} "
                f"PDE={L_pde.item():.3e} "
                f"ALM={L_alm.item():.3e} "
                f"Role={L_role.item():.3e} "
                f"Norm={L_norm.item():.3e} "
                f"Pos={L_pos.item():.3e} "
                f"w_bc={w_bc:.3f} gamma={gamma:.3f} rho_alm={rho_alm:.1f}"
            )

        # Memory management for GPU
        if device.type == 'cuda' and epoch % 100 == 0:
            torch.cuda.empty_cache()
```

Here:

- the Lagrange multipliers `lambda_bc` are updated every `k_update_lambda` epochs according to
$$\lambda_{\mathrm{bc}} \leftarrow \lambda_{\mathrm{bc}} + \rho_{\mathrm{alm}}\, c_{\mathrm{bc}},$$
with clipping to the interval $[-\texttt{Lambda_clip}, \texttt{Lambda_clip}]$;
- the penalty parameter `rho_alm` is increased by a factor `eta_rho` every `h_update_rho` epochs, up to a maximum value;
- detailed logging of all loss components and schedules is performed periodically;
- if running on GPU, the CUDA cache is occasionally cleared to control memory usage.

4.7.5 Evaluation and Metrics for a Single Seed

```python
    # ===== EVALUATION ON A GRID =====
    x_eval = torch.linspace(config.x_min, config.x_max, 400).view(-1, 1).to(device)
    with torch.no_grad():
        rho_pred = netD(x_eval) + netB(x_eval)
        Z_pred = torch.trapz(rho_pred.view(-1), x_eval.view(-1))
        rho_pred_n = rho_pred / (Z_pred + 1e-8)
        rho_true = rho_exact(x_eval)

        diff = rho_pred_n - rho_true
        rel_L2 = torch.norm(diff) / torch.norm(rho_true)
        acc_L2 = 1.0 - rel_L2
        mae = torch.mean(torch.abs(diff))
        rmse = torch.sqrt(torch.mean(diff**2))

    # BC L2
    with torch.no_grad():
        rhoD_bc = netD(x_bc)
        rhoB_bc = netB(x_bc)
        rho_bc = rhoD_bc + rhoB_bc
        bc_l2 = torch.mean((rho_bc - g_bc) ** 2)

    # PDE residual L2 on the grid
    res_test, _, _, _ = fp_residual(netD, netB, x_eval)
    pde_l2 = torch.mean(res_test**2)

    training_time = time.time() - start_time

    metrics = {
        "RelL2": rel_L2.item(),
        "AccL2": acc_L2.item(),
        "MAE": mae.item(),
        "RMSE": rmse.item(),
        "BC_L2": bc_l2.item(),
        "PDE_L2": pde_l2.item(),
        "training_time": training_time,
        "params_D": params_D,
        "params_B": params_B,
        "total_params": total_params,
    }

    logger.info(f"\n[Seed {seed}] metrics: {metrics}")
    logger.info(f"[Seed {seed}] Training time: {training_time:.2f} seconds ({training_time/60:.2f} minutes)")

    # Memory cleanup
    if device.type == 'cuda':
        torch.cuda.empty_cache()

    return netD, netB, metrics
```

At the end of training for a single seed:

- the neural density is evaluated on a fine grid of 400 points and renormalized;
- the exact stationary density is computed on the same grid;
- several metrics are computed: relative L^2 error, L^2 accuracy, mean absolute error (MAE), root-mean-square error (RMSE), boundary loss in L^2 and mean squared PDE residual;
- the total training time and the number of parameters are recorded;
- all these quantities are stored in a dictionary `metrics`, logged, and returned together with the trained networks.

4.8 Multi-seed Loop and Statistics

4.8.1 Multi-seed Training Loop

```
# =============== MULTI-SEED LOOP ==========================
logger.info(f"\n{'='*60}")
logger.info(f"START MULTI-SEED TRAINING: {config.seeds}")
logger.info(f"{'='*60}\n")

total_start_time = time.time()

all_metrics = []

best_relL2 = float("inf")
best_seed = None
best_stateD = None
best_stateB = None
best_metrics_best = None

for seed in config.seeds:
    netD, netB, metrics = train_for_seed(seed)
    all_metrics.append(metrics)

    # update "best model"
    if metrics["RelL2"] < best_relL2:
        best_relL2 = metrics["RelL2"]
        best_seed = seed
        best_stateD = copy.deepcopy(netD.state_dict())
        best_stateB = copy.deepcopy(netB.state_dict())
        best_metrics_best = metrics.copy()
```

This outer loop:

- trains the Dual-PINN with the same configuration for each seed in `config.seeds`;
- collects the metrics for each seed in `all_metrics`;
- tracks the seed with the smallest relative L^2 error and saves the corresponding network states `best_stateD`, `best_stateB` and metrics.

4.8.2 Multi-seed Statistics

```
# =============== MULTI-SEED STATISTICS ===================
total_time = time.time() - total_start_time

keys = ["RelL2", "AccL2", "MAE", "RMSE", "BC_L2", "PDE_L2", "
    training_time"]
vals = {k: np.array([m[k] for m in all_metrics]) for k in keys}

logger.info("\n===== MULTI-SEED STATISTICS (mean +- std) =====")
for k in keys:
    mean = vals[k].mean()
    std = vals[k].std()
    logger.info(f"{k:12s}: {mean: .3e} +- {std: .3e}")

logger.info(f"\nTotal time: {total_time:.2f} seconds ({total_time
    /60:.2f} minutes)")
```

After all seeds have been trained:

- the total wall-clock time is computed;
- for each metric key, a NumPy array over seeds is formed;
- the mean and standard deviation across seeds are computed and logged, providing a statistical summary of the method's performance.

4.9 Best Model, Plotting, and Saving Results

4.9.1 Restoring and Evaluating the Best Model

```
# =============== BEST MODEL: RESTORE & PLOT ======
logger.info(
    f"\nBest seed: {best_seed} "
    f"(RelL2 = {best_relL2:.3e}, "
    f"Accuracy L2 = {best_metrics_best['AccL2']*100:.2f}%)"
)

# recreate networks and load the best weights
netD_best = KAN(list(config.layers_D)).to(device)
netB_best = KAN(list(config.layers_B)).to(device)
netD_best.load_state_dict(best_stateD)
netB_best.load_state_dict(best_stateB)

# final evaluation of the best model
x_plot = torch.linspace(config.x_min, config.x_max, 400).view(-1,
    1).to(device)
with torch.no_grad():
    rho_pred = netD_best(x_plot) + netB_best(x_plot)
    Z_pred = torch.trapz(rho_pred.view(-1), x_plot.view(-1))
    rho_pred_n = rho_pred / (Z_pred + 1e-8)
```

```
    rho_true = rho_exact(x_plot)

rel_L2_best = torch.norm(rho_pred_n - rho_true) / torch.norm(rho_true)
acc_L2_best = 1.0 - rel_L2_best

logger.info(f"\n[Best model] Relative L2 error = {rel_L2_best.item():.6e}")
logger.info(f"[Best model] Accuracy in L2 (%) = {100.0 * acc_L2_best.item():.4f}")

# Final memory cleanup
if device.type == 'cuda':
    torch.cuda.empty_cache()
    logger.info(f"GPU Memory allocated: {torch.cuda.memory_allocated()/1e9:.2f} GB")
```

Here:

- the seed with the best relative L^2 error is reported;
- new KAN networks `netD_best` and `netB_best` are instantiated and loaded with the saved best weights;
- the best model is re-evaluated on a fine grid, renormalized, and the final relative L^2 error and accuracy are logged;
- GPU memory is cleaned up at the end if needed.

4.9.2 Plot of Exact vs Dual-PINN KAN Solution

```
# =============== PLOT BEST MODEL ==========================
# Create directory for results
output_dir = Path("results")
output_dir.mkdir(exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
plot_filename = output_dir / f"fokker_planck_best_seed{best_seed}_{timestamp}.png"

x_np = x_plot.cpu().numpy()
rho_true_np = rho_true.cpu().numpy()
rho_pred_np = rho_pred_n.cpu().numpy()

plt.figure(figsize=(8, 5))
plt.plot(x_np, rho_true_np, label="rho_exact (two peaks)", linewidth=2)
plt.plot(
    x_np,
    rho_pred_np,
    "--",
    label=f"Dual-PINN KAN (seed {best_seed})",
```

```
    linewidth=2,
)
plt.xlabel("x")
plt.ylabel("rho(x)")
plt.title("Fokker-Planck 1D (double-well): best model (minimum
    RelL2)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(plot_filename, dpi=300, bbox_inches='tight')
logger.info(f"\nPlot saved in: {plot_filename}")
plt.close()
```

This block:

- creates a directory `results` if it does not exist;
- generates a filename with a timestamp and the best seed index;
- plots the exact double-peak density and the best neural approximation on the same grid;
- saves the figure as a high-resolution PNG file.

4.9.3 Saving JSON and TXT Reports

```
# =============== SAVE RESULTS TO JSON AND TXT FILES =======
results_json = {
    "timestamp": timestamp,
    "device": str(device),
    "config": {
        "max_epochs": config.max_epochs,
        "N_int": config.N_int,
        "lr": config.lr,
        "layers_D": list(config.layers_D),
        "layers_B": list(config.layers_B),
        "phase0_epochs": config.phase0_epochs,
        "phase1_epochs": config.phase1_epochs,
        "phase2_epochs": config.phase2_epochs,
        "grad_clip_max_norm": config.grad_clip_max_norm,
        "seeds": list(config.seeds),
    },
    "best_model": {
        "seed": best_seed,
        "metrics": best_metrics_best,
    },
    "all_seeds_metrics": all_metrics,
    "statistics": {
        k: {
            "mean": float(vals[k].mean()),
            "std": float(vals[k].std()),
            "min": float(vals[k].min()),
```

```
            "max": float(vals[k].max()),
        }
        for k in keys
    },
    "total_training_time_seconds": total_time,
    "total_training_time_minutes": total_time / 60.0,
}

json_filename = output_dir / f"results_{timestamp}.json"
with open(json_filename, 'w') as f:
    json.dump(results_json, f, indent=2)
logger.info(f"JSON results saved in: {json_filename}")
```

A structured JSON file is built containing:

- device and timestamp;
- the main configuration parameters used in the experiment;
- the best seed and its metrics;
- the per-seed metrics and their aggregate statistics (mean, standard deviation, min, max);
- the total training time.

This JSON file is convenient for programmatic post-processing.

```
# TXT file (readable format)
txt_filename = output_dir / f"results_{timestamp}.txt"
with open(txt_filename, 'w') as f:
    f.write("=" * 70 + "\n")
    f.write("DUAL-PINN KAN TRAINING RESULTS - FOKKER-PLANCK 1D\n")
    f.write("=" * 70 + "\n\n")
    f.write(f"Timestamp: {timestamp}\n")
    f.write(f"Device: {device}\n\n")

    f.write("CONFIGURATION:\n")
    f.write("-" * 70 + "\n")
    f.write(f"  Total epochs: {config.max_epochs}\n")
    f.write(f"  Phase 0 (BC pre-training): {config.phase0_epochs}\n")
    f.write(f"  Phase 1 (uniform sampling): {config.phase1_epochs}\n")
    f.write(f"  Phase 2 (adaptive sampling): {config.phase2_epochs}\n")
    f.write(f"  Interior points: {config.N_int}\n")
    f.write(f"  Learning rate: {config.lr}\n")
    f.write(f"  Gradient clipping: {config.grad_clip_max_norm}\n")
    f.write(f"  netD architecture: {config.layers_D}\n")
    f.write(f"  netB architecture: {config.layers_B}\n")
```

```
    f.write(f"  Total parameters: {best_metrics_best['total_params']}\n")
    f.write(f"    - netD: {best_metrics_best['params_D']}\n")
    f.write(f"    - netB: {best_metrics_best['params_B']}\n")
    f.write(f"  Seeds: {config.seeds}\n\n")

    f.write("BEST MODEL:\n")
    f.write("-" * 70 + "\n")
    f.write(f"  Seed: {best_seed}\n")
    f.write(f"  Relative L2 Error: {best_metrics_best['RelL2']:.6e}\n")
    f.write(f"  Accuracy L2 (%): {best_metrics_best['AccL2']*100:.4f}%\n")
    f.write(f"  MAE: {best_metrics_best['MAE']:.6e}\n")
    f.write(f"  RMSE: {best_metrics_best['RMSE']:.6e}\n")
    f.write(f"  BC L2: {best_metrics_best['BC_L2']:.6e}\n")
    f.write(f"  PDE L2: {best_metrics_best['PDE_L2']:.6e}\n")
    f.write(f"  Training time: {best_metrics_best['training_time']:.2f}s ({best_metrics_best['training_time']/60:.2f}min)\n\n")

    f.write("MULTI-SEED STATISTICS (mean +- std):\n")
    f.write("-" * 70 + "\n")
    for k in keys:
        mean = vals[k].mean()
        std = vals[k].std()
        min_val = vals[k].min()
        max_val = vals[k].max()
        f.write(f"  {k:15s}: {mean:.6e} +- {std:.6e}  [min: {min_val:.6e}, max: {max_val:.6e}]\n")

    f.write(f"\nTIMING:\n")
    f.write("-" * 70 + "\n")
    f.write(f"  Total time: {total_time:.2f}s ({total_time/60:.2f}min)\n")
    f.write(f"  Average time per seed: {vals['training_time'].mean():.2f}s\n\n")

    f.write("RESULTS PER SEED:\n")
    f.write("-" * 70 + "\n")
    for i, (seed, metrics) in enumerate(zip(config.seeds, all_metrics)):
        f.write(f"\nSeed {seed}:\n")
        f.write(f"  RelL2: {metrics['RelL2']:.6e}, AccL2: {metrics['AccL2']*100:.4f}%\n")
        f.write(f"  MAE: {metrics['MAE']:.6e}, RMSE: {metrics['RMSE']:.6e}\n")
        f.write(f"  BC_L2: {metrics['BC_L2']:.6e}, PDE_L2: {metrics['PDE_L2']:.6e}\n")
        f.write(f"  Time: {metrics['training_time']:.2f}s\n")

```

```
    f.write("\n" + "=" * 70 + "\n")

logger.info(f"TXT results saved in: {txt_filename}")
logger.info(f"\n{'='*60}")
logger.info(f"COMPLETED! All files saved in: {output_dir}/")
logger.info(f"{'='*60}")
```

The human-readable TXT report summarizes:

- the configuration of the experiment;
- the metrics of the best model;
- the multi-seed statistics (mean, standard deviation, min, max) for each metric;
- the total time and the per-seed training times;
- the detailed metrics for each individual seed.

This is useful for direct inclusion in appendices or for quick inspection outside of Python notebooks.

—

4.10 Numerical Results: Stationary Fokker–Planck 1D (Double-Well)

This section reports the numerical performance of the proposed Dual-PINN KAN approach (implemented with `efficient-kan` [6]) on the stationary one-dimensional Fokker–Planck equation with a double-well drift. The experiment was run for multiple random seeds, selecting the best model according to the minimum relative L^2 error, and reporting aggregate statistics across seeds.

4.10.1 Experimental Setup

The training protocol consists of three stages:

1) **Phase 0:** boundary-condition pre-training (100 epochs);
2) **Phase 1:** uniform interior collocation (150 epochs);
3) **Phase 2:** residual-based adaptive collocation (950 epochs).

The total budget is 1200 epochs, with $N_{\text{int}} = 128$ interior points per epoch, learning rate 10^{-3}, and gradient clipping with max norm 1.0. The Dual-KAN decomposition uses two networks with architecture $(1, 2, 2, 1)$, totaling 160 parameters (80 per subnet). Computations were performed on `cpu`.(The total number of trainable parameters is computed and logged by the script).

4.10.2 Best Model and Qualitative Comparison

Figure 4.1 compares the exact stationary density (double-peak profile) with the best-performing Dual-PINN KAN approximation. The best model corresponds to seed 48 (selected by minimum relative L^2 error).

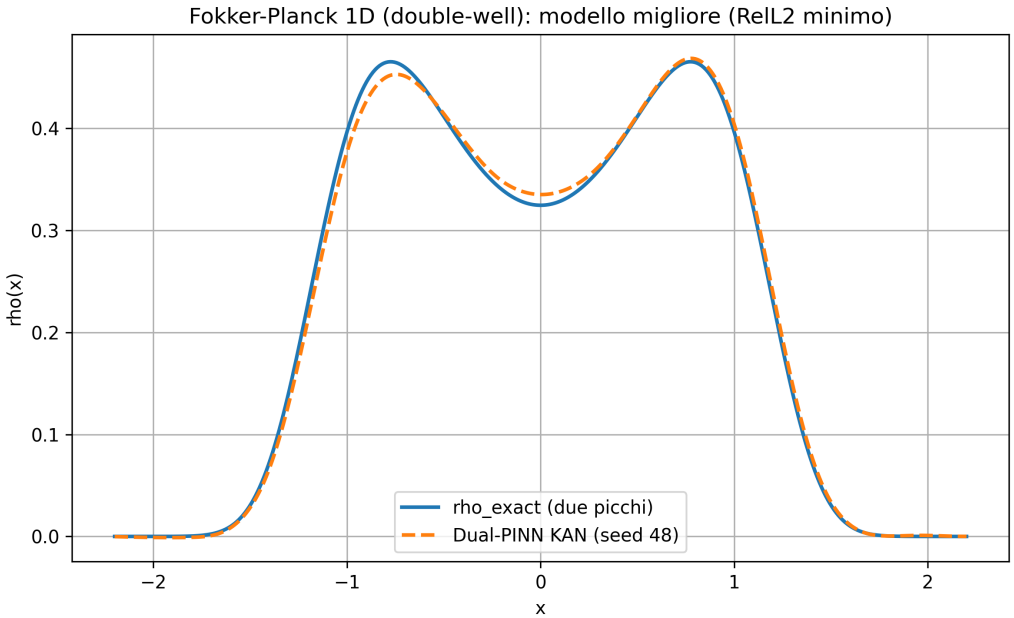


Figure 4.1: Fokker–Planck 1D (double-well): exact stationary density versus the best Dual-PINN KAN model (seed 48, minimum RelL2).

4.10.3 Quantitative Metrics (Best Seed)

Let ρ_{pred} denote the neural approximation (renormalized on the grid) and ρ_{exact} the exact stationary density.

We adopt the same convention as in the reference article[5], where $\text{AccL2} = 1 - \text{RelL2}$ is reported as an L^2-based accuracy score.

We report:

$$\text{RelL2} = \frac{\|\rho_{\text{pred}} - \rho_{\text{exact}}\|_2}{\|\rho_{\text{exact}}\|_2}, \qquad \text{AccL2} = 1 - \text{RelL2},$$

together with MAE, RMSE, boundary-condition error (BC_L2), and mean squared PDE residual (PDE_L2).

Metric	**Best model (seed 48)**
Relative L^2 error (RelL2)	2.704810×10^{-2}
L^2 accuracy (AccL2)	97.2952%
MAE	5.726786×10^{-3}
RMSE	7.896746×10^{-3}
BC_L2	1.708685×10^{-8}
PDE_L2	2.614324×10^{-4}
Training time	18.89s

Table 4.1: Performance of the best Dual-PINN KAN model (selected by minimum RelL2).

4.10.4 Multi-seed Performance and Robustness

To assess robustness, the same configuration was trained over five seeds $(42, 44, 46, 48, 50)$. Table 4.2 reports mean $\pm$ standard deviation, together with the observed min/max across seeds.

Metric	**Mean $\pm$ Std**	**[Min, Max]**
RelL2	$7.530219 \times 10^{-2} \pm 4.782946 \times 10^{-2}$	$[2.704810 \times 10^{-2}, 1.605492 \times 10^{-1}]$
AccL2	$9.246978 \times 10^{-1} \pm 4.782946 \times 10^{-2}$	$[8.394508 \times 10^{-1}, 9.729519 \times 10^{-1}]$
MAE	$1.648014 \times 10^{-2} \pm 1.057943 \times 10^{-2}$	$[5.726786 \times 10^{-3}, 3.526905 \times 10^{-2}]$
RMSE	$2.198462 \times 10^{-2} \pm 1.396390 \times 10^{-2}$	$[7.896746 \times 10^{-3}, 4.687264 \times 10^{-2}]$
BC_L2	$2.864734 \times 10^{-8} \pm 3.397316 \times 10^{-8}$	$[2.778100 \times 10^{-9}, 9.584905 \times 10^{-8}]$
PDE_L2	$2.242991 \times 10^{-4} \pm 2.657185 \times 10^{-5}$	$[1.861533 \times 10^{-4}, 2.614324 \times 10^{-4}]$
Training time	$1.891950 \times 10^{1} \pm 1.479152 \times 10^{-1}$ s	[18.75s, 19.13s]

Table 4.2: Multi-seed statistics over five runs: mean $\pm$ standard deviation, and [min, max] across seeds.

Bibliography

[1] A. N. Kolmogorov, *On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition*, Doklady Akademii Nauk SSSR, vol. 114, no. 5, pp. 953–956, 1957.

[2] A. N. Kolmogorov, *On the representation of continuous functions of several variables as superpositions of continuous functions of one variable and addition*, in *Sixteen Papers on Analysis*, Translations of the American Mathematical Society, Series 2, vol. 28, American Mathematical Society, Providence, RI, 1963.

[3] A. G. Vitushkin and G. M. Khenkin, *Linear Superpositions of Functions*, Russian Mathematical Surveys, vol. 22, no. 1, p. 77, 1967. DOI: 10.1070/RM1967v022n01ABEH001204

[4] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark, *KAN: Kolmogorov-Arnold Networks*, arXiv preprint arXiv:2404.19756, 2025.

[5] N. Abbas, V. Colao, D. Macri, W. Spataro, *A Multi-Phase Dual-PINN Framework: Soft Boundary-Interior Specialization via Distance-Weighted Priors*, arXiv preprint arXiv:2511.23409, 2025.

[6] B. LeBleu, *efficient-KAN: An Efficient Implementation of Kolmogorov-Arnold Networks*, GitHub repository, `https://github.com/Blealtan/efficient-kan`, 2024.

[7] C. de Boor, *A Practical Guide to Splines*, Vol. 27, Springer-Verlag, New York, 1978.

[8] U. G. S. Campolo, *KAN Dual-PINN Solver for the 1D Fokker–Planck Equation*, GitHub repository, `https://github.com/hugo9777/kan-pde-thesis`, 2026.

[9] N. Dunford and J. T. Schwartz, *Linear Operators. Part I: General Theory*, Interscience Publishers, New York–London, 1958.